Virtual Machines

One machine, built up chapter by chapter

Set Lonnert

Virtual Machines: one machine, built up chapter by chapter
by Set Lonnert

The screen edition. The code in this book is taken from the repository `https://github.com/Feyerabend/vm`, at commit `8fadcf3`, where this book is also found, in `book/`.

This book, like the code it is made from, is free and unencumbered, released into the public domain under the Unlicense: anyone may copy, print, share, change, translate, sell or give it away, in any form and for any purpose, without asking.

Contents

Preface **xi**

Introduction **1**

I The machine **7**

1 A stack machine **9**
1.1 The instructions 10
1.2 The code 10
1.3 Run it 12
1.4 What is missing, on purpose 13
1.5 Exercises 13

2 Jumps **15**
2.1 Juggling the stack 16
2.2 The instructions so far 17
2.3 The difference 17
2.4 Run it 19
2.5 What is still missing 19
2.6 Exercises 19

3 Calls and frames **21**
3.1 What CALL and RET do 21
3.2 Recursion for free 23
3.3 New instructions 24
3.4 Run it 24

3.5 Exercises 24

4 Traps **27**
4.1 What is checked 28
4.2 The cost 32
4.3 New instructions 32
4.4 Run it 32
4.5 Exercises 33

5 Programs in files **35**
5.1 The pieces 35
5.2 The format 36
5.3 Two kinds of failure 37
5.4 Not yet an assembler 38
5.5 Run it 39
5.6 Exercises 39

II Tools **41**

6 An assembler **43**
6.1 Two passes 45
6.2 Checks 46
6.3 One table, two tools 47
6.4 The disassembler 48
6.5 The listing 48
6.6 Run it 49
6.7 Exercises 49

7 Watching the machine run **51**
7.1 Splitting the loop 51
7.2 The trace 52
7.3 The debugger 53
7.4 Replaying a trace 54
7.5 Tests 54
7.6 Exercises 54

8 A specification **57**
8.1 What writing it down decided . . . 57
8.2 Two machines . . . 58
8.3 The conformance suite . . . 59
8.4 Comparing traces . . . 60
8.5 Run it . . . 60
8.6 Exercises . . . 60

III Languages on the machine **61**

9 A compiler for expressions **63**
9.1 Stage 1: scanning . . . 64
9.2 Stage 2: parsing . . . 64
9.3 Stage 3: generating code . . . 66
9.4 The whole pipeline . . . 66
9.5 Errors . . . 67
9.6 Testing with random programs . . . 67
9.7 Exercises . . . 67

10 enkel/0 **69**
10.1 Four stages . . . 70
10.2 The machine grows: specification version 2 . . . 72
10.3 What the code looks like . . . 72
10.4 rval . . . 73
10.5 A bug, thirty lines in . . . 73
10.6 Tests . . . 73
10.7 Exercises . . . 74

11 basal/0 **75**
11.1 The language . . . 76
11.2 No syntax tree . . . 76
11.3 Syntactic sugar . . . 77
11.4 A run-time library . . . 78
11.5 GOSUB on a machine with frames . . . 79
11.6 Tests . . . 80
11.7 Exercises . . . 80

12 Three ways to parse **81**
12.1 The three parsers 82
12.2 Proving they agree 83
12.3 Where they differ: error messages 84
12.4 Speed 84
12.5 Which to use? 84
12.6 Run it 85
12.7 Exercises 85

IV The run-time **87**

13 A heap **89**
13.1 Four instructions 90
13.2 Values that know what they are 90
13.3 Seeing references 91
13.4 Specification version 3 91
13.5 Run it 93
13.6 Exercises 93

14 Garbage collection **95**
14.1 Mark and sweep 96
14.2 Copying 97
14.3 Which is better? 99
14.4 Testing a collector 99
14.5 Invisible, and proved so 100
14.6 Run it 100
14.7 Exercises 100

15 Closures **103**
15.1 The language: lambda/0 104
15.2 What a closure is 104
15.3 Closure conversion 105
15.4 One new instruction 107
15.5 Safety, from the tags 107
15.6 Garbage, at last 108
15.7 Data from nothing but functions 108

15.8 A limit: 256 frames 109
15.9 Run it 109
15.10Exercises 109

V Further **111**

16 Dispatch **113**
16.1 One set of instructions, three loops 114
16.2 They are the same machine 116
16.3 How much faster? 116
16.4 Run it 117
16.5 Exercises 117

17 Registers **119**
17.1 From stack to registers 120
17.2 What is refused 121
17.3 The register machine 122
17.4 Measured 122
17.5 Run it 123
17.6 Exercises 123

18 Optimising **125**
18.1 A peephole optimiser 125
18.1.1 Changing nothing, including the failures 126
18.2 Superinstructions 127
18.3 Measured 128
18.4 Run it 129
18.5 Exercises 130

19 Verify once, then trust **131**
19.1 The verifier 131
19.2 Two machines from one 132
19.3 What is still checked 134
19.4 Is the verifier right? 134
19.5 Measured 135
19.6 Run it 135

19.7 Exercises 136

20 Native code **137**
20.1 From instructions to C 137
20.2 The same machine, proved again 138
20.3 Verified programs need fewer checks 139
20.4 Measured 139
20.5 What a JIT would add 140
20.6 Run it 140
20.7 Exercises 140

21 The machine in a browser **141**
21.1 A machine with no operating system 142
21.2 Two hosts 142
21.3 The fourth machine 145
21.4 The playground 145
21.5 Run it 145
21.6 Exercises 146

22 The machine as hardware **147**
22.1 A CPU in one page 148
22.2 Talking to the outside 149
22.3 The fifth machine 149
22.4 The end of the road 150
22.5 Run it 150
22.6 Exercises 151

Appendices **153**

A The Metcalfe parsing machine **153**
A.1 The machine 153
A.2 Compared with ours 154
A.3 The limits of undoing 155
A.4 Run it 155
A.5 Exercises 156

B p-code, Pascal's machine **157**
B.1 The P-machine 158
B.2 Levels and static links 158
B.3 Compared with ours 159
B.4 Run it 159
B.5 P4: the machine Pascal travelled on 160
B.6 pascal/0 161
B.7 Run it 162
B.8 Exercises 162

C CHIP-8 **165**
C.1 The machine 165
C.2 Drawing by XOR 166
C.3 Compared with ours 167
C.4 Run it 167
C.5 Exercises 167

D SWEET16 **169**
D.1 The machine 169
D.2 The prior result 170
D.3 Compared with ours 171
D.4 Run it 171
D.5 Exercises 172

E PostScript **173**
E.1 The machine 174
E.2 Painting 175
E.3 Compared with ours 175
E.4 Run it 176
E.5 Exercises 176

F The Java Virtual Machine **179**
F.1 The machine 179
F.2 Compared with ours 180
F.3 Run it 181
F.4 Exercises 182

G The instruction set **183**
G.1 Traps 185
G.2 The specification 185

H The file format and the specification **187**
H.1 The .vm file format, version 1 187
H.1.1 Why each part is there 188
H.1.2 What is not here 188
H.2 The machine: specification, version 4 188
H.2.1 1. Values 189
H.2.2 2. The state of the machine 189
H.2.3 2a. The heap 190
H.2.4 3. Running 190
H.2.5 4. Basic operations 191
H.2.6 5. Instructions 191
H.2.7 6. Traps 194
H.2.8 7. Program files 194
H.2.9 8. Tracing 194
H.2.10 9. What is left unspecified 195

Preface

Computing is a game of layers. We build abstractions to keep from drowning in detail, only to find that the layers themselves become a new kind of territory to map. This book is a map of one such layer: the virtual machine.

The traditional path to understanding how software meets hardware usually requires a long detour through assembly language, compiler theory, and processor architecture. These are often taught as separate, monumental subjects. The goal of this book is to bring them together into a single, cohesive narrative by building one working system from scratch, one clear concept at a time.

We begin with the absolute minimum: a forty-line loop in C that manipulates a stack of numbers. From that modest foundation, we add jumps for control flow, frames for recursive functions, and structural defenses to catch errors before they escape. We then step outside the core runtime to construct the tooling: an assembler, a debugger, and high-level languages like enkel/0 and basal/0. Eventually, we explore advanced runtime mechanics, comparing garbage collection strategies side by side, before pushing the machine to its absolute limits: compiling to native code, running inside a web browser, and finally translating our software design into hardware logic.

By looking at the complete implementation at every milestone, the illusion of complexity vanishes. You will see that a virtual machine is not an impenetrable black box, but a sequence of small, deliberate design choices.

This text is designed for the curious programmer, the systems enthusiast, and anyone who has ever looked at a language runtime

and wondered, *how does it actually work?* The best way to read this book is with a code editor open, a terminal ready, and a willingness to break things just to see how they break.

The machine is simple enough to understand completely, yet deep enough to carry you from a basic loop all the way to bare metal. I invite you to turn the page, compile the first chapter, and begin building.

Introduction: the power of abstraction

What a virtual machine is, where the idea came from, and how this book builds one: a chapter at a time, each one the last plus one idea.

My own journey into virtual machines began with UCSD Pascal on the Apple II around 1980. The *p-machine* it ran on was a stack machine much like the one in chapter 1, and it was why the same Pascal program could run on an Apple, a PDP-11 or an IBM PC.

The trick is worth stopping at, because the whole book follows from it. The Pascal compiler did not translate programs for the Apple's 6502, nor for the PDP-11, nor for the IBM PC's 8088. It translated them for a machine that did not exist: the p-machine, a simple computer with a stack, described on paper. On each real computer, a small program, the *interpreter*, pretended to be that machine. Write the interpreter once for a new computer, a few pages of code, and every Pascal program, the compiler included, ran there unchanged.

One idea, many names

The words *virtual machine*, *interpreter* and *emulator* overlap a great deal, and it is not worth getting stuck on the definitions. What matters is the abstraction: a layer that hides the machine below from

the programs above. The Apollo Guidance Computer had one, Busicom's calculator code on the Intel 4004 was one, and Steve Wozniak's SWEET16 was one.

Still, the three words lean in different directions, and it helps to know which way:

- An *emulator* imitates a machine that exists, or existed, in hardware, closely enough to run its programs: an Apple II on a laptop.
- An *interpreter* runs a program by reading it and doing what it says, step by step, without first turning it into the code of the computer underneath.
- A *virtual machine* is a machine designed to be run in software from the start: its instructions are chosen to be easy to interpret, and easy for a compiler to produce. The p-machine is one, and so are the Java Virtual Machine and WebAssembly.

The machine of this book is a virtual machine in that last sense, and it is run by an interpreter. By chapter 22 it is also, for a moment, hardware.

One machine, built up chapter by chapter

This book builds a single virtual machine, and keeps building it. Each chapter is a folder in the repository with a complete, runnable machine that is the previous chapter plus one idea. It starts as simple as a virtual machine can be, forty lines of C. It does not stay there.

Part I, The machine. From a loop and a stack to jumps, calls with frames, errors that stop the machine with a message, and a file format for programs.

Part II, Tools. An assembler, so programs are no longer written as numbers; a tracer and a debugger, to watch the machine run; and a written specification, with a second machine in Python that must pass the same tests.

Part III, Languages on the machine. Compilers: first for arithmetic, then for enkel/0, a small Pascal-like language, and basal/0, a small BASIC; and other ways to parse.

Part IV, The run-time. A heap for strings, arrays and records; two garbage collectors, compared side by side; and closures.

Part V, Further. Making the loop fast, a register machine, optimisation, a verifier that checks programs before they run, compiling to native code, the machine in a web browser, and the machine as hardware.

The appendices are studies of machines that are not ours, each a small runnable version in Python: a parsing machine, Pascal's p-code, CHIP-8, Wozniak's SWEET16, PostScript, and the Java Virtual Machine. Each one made some decision differently from our machine, and seeing the other choice is the point.

How to read this book

Read the chapters in order, and compare each machine with the one before. In the repository, `diff ch02-jumps/vm.c ch03-calls/vm.c` is a whole chapter: the difference *is* the lesson. The book shows that difference as it is, with the lines that stay in gray and the new ones in black.

Every chapter has the same shape. It opens with its goal: what the machine can do at the end of it that it could not at the start. Then comes the idea, then the code, then how to run it. Each chapter also says what it leaves out, *on purpose*, and which later chapter takes it up; and it ends with exercises. The exercises are not tests. They are the next things to try with the machine in front of you, and most of them have more than one good answer.

This is a book to do, more than to read. The machines are small enough to read in one sitting, and meant to be changed: add an instruction, break something and see what happens, then read on and see how the next chapter mends it.

Other roads to the same place

There are well-trodden ways into this territory, and it is worth saying how this book stands among them, since it borrows from several and

differs from all.

The classic way is the textbook of compilers, from the "Dragon Book" of Aho, Sethi and Ullman (1986) onwards: each stage of translation in depth, with its theory, one stage at a time. It is thorough, but the stages are studied apart, and the machine that finally runs the code is somebody else's. Books on virtual machines as such, like Smith and Nair's *Virtual Machines* (2005), survey the whole field, from interpreters to hypervisors, and build none of it. This book goes the other way: one small machine, built and run, with the theory brought in only when the machine needs it.

The nearest relative is *The Elements of Computing Systems* by Nisan and Schocken (2005), known as Nand2Tetris. It also builds one system in many small steps, and it too has a stack-based virtual machine at its centre. But it runs the other way: it begins with a single logic gate and climbs up through hardware, assembler, virtual machine and compiler to an operating system. This book begins with the software, a loop in C, and climbs *down*, ending where Nand2Tetris begins, with the machine as a circuit.

Robert Nystrom's *Crafting Interpreters* (2021) is another close relative: its second half builds, in C, a bytecode virtual machine with a stack, closures and a garbage collector, much as Parts I and IV do here, and it shows its code as it grows. It follows one language from beginning to end, where this book puts several languages, and several kinds of compiler, on one machine.

Further back, Niklaus Wirth showed in *Algorithms + Data Structures = Programs* (1976) and *Compiler Construction* (1996) that a complete compiler and its machine can be small enough to read whole, which is the spirit of every chapter here; appendix B builds two machines from his group in Zurich. Donald Knuth wrote *The Art of Computer Programming* for MIX, a computer that never existed, so that its programs would not be tied to any one real computer: an early argument for a virtual machine. And *Structure and Interpretation of Computer Programs* (1985) ends by turning its interpreter into a register machine with a garbage collector, as chapters 14 and 17 do.

What this book adds to these is perhaps three habits. The first is

the one already described: every chapter is a whole machine, and the difference from the last is the lesson. The second is that the machine is *written down*: from chapter 8 on, a specification says what it does, and one set of tests holds five quite different implementations to it, in C, in Python, as native code, in a web browser and as a circuit. The third is measuring: when a chapter makes the machine faster, it says by how much, on which compiler, and also when the answer was disappointing.

The code

The code is in the repository `https://github.com/Feyerabend/vm`. Nothing in this book is typed in by hand: every listing, every difference between two chapters, and every line a program prints is taken from the repository when the book is made. This copy was made from commit `8fadcf3`.

```
make test        # build and test every chapter, and the appendix
```

All you need is a C compiler, `make` and, from chapter 5 onward, Python 3. The last two chapters need a little more: chapter 21 needs `clang` with `wasm-ld` and Node.js, and chapter 22 needs Icarus Verilog. The appendices need only Python; if Free Pascal or Java is installed, the studies of p-code and of the Java Virtual Machine also check themselves against the real thing. Nothing is downloaded while building.

These machines are for reading, not for production. Things a real implementation must do are left out when they would hide the idea a chapter is about, and each chapter says what it leaves out.

Where the chapters came from

The repository began as a set of nineteen loosely connected experiments, which are kept unchanged in its `archive/` folder; their ideas were carried into the chapters of this book. The basic code of the earliest machines was inspired by Bartosz Sypytkowski's *simple virtual machine*.

Part I

The machine

Chapter 1

A stack machine

A machine that runs a program: a loop, a program counter and a stack of numbers, in less than a hundred lines of C.

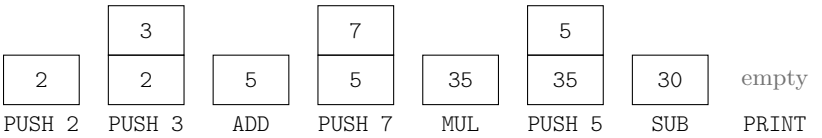
A *virtual machine*, at its smallest, is a loop:

1. read the instruction at the *program counter* `pc`,
2. move `pc` past it,
3. do what the instruction says,
4. repeat until told to stop.

The machine in `vm.c` keeps its working values on a *stack*. `PUSH 2` puts a 2 on top. `ADD` takes the two top values off, adds them, and puts the result back. Nothing is named; everything is "the value on top" or "the one just under it".

```
PUSH 2      stack: 2
PUSH 3      stack: 2 3
ADD         stack: 5
PUSH 7      stack: 5 7
MUL         stack: 35
PUSH 5      stack: 35 5
SUB         stack: 30
PRINT       prints 30, stack is empty
```

Drawn, with the top of the stack at the top, and one picture after each instruction:



This is the same order of operations as *Reverse Polish Notation* on an HP calculator, and as the language FORTH: operands first, then the operation. $(2 + 3) \times 7 - 5$ needs no parentheses once it is written as `2 3 + 7 * 5 -`.

1.1 The instructions

Code	Name	Operand	Effect
0	`HALT`		stop
1	`PUSH`	`n`	push `n`
2	`ADD`		pop `b`, pop `a`, push `a + b`
3	`SUB`		pop `b`, pop `a`, push `a - b`
4	`MUL`		pop `b`, pop `a`, push `a * b`
5	`PRINT`		pop `a`, print it

1.2 The code

This is the whole machine. `run` is the loop: it fetches the instruction at `pc`, moves `pc` on, and the `switch` does what the instruction says. The program is the array `program`, written into the C source.

ch01-stack/vm.c

```
/*
 * Chapter 1: a stack machine.
 *
 * The whole machine is three things: an array of code, a program
 * counter pointing into it, and a stack of integers. Each step
   ↪ reads
 * one instruction at pc and does what it says. That is all.
 *
 * The program is written directly into the C source below.
   ↪ Changing
```

```
 * the program means recompiling the machine, which we fix in
   ↪ chapter 5.
 */

#include <stdio.h>

enum {
    HALT,       /* stop the machine                          */
    PUSH,       /* PUSH n: push the number n                 */
    ADD,        /* pop b, pop a, push a + b                  */
    SUB,        /* pop b, pop a, push a - b                  */
    MUL,        /* pop b, pop a, push a * b                  */
    PRINT       /* pop a, print it on a line of its own      */
};

#define STACK_SIZE 256

static int stack[STACK_SIZE];
static int sp = 0;              /* number of values on the stack */

static void push(int v) {
    stack[sp++] = v;
}

static int pop(void) {
    return stack[--sp];
}

static void run(const int *code) {
    int pc = 0;

    for (;;) {
        int op = code[pc++];
        int a, b;

        switch (op) {
        case HALT:
            return;
        case PUSH:
            push(code[pc++]);
            break;
        case ADD:
            b = pop();
            a = pop();
            push(a + b);
            break;
        case SUB:
            b = pop();
            a = pop();
```

```
57              push(a - b);
58              break;
59          case MUL:
60              b = pop();
61              a = pop();
62              push(a * b);
63              break;
64          case PRINT:
65              printf("%d\n", pop());
66              break;
67          }
68      }
69  }
70
71  /* (2 + 3) * 7 - 5, then 6 * 7 */
72  static const int program[] = {
73      PUSH, 2,
74      PUSH, 3,
75      ADD,
76      PUSH, 7,
77      MUL,
78      PUSH, 5,
79      SUB,
80      PRINT,
81      PUSH, 6,
82      PUSH, 7,
83      MUL,
84      PRINT,
85      HALT
86  };
87
88  int main(void) {
89      run(program);
90      return 0;
91  }
```

1.3 Run it

```
make test
```

The program computes $(2+3)\times 7-5$, and then 6×7, and prints

```
30
42
```

1.4 What is missing, on purpose

- Nothing checks the stack. Pop an empty stack and the machine reads garbage; push too much and it writes past the array. Chapter 4 turns these into proper errors.
- The program lives inside the C source. Chapter 5 gives programs a file format of their own.
- There is no way to repeat anything or make a decision. That is the next chapter.

1.5 Exercises

1. Add `DIV`. What should happen on division by zero? (Write down your answer; chapter 4 gives one.)
2. Rewrite the program to compute $1+2+3+4+5$ in two different instruction orders. Which one needs the deeper stack?
3. What is the largest stack depth the given program ever reaches?

Chapter 2

Jumps

The machine can repeat and choose: two jumps make every loop and every `if`, and five instructions move values about on the stack.

Chapter 1's machine ran each instruction once, top to bottom. To repeat something, or to choose between two things, the machine has to be able to put a different number in `pc`.

- `JMP addr` always continues at `addr`.
- `JZ addr` pops a value and continues at `addr` only if it was zero; otherwise it carries on with the next instruction.

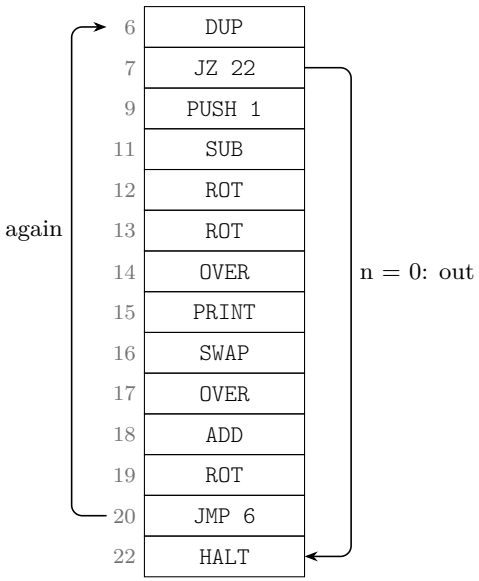
With these two every loop and every `if` can be built. A `while` loop is a `JZ` out of the loop at the top and a `JMP` back at the bottom; that is the shape of the program in `vm.c`, which prints the first ten Fibonacci numbers:

ch02-jumps/vm.c, lines 139–153

```
139 static const int program[] = {
140     /*  0 */ PUSH, 0,           /* a                        */
141     /*  2 */ PUSH, 1,           /* b                        */
142     /*  4 */ PUSH, 10,          /* n                        */
143     /* loop: */
144     /*  6 */ DUP, JZ, 22,       /* a b n, leave if n = 0 */
145     /*  9 */ PUSH, 1, SUB,      /* a b n-1                  */
146     /* 12 */ ROT, ROT,          /* n a b                    */
147     /* 14 */ OVER, PRINT,       /* print a                  */
```

```
148     /* 16 */ SWAP, OVER, ADD,    /* n b (a + b)           */
149     /* 19 */ ROT,                /* b (a + b) n           */
150     /* 20 */ JMP, 6,
151     /* end: */
152     /* 22 */ HALT
153 };
```

Leave out the operands and the comments, and the shape is easy to see. `JMP 6` goes back to the top of the loop every time; `JZ 22` leaves it, once, when the count has reached zero:



2.1 Juggling the stack

Values on a stack have no names, so to use one twice, or to reach one that is not on top, the machine needs instructions that only rearrange. These come straight from FORTH:

Name	Before	After
DUP	a	a a
DROP	a	
SWAP	a b	b a
OVER	a b	a b a
ROT	a b c	b c a

LT and EQ compare and leave 1 (true) or 0 (false), which is exactly what JZ wants to see.

2.2 The instructions so far

Code	Name	Operand	Effect
0	HALT		stop
1	PUSH	n	push n
2	ADD		a b → a + b
3	SUB		a b → a - b
4	MUL		a b → a * b
5	PRINT		a → (print a)
6	JMP	addr	pc = addr
7	JZ	addr	a → ; if a == 0 then pc = addr
8	DUP		a → a a
9	DROP		a →
10	SWAP		a b → b a
11	OVER		a b → a b a
12	ROT		a b c → b c a
13	LT		a b → (a < b)
14	EQ		a b → (a == b)

Codes 0 to 5 are the same as in chapter 1. A program written for chapter 1 still runs here; the machine has grown, not changed.

2.3 The difference

This is what run gained: one case for each new instruction, and nothing else changed. JMP and JZ are the only ones that touch pc.

run in ch01-stack/vm.c → ch02-jumps/vm.c

```
@@ -30,4 +30,54 @@
             printf("%d\n", pop());
             break;
+        case JMP:
+            pc = code[pc];
+            break;
+        case JZ:
+            a = pop();
+            if (a == 0)
+                pc = code[pc];
+            else
+                pc++;
+            break;
+        case DUP:
+            a = pop();
+            push(a);
+            push(a);
+            break;
+        case DROP:
+            pop();
+            break;
+        case SWAP:
+            b = pop();
+            a = pop();
+            push(b);
+            push(a);
+            break;
+        case OVER:
+            b = pop();
+            a = pop();
+            push(a);
+            push(b);
+            push(a);
+            break;
+        case ROT: {
+            int c = pop();
+            b = pop();
+            a = pop();
+            push(b);
+            push(c);
+            push(a);
+            break;
+        }
+        case LT:
+            b = pop();
+            a = pop();
+            push(a < b);
+            break;
```

```
+         case EQ:
+             b = pop();
+             a = pop();
+             push(a == b);
+             break;
          }
      }
```

2.4 Run it

```
make test
```

```
0
1
1
2
3
5
8
13
21
34
```

2.5 What is still missing

The Fibonacci program works, but follow the stack comments and see how much of it is shuffling. With more than three values in play, juggling becomes the whole program. Chapter 3 gives values a place to stay: *frames*.

2.6 Exercises

1. Write a program that prints the numbers from 1 to 10.
2. Compute 7×6 using only `ADD` and a loop.
3. `JZ` jumps on false. Add `JNZ`, which jumps on true, and rewrite the loop with it. Is the program shorter?
4. Our machine has `LT` but not `GT`. Show that it does not need it.

Chapter 3

Calls and frames

Functions: every call gets values of its own, in a frame, and finds its way back; and a function can call itself.

In chapter 2 every value lived on one shared stack, and keeping track of them was the program's whole job. A *function* changes that. When `fact` is called it needs:

1. **a way back**: the address just after the `CALL`,
2. **its own values**: an `n` that the caller's values, and the values of other calls to `fact`, cannot disturb,
3. **a way to hand back a result**.

The machine in `vm.c` keeps a second stack, of *frames*. A frame records two numbers: where to return to (`ret`), and where this call's values begin on the data stack (`fp`, the *frame pointer*).

ch03-calls/vm.c, lines 51–54

```
51 struct frame {
52     int ret;
53     int fp;
54 };
```

3.1 What CALL and RET do

`CALL addr n` is used after the caller has pushed `n` arguments:

```
data stack before:   ... x  y  | 5          (5 is the argument)
                                 ^ fp of the new call
```

1. push a frame holding the current `pc` and `fp`,
2. set `fp` to point at the first argument,
3. jump to `addr`.

Inside the function, `LOAD 0` reads the first argument, `LOAD 1` the second, and so on: the arguments *are* the function's first locals. `STORE k` writes one.

`RET` takes the value on top as the result, throws away everything the call left above `fp`, pops the frame to restore the caller's `pc` and `fp`, and pushes the result. To the caller, `CALL fact 1` behaves like one instruction that turned `n` into `fact(n)`.

run in ch02-jumps/vm.c → ch03-calls/vm.c

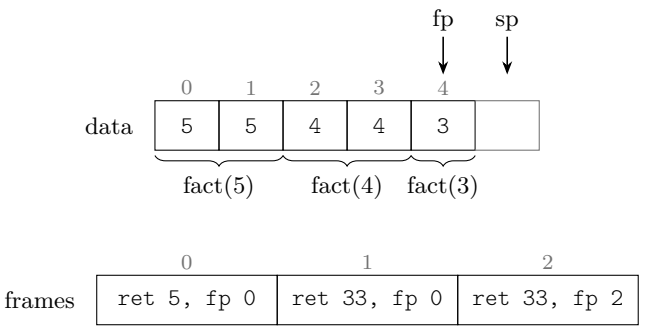
```
@@ -80,4 +80,28 @@
             push(a == b);
             break;
+        case LOAD:
+            push(stack[fp + code[pc++]]);
+            break;
+        case STORE:
+            stack[fp + code[pc++]] = pop();
+            break;
+        case CALL: {
+            int addr = code[pc++];
+            int nargs = code[pc++];
+            frames[fsp].ret = pc;
+            frames[fsp].fp = fp;
+            fsp++;
+            fp = sp - nargs;
+            pc = addr;
+            break;
+        }
+        case RET:
+            a = pop();
+            sp = fp;
+            fsp--;
+            pc = frames[fsp].ret;
+            fp = frames[fsp].fp;
+            push(a);
+            break;
         }
```

```
    }
```

3.2 Recursion for free

Because frames are on a stack, `fact(5)` calling `fact(4)` calling `fact(3)` … each has its own frame and its own `n`. Nothing in the machine knows or cares that it is the same function. This is the idea Pascal's and C's run-time stacks are built on, and it is the *activation record* of the old `vm3`.

Here is the machine inside `fact(3)`, which was called by `fact(4)`, which was called by `fact(5)`. Each call has its own `n` at the bottom of its frame, and the copy of `n` that its caller keeps for the multiplication afterwards. Each saved frame holds the caller's `fp`, and where to go back to: 5, in the main program, or 33, just after the `CALL` inside `fact`.



ch03-calls/vm.c, lines 186–211

```
186 static const int program[] = {
187     /* main: */
188     /*  0 */ PUSH, 5,
189     /*  2 */ CALL, 13, 1,       /* fact(5)                */
190     /*  5 */ PRINT,
191     /*  6 */ PUSH, 10,
192     /*  8 */ CALL, 13, 1,       /* fact(10)               */
193     /* 11 */ PRINT,
194     /* 12 */ HALT,
```

```
195
196     /* fact: */
197     /* 13 */ LOAD, 0,
198     /* 15 */ PUSH, 2,
199     /* 17 */ LT,
200     /* 18 */ JZ, 23,            /* n >= 2: go recurse    */
201     /* 20 */ PUSH, 1,
202     /* 22 */ RET,               /* return 1              */
203     /* recurse: */
204     /* 23 */ LOAD, 0,           /* n                     */
205     /* 25 */ LOAD, 0,
206     /* 27 */ PUSH, 1,
207     /* 29 */ SUB,               /* n - 1                 */
208     /* 30 */ CALL, 13, 1,       /* fact(n - 1)           */
209     /* 33 */ MUL,               /* n * fact(n - 1)       */
210     /* 34 */ RET
211 };
```

3.3 New instructions

Code	Name	Operand	Effect
15	LOAD	k	push local k
16	STORE	k	pop into local k
17	CALL	addr n	new frame; first n stack values are its locals
18	RET		pop result, drop frame, push result

Codes 0 to 14 are unchanged from chapter 2.

3.4 Run it

```
make test
```

```
120
3628800
```

3.5 Exercises

1. Write `square(x)` and use it to print `square(3) + square(4)`.

2. Write `gcd(a, b)` with two arguments, recursively: $\gcd(a, 0) = a$, $\gcd(a, b) = \gcd(b, a \bmod b)$. You will need a `MOD` instruction first.
3. A function with a local variable that is not an argument: the function can `PUSH 0` on entry to make room for it, and use it as local `n`. Rewrite factorial as a loop this way.
4. What happens with `fact(100000)`? Why? (Chapter 4.)

Chapter 4

Traps

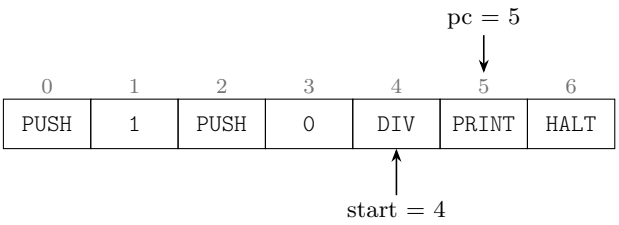
The machine never does whatever the C underneath happens to do: everything that can go wrong is checked, and stops the machine with a message that says what and where.

Until now, "what happens if…" had only one answer: *whatever the C underneath happens to do.* Pop an empty stack and the machine reads whatever lies before the array. Jump to address 1000 in a twenty-word program and it runs whatever is there. Sometimes it crashes, sometimes it prints nonsense, and sometimes, worst of all, it seems to work.

A *trap* replaces all of that with one rule: before doing anything that could go wrong, check; if it would go wrong, stop, and say what and where.

```
$ ./vm divzero
trap: division by zero at 4
```

The address is the start of the instruction that failed, not wherever `pc` has moved on to, because that is the address a person debugging the program needs. That is `start`, set before each instruction, and `trap` reports it:



ch04-traps/vm.c, lines 61–69

```
61 /*
62  * Stop the machine. The message names the instruction that was
63  * running, not wherever pc happens to point by now.
64  */
65 static void trap(const char *why) {
66     fflush(stdout);
67     fprintf(stderr, "trap: %s at %d\n", why, start);
68     exit(1);
69 }
```

4.1 What is checked

Trap	When
stack overflow	a push onto a full stack
stack underflow	a pop below the current frame
too many nested calls	a `CALL` with all frames in use
not enough arguments	`CALL addr n` with fewer than `n` values in the frame
no such local	`LOAD k` or `STORE k` outside the current frame
return without call	`RET` at the top level
division by zero	`DIV` or `MOD` by 0
unknown instruction	an opcode the machine does not have
jump out of program	a `JMP`, `JZ` or `CALL` target outside the code
missing operand	an instruction cut off by the end of the code
ran off the end of the program	`pc` leaves the code without a `HALT`

Stack underflow is checked against the current frame, not against the bottom of the stack. A function cannot pop its caller's values, even by accident. This is a choice; chapter 3's machine would have let it.

ch04-traps/vm.c, lines 77–81

```
77 static int pop(void) {
78     if (sp == fp)
79         trap("stack underflow");
80     return stack[--sp];
81 }
```

Most of the checks are in small functions that every instruction uses to read what follows it: an operand, an address to jump to, the number of a local.

ch04-traps/vm.c, lines 83–87

```
83 static int operand(void) {
84     if (pc >= code_len)
85         trap("missing operand");
86     return code[pc++];
87 }
```

ch04-traps/vm.c, lines 89–94

```
89 static int address(void) {
90     int addr = operand();
91     if (addr < 0 || addr >= code_len)
92         trap("jump out of program");
93     return addr;
94 }
```

ch04-traps/vm.c, lines 96–101

```
96 static int local(void) {
97     int k = operand();
98     if (k < 0 || fp + k >= sp)
99         trap("no such local");
100    return fp + k;
101 }
```

So `run` changes less than the list of traps suggests: it uses these instead of reading `code[pc++]` itself, and checks what only it can check.

run in ch03-calls/vm.c → ch04-traps/vm.c

```
@@ -1,8 +1,12 @@
-static void run(const int *code) {
-    int pc = 0;
+static void run(void) {
+    pc = 0;

     for (;;) {
-        int op = code[pc++];
-        int a, b;
+        int op, a, b;
+
+        start = pc;
+        if (pc < 0 || pc >= code_len)
+            trap("ran off the end of the program");
+        op = code[pc++];

         switch (op) {
@@ -10,5 +14,5 @@
             return;
         case PUSH:
-            push(code[pc++]);
+            push(operand());
             break;
         case ADD:
@@ -31,12 +35,11 @@
             break;
         case JMP:
-            pc = code[pc];
+            pc = address();
             break;
         case JZ:
+            b = address();
             a = pop();
             if (a == 0)
-                pc = code[pc];
-            else
-                pc++;
+                pc = b;
             break;
         case DUP:
@@ -81,12 +84,18 @@
             break;
```

```
         case LOAD:
-            push(stack[fp + code[pc++]]);
+            a = local();
+            push(stack[a]);
             break;
         case STORE:
-            stack[fp + code[pc++]] = pop();
+            a = local();
+            stack[a] = pop();
             break;
         case CALL: {
-            int addr = code[pc++];
-            int nargs = code[pc++];
+            int addr = address();
+            int nargs = operand();
+            if (nargs < 0 || nargs > sp - fp)
+                trap("not enough arguments");
+            if (fsp == FRAMES)
+                trap("too many nested calls");
             frames[fsp].ret = pc;
             frames[fsp].fp = fp;
@@ -97,4 +106,6 @@
         }
         case RET:
+            if (fsp == 0)
+                trap("return without call");
             a = pop();
             sp = fp;
@@ -104,4 +115,20 @@
             push(a);
             break;
+        case DIV:
+            b = pop();
+            a = pop();
+            if (b == 0)
+                trap("division by zero");
+            push(a / b);
+            break;
+        case MOD:
+            b = pop();
+            a = pop();
+            if (b == 0)
+                trap("division by zero");
+            push(a % b);
+            break;
+        default:
+            trap("unknown instruction");
         }
     }
```

4.2 The cost

Every instruction now does extra comparisons, so the machine is slower. How much that costs is measured in chapter 19. Chapter 19 shows another way: check a program *once*, before it runs, and leave out most of the checks while it runs. That is what the Java Virtual Machine's verifier does.

4.3 New instructions

Code	Name	Operand	Effect
19	`DIV`		`a b` → `a / b`, rounded toward zero; trap if `b = 0`
20	`MOD`		`a b` → `a % b`, with the sign of `a`; trap if `b = 0`

4.4 Run it

`./vm` runs Euclid's algorithm. `./vm divzero`, `./vm deep` and the others run programs that are broken on purpose; the list is at the bottom of `vm.c`.

```
make test
```

The test checks that the good program prints the right numbers,

```
21
1
```

and that every broken one stops with exit status 1 and the right message:

```
divzero: trap: division by zero at 4
underflow: trap: stack underflow at 2
deep: trap: stack overflow at 7
forever: trap: too many nested calls at 3
badop: trap: unknown instruction at 2
runoff: 1
runoff: trap: ran off the end of the program at 3
```

```
badjump: trap: jump out of program at 0
badret: trap: return without call at 2
```

4.5 Exercises

1. `ADD` and `MUL` can overflow a C `int`, and in C that is undefined behaviour. Make them trap instead. (Hint: `__builtin_add_overflow` in GCC and Clang, or check before adding.)
2. `INT_MIN / -1` also overflows. Find it, then trap on it.
3. Some languages prefer to *recover* from errors rather than stop. What would the machine need so that a program could catch a division by zero itself?

Chapter 5

Programs in files

The machine is built once, and runs programs from files, which it checks before it runs a single instruction.

The machine of chapter 4 had its programs compiled into it. To run a different program, you changed the C source and recompiled the machine. That is fine for a demonstration and useless for anything else.

This chapter separates the two. The machine is built once; programs are files it reads.

```
$ python3 tobin.py samples/fib.txt samples/fib.vm
$ ./vm samples/fib.vm
0
1
1
...
```

No instructions are new. Only `main` changed, and a loader was added in front of `run`: `run` is the same, word for word, as in chapter 4.

5.1 The pieces

- `FORMAT.md` describes the file format: a magic number, a version, a length, then the code.
- `vm.c` loads a file, checks it, and runs it.

- `tobin.py` turns a text listing such as `PUSH 2 PUSH 3 ADD PRINT HALT` into a `.vm` file.
- `samples/` holds the programs of earlier chapters as text, and one new one, `primes.txt`.
- `badfiles.py` writes files broken in every way the loader checks for, so the test can make sure each is refused.

5.2 The format

A `.vm` file holds one program. All numbers in it are 32-bit signed integers, stored *little-endian*: least significant byte first.

Offset	Size	Contents
0	3	the magic number: the bytes `S`, `V`, `M`
3	1	the format version: `1`
4	4	n, the number of code words, 1 to 65536
8	$4 \times n$	the code words, from address 0 upward

The *magic number* lets the machine tell a program from any other file at once, rather than running a JPEG and trapping a few instructions later. The version makes it possible to change the format later without old machines quietly misreading new files. The length lets the machine check that the whole program is there, and that nothing follows it. The whole format, and why each part is there, is in appendix H.

The loader reads each word from its four bytes, so it never depends on the byte order of the computer it runs on:

ch05-files/vm.c, lines 256–265

```
256 /* A 32-bit little-endian word, whatever the byte order of this
      ↪ computer. */
257 static long word32(const unsigned char *b) {
258     unsigned long u = (unsigned long) b[0]
259                     | (unsigned long) b[1] << 8
260                     | (unsigned long) b[2] << 16
261                     | (unsigned long) b[3] << 24;
262     if (u >= 0x80000000UL)
263         return (long) (u - 0x80000000UL) - 0x7fffffffL - 1;
```

```
264     return (long) u;
265 }
```

ch05-files/vm.c, lines 267–294

```
267 static void load(const char *path) {
268     unsigned char header[8], b[4];
269     long n, i;
270     FILE *f = fopen(path, "rb");
271 
272     if (f == NULL)
273         bad_file(path, "cannot open");
274     if (fread(header, 1, 8, f) != 8)
275         bad_file(path, "too short to be a program");
276     if (header[0] != 'S' || header[1] != 'V' || header[2] != 'M')
277         bad_file(path, "not a program (wrong magic number)");
278     if (header[3] != 1)
279         bad_file(path, "unknown format version");
280     n = word32(header + 4);
281     if (n < 1 || n > MAX_CODE)
282         bad_file(path, "bad code length");
283     for (i = 0; i < n; i++) {
284         if (fread(b, 1, 4, f) != 4)
285             bad_file(path, "code ends early");
286         words[i] = (int) word32(b);
287     }
288     if (fgetc(f) != EOF)
289         bad_file(path, "extra bytes after the code");
290     fclose(f);
291 
292     code = words;
293     code_len = (int) n;
294 }
```

5.3 Two kinds of failure

A file can be wrong *as a file*: not a program at all, from a newer version, cut short. Those are caught before any instruction runs, and the machine exits with status 2. A file can also be a perfectly good file holding a wrong program, which divides by zero or jumps off the end. Those are traps, as in chapter 4, with status 1.

Keeping the two apart matters for whoever reads the message: one says *this file is damaged*, the other says *this program has a bug*.

ch05-files/vm.c, lines 251–254

```
251 static void bad_file(const char *path, const char *why) {
252     fprintf(stderr, "%s: %s\n", path, why);
253     exit(2);
254 }
```

5.4 Not yet an assembler

`tobin.py` translates names to numbers and nothing else. Look at `samples/primes.txt`: every jump target is an address counted by hand and written into a comment, and adding one instruction near the top means recounting everything below it. That is the problem chapter 6 solves with *labels*.

```
# The primes below 50, by trial division.
#
# main keeps n as local 0 and counts it from 2 up to 49.
# prime(n) returns 1 if n is prime, otherwise 0. Its locals are
# n (local 0, the argument) and d (local 1, made by pushing 2).

PUSH 2              #  0  n = 2
LOAD 0 PUSH 50 LT   #  2  loop: n < 50 ?
JZ 28               #  7  no: stop
LOAD 0 CALL 29 1    #  9  prime(n)
JZ 19               # 14  not prime: skip printing
LOAD 0 PRINT        # 16  print n
LOAD 0 PUSH 1 ADD   # 19  n + 1
STORE 0             # 24  n = n + 1
JMP 2               # 26
HALT                # 28  end

PUSH 2              # 29  prime: d = 2
LOAD 1 LOAD 1 MUL   # 31  test: d * d
LOAD 0 SWAP LT      # 36  n < d * d ?
JZ 45               # 40  no: try dividing
PUSH 1 RET          # 42  yes: n is prime
LOAD 0 LOAD 1 MOD   # 45  n mod d
JZ 61               # 50  d divides n: not prime
LOAD 1 PUSH 1 ADD   # 52  d + 1
STORE 1             # 57  d = d + 1
JMP 31              # 59  test again
PUSH 0 RET          # 61  not prime
```

5.5 Run it

```
make test
```

Every sample must print what it should, and every broken file must be refused with exit status 2 and the right message:

```
tests/empty.vm: too short to be a program
tests/huge.vm: bad code length
tests/magic.vm: not a program (wrong magic number)
tests/negative.vm: bad code length
tests/short.vm: too short to be a program
tests/trailing.vm: extra bytes after the code
tests/truncated.vm: code ends early
tests/version.vm: unknown format version
tests/zero.vm: bad code length
```

5.6 Exercises

1. Write a program in a text file that prints the squares of 1 to 10, turn it into a `.vm` file, and run it.
2. Use a hex viewer (`xxd samples/gcd.vm`) and find the magic number, the length, and the first instruction.
3. Add one instruction near the top of `primes.txt`. How many numbers must you change? Which ones?
4. Make `tobin.py` check that an instruction which needs an operand actually gets a number after it, and not another instruction name.

Part II

Tools

Chapter 6

An assembler

Programs written with names instead of numbers: labels for addresses, counted by a two-pass assembler, and a disassembler that turns a program file back into text.

Part II begins. The machine itself is finished for now: `vm.c` is chapter 5's, unchanged. This part is about the tools that make the machine usable.

Look again at chapter 5's `primes.txt`. Every jump target is a number counted by hand, and every comment carries an address so the counting can be checked. Insert one instruction near the top and every number below it is wrong.

Here is the same program in `samples/primes.asm`:

ch06-asm/samples/primes.asm

```
; The primes below 50, by trial division.
;
; Compare with ch05-files/samples/primes.txt: the same program, with
; every address counted by hand.

; main: local 0 is n, counting from 2 up to 49
main:   PUSH 2
next:   LOAD 0
        PUSH 50
        LT
        JZ end
```

```
12          LOAD 0
13          CALL prime 1
14          JZ skip
15          LOAD 0
16          PRINT
17  skip:   LOAD 0
18          PUSH 1
19          ADD
20          STORE 0
21          JMP next
22  end:    HALT
23
24  ; prime(n): 1 if n is prime, else 0
25  ; local 0 is n, local 1 is the divisor d, starting at 2
26  prime:  PUSH 2
27  test:   LOAD 1
28          LOAD 1
29          MUL
30          LOAD 0
31          SWAP
32          LT              ; n < d * d ?
33          JZ divide
34          PUSH 1          ; no divisor up to the square root: prime
35          RET
36  divide: LOAD 0
37          LOAD 1
38          MOD
39          JZ composite    ; d divides n
40          LOAD 1
41          PUSH 1
42          ADD
43          STORE 1
44          JMP test
45  composite:
46          PUSH 0
47          RET
```

`next:` is a *label*. It names the address of the instruction that follows it, and `JMP next` jumps there. Nobody counts; the assembler does. Both files produce exactly the same bytes, and the test checks this for the samples.

6.1 Two passes

How can `JZ end` be translated before the assembler has seen where `end` is? It cannot, in one reading. So `asm.py` reads the source twice:

1. **Counting.** Go through every line, work out how many words its instruction needs, and so the address of every line. Whenever a label appears, write down the current address.
2. **Writing.** Go through again and produce the code. Every label now has a known address, including the ones defined further down.

That is the *two-pass assembler*, and it has looked like this since the 1950s. The two passes are two short methods:

ch06-asm/asm.py, lines 101–110

```
101     def pass1(self, lines):
102         address = 0
103         for line in lines:
104             for label in line.labels:
105                 if label in self.labels:
106                     self.error(line, f"label '{label}' is defined
    ↪ twice")
107                 else:
108                     self.labels[label] = address
109             line.address = address
110             address += size(line)
```

ch06-asm/asm.py, lines 129–156

```
129     def pass2(self, lines):
130         code = []
131         for line in lines:
132             if line.op is None:
133                 continue
134             if line.op.lower() == ".word":
135                 if len(line.operands) != 1:
136                     self.error(line, ".word takes one number")
137                     code.append(0)
138                 else:
139                     code.append(self.operand(line, "n",
    ↪ line.operands[0]))
140                 continue
141             entry = isa.BY_NAME.get(line.op.upper())
```

```
142             if entry is None:
143                 self.error(line, f"unknown instruction
    ↪ '{line.op}'")
144                 code.append(0)
145                 continue
146             op, kinds = entry
147             if len(line.operands) != len(kinds):
148                 self.error(line, f"{line.op.upper()} takes
    ↪ {len(kinds)} "
149                                  f"operand{'' if len(kinds) == 1
    ↪ else 's'}, "
150                                  f"found {len(line.operands)}")
151                 code.extend([op] + [0] * len(kinds))
152                 continue
153             code.append(op)
154             for kind, word in zip(kinds, line.operands):
155                 code.append(self.operand(line, kind, word))
156         return code
```

6.2 Checks

A good tool catches mistakes before they become traps. The assembler checks that:

- every instruction exists,
- it has exactly the number of operands it needs,
- each operand is the right kind: a number, a local, or an address,
- every label used is defined, and none is defined twice,
- every number fits in 32 bits.

It reports *all* the mistakes it finds, not just the first, each with its line number. This file is wrong on every line after the first instruction,

ch06-asm/tests/errors.asm

```
1 ; Every line below the first is wrong in some way the assembler
2 ; must notice. The test checks that it reports all of them.
3
4 start:  PUSH 1
5         PUSH
6         PUSH 1 2
7         PUSH ten
8         JUMP start
9         JMP nowhere
```

```
        LOAD -1
        CALL start
start:  HALT
        .word
        PUSH 99999999999
```

and these are the messages it must produce:

```
tests/errors.asm:5: PUSH takes 1 operand, found 0
tests/errors.asm:6: PUSH takes 1 operand, found 2
tests/errors.asm:7: expected a number, found 'ten'
tests/errors.asm:8: unknown instruction 'JUMP'
tests/errors.asm:9: undefined label 'nowhere'
tests/errors.asm:10: a local number cannot be negative: -1
tests/errors.asm:11: CALL takes 2 operands, found 1
tests/errors.asm:12: label 'start' is defined twice
tests/errors.asm:13: .word takes one number
tests/errors.asm:14: number out of range: 99999999999
```

6.3 One table, two tools

`isa.py` describes the instruction set as data: name, code, and the kinds of operand each instruction takes. Both the assembler and the disassembler read that one table.

ch06-asm/isa.py, lines 15–38

```
INSTRUCTIONS = [
    # (code, name, operand kinds)
    (0, "HALT", ()),
    (1, "PUSH", ("n",)),
    (2, "ADD", ()),
    (3, "SUB", ()),
    (4, "MUL", ()),
    (5, "PRINT", ()),
    (6, "JMP", ("addr",)),
    (7, "JZ", ("addr",)),
    (8, "DUP", ()),
    (9, "DROP", ()),
    (10, "SWAP", ()),
    (11, "OVER", ()),
    (12, "ROT", ()),
    (13, "LT", ()),
    (14, "EQ", ()),
    (15, "LOAD", ("k",)),
```

```
    (16, "STORE", ("k",)),
    (17, "CALL", ("addr", "n")),
    (18, "RET", ()),
    (19, "DIV", ()),
    (20, "MOD", ()),
]
```

Before, the instruction set was written down in `vm.c`, in `tobin.py` and in `ISA.md`, three copies that could drift apart. Now the Python tools share one. The C machine still has its own copy; chapter 8 makes sure they agree.

6.4 The disassembler

`disasm.py` goes the other way: from a `.vm` file to assembly language.

```
$ python3 disasm.py samples/gcd.vm
        PUSH 1071           ; 0
        PUSH 462            ; 2
        CALL L17 2          ; 4
        ...
L17:    LOAD 1              ; 17
        JZ L32              ; 19
```

Label names are not stored in the file, only addresses, so the disassembler invents `L17` for address 17. A word it cannot read as an instruction becomes `.word`. Either way, what it writes assembles back into the identical file, and the test checks that for every sample.

6.5 The listing

`asm.py -l` prints, beside each source line, its address and the words it became:

```
   4  17 17 2                  CALL gcd 2
```

When a program misbehaves at "address 28", the listing tells you which line that was. Chapter 7 makes much more of this.

6.6 Run it

```
make test
```

6.7 Exercises

1. Write a program that prints the multiplication table for 7, using labels throughout.
2. Add `EQU`, so that `size EQU 50` lets `PUSH size` mean `PUSH 50`. Which pass has to handle it?
3. The disassembler cannot tell code from data. Write a program where it gets confused, and explain why no disassembler can avoid this in general.
4. Make the assembler warn about code that can never be reached, such as an instruction directly after a `JMP` with no label.

Chapter 7

Watching the machine run

The machine shows its work: a trace of every step, a debugger that stops, looks and steps, and a replay that goes back through a run after it is over.

When a program prints the wrong number, all the machine has told you is the wrong number. Everything that led to it happened out of sight. This chapter makes the machine show its work, in three ways:

```
./vm program.vm          run, as before
./vm -t program.vm       trace: one line per step, on stderr
./vm -d program.vm       debug: stop, look, step, continue
```

and a fourth tool, `replay.py`, explores a saved trace after the run is over, backwards as well as forwards.

7.1 Splitting the loop

Until now `run()` was one loop with the whole machine inside it. Here the body of the loop becomes a function of its own, `step()`, which runs exactly one instruction. Then:

- `run()` calls `step()` until it returns 0,
- `trace()` calls `step()` and prints a line after each call,
- `debug()` calls `step()` when you tell it to.

ch07-trace/vm.c, lines 288–291

```
288 static void run(void) {
289     while (step())
290         ;
291 }
```

ch07-trace/vm.c, lines 293–314

```
293 /*
294  * Tracing: after every instruction, write a line to stderr with
       ↪ the
295  * step number, the address, the instruction, and the stack after
       ↪ it.
296  * The lines can be saved in a file and explored with replay.py.
297  */
298 static void trace(void) {
299     long n = 0;
300     int at, going, width;
301 
302     do {
303         at = pc;
304         going = step();
305         fflush(stdout);
306         fprintf(stderr, "%6ld %5d  ", ++n, at);
307         width = show_instruction(stderr, at);
308         if (sp > 0 || fsp > 0) {
309             fprintf(stderr, "%*s  ", width < 14 ? 14 - width : 0,
       ↪ "");
310             show_stack(stderr);
311         }
312         fprintf(stderr, "\n");
313     } while (going);
314 }
```

Every debugger, from the 1960s' DDT to gdb, is built on the same split: a machine that can be told to take *one step* and then be looked at.

The machine also has to know the names of its own instructions now, to print them. It has a `names` table, and a `noperands` table telling how many operands follow each instruction.

7.2 The trace

```
$ ./vm -t samples/gcd.vm
     1     0  PUSH 1071        1071
     2     2  PUSH 462         1071 462
     3     4  CALL 17 2        | 1071 462
     4    17  LOAD 1           | 1071 462 462
   ...
    17    28  CALL 17 2        | 1071 462 | 462 147 | 147 21
```

Each line holds the step number, the address, the instruction, and the stack *after* the instruction ran. A bar `|` marks where a call's frame begins, so you can watch the recursion go down, three calls deep, and come back up again. What the program prints still goes to standard output; the trace goes to standard error, so the two can be kept apart: `./vm -t gcd.vm 2> trace.txt`.

7.3 The debugger

```
$ ./vm -d samples/gcd.vm
    0  PUSH 1071
(vm) b 32
(vm) c
   32  LOAD 0
(vm) p
stack: | 1071 462 | 462 147 | 147 21 | 21 0
frame 4: returns to 31, locals start at 6
...
```

Command	Does
`s [n]`	step one instruction, or `n`
`c`	continue until a breakpoint or the end
`b addr`	set a breakpoint
`d addr`	delete a breakpoint
`p`	print the stack and the frames
`l [n]`	list the next `n` instructions; `*` marks breakpoints
`q`	quit

The addresses are the ones in the assembler's listing (`asm.py -l`) or the disassembler's output. If a program traps under the debugger, the machine prints the stack as it was at that moment. The values the failing instruction took off the stack are gone by then: in `tests/divzero.debug.out` the 100 and the 0 of the division are no longer there.

7.4 Replaying a trace

The debugger only goes forward. But when you find a wrong value, the question is nearly always *where did it come from?*, which is a question about the past. A trace is the whole past, written down.

```
$ ./vm -t samples/gcd.vm 2> trace.txt
$ python3 replay.py trace.txt
(replay) v 21          find the first step where 21 is on top
    16    27  MOD             | 1071 462 | 462 147 147 21
(replay) w             show the steps around it
(replay) b 28          back to the previous step at address 28
```

Debuggers that can go backwards are called *reverse* or *time-travel* debuggers. gdb can do it, slowly; `rr` does it by recording a run, which is the idea here at its smallest.

7.5 Tests

The tests run a scripted debugger session and a scripted replay session, and compare what they print, along with a whole trace, against saved files in `tests/`. If you change the trace format on purpose, check the new output by eye, then run `make expected` to save it as the new expected output.

```
make test
```

7.6 Exercises

1. Add `w addr` to the debugger: a *watchpoint* that stops whenever the instruction at `addr` changes the top of the stack.
2. Make `b` accept a label as well as an address. What would the machine need to know, and where could that come from? (Look at the end of appendix H.)
3. Write a profiler: count how many times each address is executed and print the ten busiest. Which part of `primes` is hottest?
4. A trace of a long run is enormous. Instead of writing every step, record only what cannot be worked out again: here, nothing at

all, since the machine is deterministic. What would change once the machine could read input?

Chapter 8

A specification

The machine is written down: a specification decides what was left to chance, a second machine in Python is built from it, and one test suite holds both to it.

What *is* this machine? Until now the answer was "whatever `vm.c` does". That answer has two problems.

First, `vm.c` does some things by accident. Ask chapter 7's machine for `-2147483648 / -1` and, on an ordinary PC, the whole machine dies with a hardware exception, taking the program's earlier output with it. Nobody decided that; it is what the C compiler and the processor happen to do with a case C leaves undefined.

Second, there can only ever be one machine. A second implementation, in Python, in JavaScript, in hardware, can only copy `vm.c`, bugs included, and nobody can say which differences matter.

So this chapter writes the machine down: `SPEC.md`, the *specification*, which is appendix H at the end of this book. Then it builds a second machine from the document, not from the C, and a test suite that holds both to it.

8.1 What writing it down decided

Some questions have no answer until somebody writes one:

- **Overflow.** `2147483647 + 1` is now `-2147483648`, by definition: arithmetic wraps at 32 bits. The C machine computes with unsigned numbers to get there without undefined behaviour.
- **Division.** Rounds towards zero, and the remainder takes the sign of the dividend, as in C. `-2147483648 / -1` wraps to `-2147483648`; it no longer crashes anything.
- **Order.** `JZ 9999` on an empty stack: is that a bad jump or a stack underflow? The spec gives every instruction its steps in order, and the first failing step wins. (A bad jump.)
- **A bug.** In chapter 7, `STORE` checked which local to write *before* popping the value. So `STORE 0` with one value in the frame wrote into the slot it was about to free. Writing the steps out in order showed it. `STORE` now pops first.

Equally important is what the spec does *not* fix: usage messages, the debugger, anything a program cannot observe. Those are up to each implementation.

8.2 Two machines

`vm.c` is chapter 7's machine with the changes above.

`vm.py` is a new machine in Python, written from the spec. It is not a line-by-line translation, and the differences are where the lessons are. Python's integers never overflow, so it has to wrap by hand. Python's `//` rounds towards minus infinity, so `-7 // 2` is `-4` in Python but must be `-3` here. The same division, in the two machines:

ch08-spec/vm.c, lines 180–185

```
180 /* INT32_MIN / -1 is the one quotient too big for 32 bits. It
      ↪ wraps. */
181 static int wrap_div(int a, int b) {
182     if (b == -1)
183         return wrap_sub(0, a);
184     return a / b;
185 }
```

ch08-spec/vm.py, lines 29–31

```
29 def wrap(n):
30     """Wrap an integer of any size into a signed 32-bit value."""
31     return (n + 2**31) % 2**32 - 2**31
```

ch08-spec/vm.py, lines 34–37

```
34 def divide(a, b):
35     """The quotient rounded towards zero, as in C, before
   ↪ wrapping."""
36     q = abs(a) // abs(b)
37     return q if (a < 0) == (b < 0) else -q
```

The Python machine supports tracing (`-t`) in exactly the format of section 8 of the spec, but not the debugger, which the spec leaves out.

8.3 The conformance suite

`suite/` holds 31 small programs, each with an `.expect` file saying what the spec says must happen: what it prints, its exit status, and its error message. There is at least one test for every instruction and every trap, for the exact limits (256 values on the stack are fine and 257 are not, and the same for frames), and for each rule above. `conform.py` runs them all, plus nine damaged program files, against any machine:

```
$ python3 conform.py ./vm
./vm: 40 of 40 passed
$ python3 conform.py python3 vm.py
python3 vm.py: 40 of 40 passed
```

The expectations were written from the spec, before running either machine. Run against chapter 7's machine, the suite fails three tests, exactly the three changes above. Run against a Python machine that uses Python's `//`, it fails one test, the division test.

8.4 Comparing traces

Two machines can print the same answers by different routes. `compare.py` runs both with `-t` and compares the traces line by line, every step and every stack:

```
$ python3 compare.py ./vm "python3 vm.py" build/*.vm samples/*.vm
44 of 44 programs traced identically
```

When they differ, it shows the first step where they part ways:

```
build/division.vm: the machines differ
  first different line, number 7:
      ./vm:      7    10  DIV             -3
    vm.py:       7    10  DIV            -4
```

Testing one implementation against another like this is called *differential testing*. It finds bugs that nobody thought to write a test for, as long as the two machines were written independently.

8.5 Run it

```
make test
```

8.6 Exercises

1. Write a third machine, in any language you like, from `SPEC.md` alone, without looking at the other two. Run `conform.py` on it. Which tests failed the first time, and was it your machine or the spec that was unclear?
2. Find something the spec still leaves open that a program *can* observe. (Hint: section 9 lists one. Are there others?)
3. Write a program that generates random programs, and use `compare.py` to run them on both machines. This is *fuzzing*. What do you need to do so that the random programs halt?
4. The suite has 40 tests. Is every line of `vm.c` run by at least one of them? Use `gcc --coverage` and `gcov` to find out.

Part III

Languages on the machine

Chapter 9

A compiler for expressions

A first compiler: arithmetic written as text becomes a program for the machine, by way of tokens, a syntax tree and assembly.

Part III begins. Since chapter 6 we have written programs in assembly language, one instruction at a time. Now we write them in a language of our own, and let a compiler write the assembly.

The language is deliberately tiny: arithmetic, and nothing else.

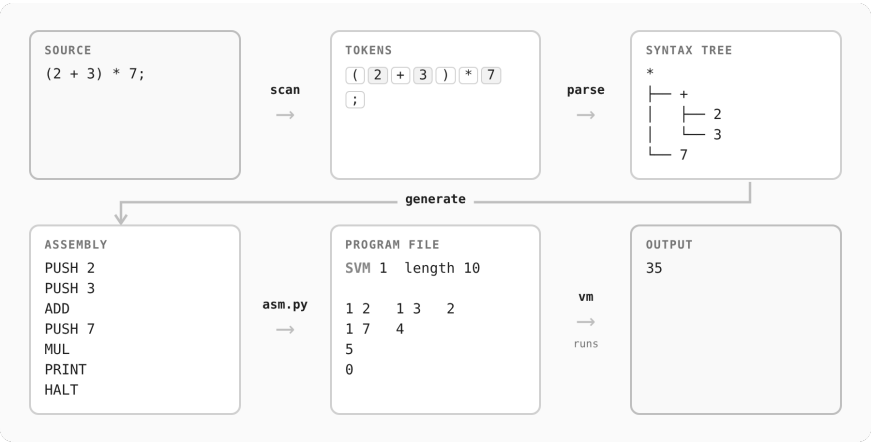
ch09-expr/samples/basics.expr

```
# Each expression is printed on a line of its own.

1 + 2;
(2 + 3) * 7 - 5;
2 + 3 * 7 - 5;          # * binds tighter than + and -
100 - 10 - 1;           # left to right: (100 - 10) - 1
2 * (3 + 4) * 5;
-7 / 2;                 # rounds towards zero
-7 % 2;
--5;                    # minus minus five
-(2 + 3) * -(4 - 6);
```

Each expression is printed when the program runs. There are no variables, no loops, no decisions. That is the point: with nothing else in the way, the three stages of a compiler can be seen clearly. Chapter 10 builds a whole language on the same three stages.

The machine is chapter 8's, unchanged.



9.1 Stage 1: scanning

The *scanner* reads characters and produces *tokens*, the words of the language. It knows what a number looks like and which characters are operators, and it throws away spaces and comments. It knows nothing about grammar: to the scanner, `) 3 (` is three perfectly good tokens.

```
$ python3 expr.py -tokens samples/basics.expr
3:1  number   '1'
3:3  +        '+'
3:5  number   '2'
3:6  ;        ';'
```

Every token remembers its line and column, so that every later error can say exactly where it is.

9.2 Stage 2: parsing

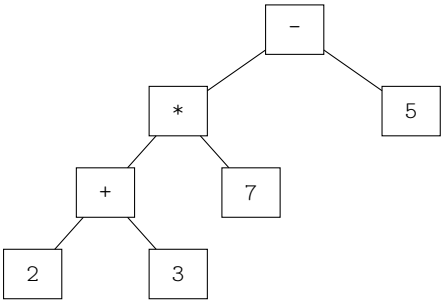
The *parser* checks the tokens against the grammar:

```
program    = { expression ";" } .
expression = term { ("+" | "-") term } .
term       = unary { ("*" | "/" | "%") unary } .
unary      = "-" unary | factor .
factor     = number | "(" expression ")" .
```

and builds a *syntax tree*, which this prints:

```
$ python3 expr.py -tree samples/basics.expr
```

Here is the tree for `(2 + 3) * 7 - 5`, drawn:



The parser in `expr.py` has one function per grammar rule, and each calls the functions for the rules it mentions. This is a *recursive descent* parser, the kind Niklaus Wirth used for PL/0 and Pascal and the kind `archive/cmp2` began with.

ch09-expr/expr.py, lines 157–162

```
157     def expression(self):
158         node = self.term()
159         while self.peek().kind in ("+", "-"):
160             token = self.take(self.peek().kind)
161             node = Binary(token.kind, node, self.term(), token)
162         return node
```

ch09-expr/expr.py, lines 164–169

```
164     def term(self):
165         node = self.unary()
166         while self.peek().kind in ("*", "/", "%"):
167             token = self.take(self.peek().kind)
168             node = Binary(token.kind, node, self.unary(), token)
169         return node
```

Two properties of arithmetic live in the shape of the grammar, not in any special code:

- **Precedence.** `*` binds tighter than `+` because `terms` are built inside `expressions`. In `2 + 3 * 7`, the `3 * 7` is finished before the `+` sees it.
- **Associativity.** `100 - 10 - 1` is `(100 - 10) - 1` because `expression` loops, adding each new term to the tree built so far. Written as recursion on the right instead, it would compute `100 - (10 - 1)`, and the tests catch exactly that mistake.

The tree has no parentheses. Once the shape is right, they have done their job.

9.3 Stage 3: generating code

For a stack machine, code generation is almost nothing. To compute `left + right`: generate code for `left`, which leaves its value on the stack; generate code for `right`, which leaves its value on top; then `ADD`. Every node works the same way: children first, then the node itself. This order is called *post-order*, and for a stack machine it *is* the code generator.

ch09-expr/expr.py, lines 224–234

```
224 def generate(node, out):
225     if isinstance(node, Number):
226         out.append(f"        PUSH {node.value}")
227     elif isinstance(node, Negate):
228         out.append("        PUSH 0")
229         generate(node.operand, out)
230         out.append("        SUB")
231     else:
232         generate(node.left, out)
233         generate(node.right, out)
234         out.append(f"        {OPCODES[node.op]}")
```

Unary minus becomes `0 - x`: `PUSH 0`, the code for `x`, `SUB`. The machine has no `NEG` instruction and does not need one.

9.4 The whole pipeline

```
python3 expr.py prog.expr > prog.asm      compile
python3 asm.py prog.asm prog.vm           assemble
./vm prog.vm                              run
```

The `Makefile` has rules for both steps, so `make samples/basics.vm` does the first two.

9.5 Errors

A compiler meets far more wrong programs than right ones, so its error messages matter. Each says where, and shows it:

```
tests/paren.expr:2:7: expected ')', found ';'
    (3 + 4;
          ^
```

Division by zero is *not* a compile error, even in `10 / (5 - 5)` where the compiler could know. The compiler does not evaluate anything; the machine traps when the program runs. (Chapter 18's optimiser does work out constant expressions, and leaves `10 / 0` alone for exactly this reason: folding it away would remove the trap, and change what the program does.)

9.6 Testing with random programs

`fuzz.py` writes 500 random expressions with as few parentheses as the grammar allows, and works out in Python what each must print, following the specification: 32-bit wrapping, division rounding towards zero. The test compiles them, runs them, and compares. A parser that gets associativity wrong fails 464 of the 500.

```
make test
```

9.7 Exercises

1. Leave out the ; after line 2 of a program. The error message points at line 3. Why? Make the message say where the ; was missing instead.

2. Add `^` for powers. Powers usually associate to the *right*: `2 ^ 3 ^ 2` is `2 ^ 9`. Where in the grammar does it go, and why is right recursion now correct?
3. The scanner treats `2 3;` as two numbers, and the parser then rejects it. Could the scanner have caught it? Should it?
4. Count the stack depth each expression needs. Add a check to the compiler that refuses expressions too deep for the machine's 256 values, before they are ever run.

Chapter 10

enkel/0

A whole language on the machine: variables, arrays, procedures, recursion, loops and decisions, compiled in four stages.



Chapter 9 compiled arithmetic. This chapter compiles a language: variables, arrays, procedures, recursion, loops and decisions.

ch10-enkel/samples/gcd.p

```
procedure gcd[a, b];
  begin
    while a # b do
      begin
        if b < a
          then a is a - b
          else b is b - a
      end;
      return a
  end;

begin
  call gcd[15, 6];
   print rval
end.
```

The language is *enkel/0*, from the earlier collection (`archive/enkel0`); *enkel* is Swedish for *simple*. Its grammar, from `LANG.md`:

ch10-enkel/LANG.md, lines 12–35

```
12 program    = block "." .
13
14 block      = [ "const" name "=" number { "," name "=" number } ";"
   ↪ ]
15              [ "array" name ":" number { "," name ":" number } ";"
   ↪ ]
16              [ "var" name { "," name } ";" ]
17              { "procedure" name "[" [ name { "," name } ] "]" ";"
   ↪ block ";" }
18              statement .
19
20 statement  = variable "is" expression
21            | "call" name "[" [ expression { "," expression } ] "]"
22            | "begin" statement { ";" statement } "end"
23            | "if" condition "then" statement [ "else" statement ]
24            | "while" condition "do" statement
25            | "do" statement "while" condition
26            | "return" [ expression ]
27            | "print" expression
28            | "emit" expression
29            | (nothing) .
30
31 condition  = expression [ ( "=" | "#" | "<" | "<=" | ">" | ">=" )
   ↪ expression ] .
32 expression = term { ( "+" | "-" | "or" | "xor" ) term } .
33 term       = factor { ( "*" | "/" | "%" | "and" ) factor } .
34 factor     = variable | number | "(" expression ")" | "-" factor .
35 variable   = name [ "." ( name | number | "(" expression ")" ) ] .
```

All ten of its old sample programs compile and run here, unchanged apart from their line endings.

10.1 Four stages

`enkel.py` has the stages of chapter 9 and one more:

Stage	From	To	New here
scan	text	tokens	keywords, names, two-character symbols like `<=`, comments
parse	tokens	syntax tree	statements, declarations, procedures
resolve	tree	tree with meanings	everything
generate	tree	assembly	jumps, calls, variables

Resolving is what names make necessary. The parser knows that `x is 1` is an assignment to something called `x`. It does not know, and cannot know, whether `x` is a local variable, a global one, a constant that must not change, or nothing at all. Answering that needs the declarations, and they may come later in the program: a procedure can call one declared after it. So the resolver walks the whole tree after parsing and gives each name its meaning:

Meaning	Where it lives	Read with	Written with
constant	nowhere: it is a number	`PUSH n`	(refused)
global variable	memory	`PUSH addr`, `LOADM`	`PUSH addr`, value, `STOREM`
array element	memory	`PUSH base`, index, `ADD`, `LOADM`	same, then `STOREM`
local variable	the call's frame	`LOAD k`	`STORE k`

ch10-enkel/enkel.py, lines 449–456

```
449     def lookup(self, token, name):
450         if name in self.scope:
451             return self.scope[name]
452         if name in self.globals:
453             return self.globals[name]
454         if name in self.procedures:
455             raise CompileError(f"'{name}' is a procedure, not a
    ↪ value", token)
456         raise CompileError(f"'{name}' is not declared", token)
```

Most of the compiler's error messages come from this stage:

```
tests/args.p:5:10: 'add' takes 2 arguments, given 1
        call add[1]
             ^
```

10.2 The machine grows: specification version 2

Global variables and arrays need somewhere to live that is not a call's frame, since they outlive every call. `emit` prints characters. And enkel/0 has `and`, `or` and `xor`. So the machine gains, in `SPEC.md` version 2:

Code	Name	Effect
21	AND	`a b` → `a and b`, bit by bit
22	OR	`a b` → `a or b`
23	XOR	`a b` → `a xor b`
24	LOADM	`addr` → `memory[addr]`
25	STOREM	`addr v` → ; `memory[addr] = v`
26	EMIT	`c` → ; print the byte `c`

Memory is 4096 values, all zero at the start. Both machines, the C one and the Python one, were extended, and the conformance suite grew by nine tests for the new instructions and their traps. Everything from version 1 still passes.

Note what the machine did *not* get: no `GT`, no `NEG`, no `NOT`, no `NE`. The compiler builds `a > b` as `b < a` (`SWAP`, `LT`), `-x` as `0 - x`, and `a # b` as `(a = b) = 0`. Every instruction the machine has must be implemented, specified and tested twice; every one it does without is free.

10.3 What the code looks like

```
$ python3 enkel.py samples/gcd.p
...
; 5: if b < a
        LOAD 1
        LOAD 0
        LT
        JZ L3
; 6: then a is a - b
        LOAD 0
        LOAD 1
        SUB
        STORE 0
        JMP L4
```

```
L3:
...
```

Each statement is preceded by the source line it came from, so the assembly can be read beside the program, and the tracer and debugger of chapter 7 work on it as on any other.

The code is not clever. `return a` at the end of `gcd` is followed by a `PUSH 0`, `RET` that can never run: the compiler always ends a procedure that way, in case the last statement was not a `return`. Chapter 18 deals with code like this.

10.4 rval

In enkel/0 a call is a statement, not part of an expression, and its result arrives in a global variable called `rval`. This is unusual, and it was a deliberate choice: it keeps expressions simple, since evaluating one can never call a procedure. It has a cost too. Look at `samples/recursive-factorial.p`, which works only because a procedure that ends without `return` leaves `rval` alone, so the value set by the deepest call travels back up. The compiler keeps exactly that rule: only `return` with a value sets `rval`.

10.5 A bug, thirty lines in

The old `fibonacci-dynamic.p` declared `array A:15` and wrote to `A.15`, one element past the end. It never failed, because nothing checked; the value went into whatever memory came next, which happened to be unused. Here the sample says `A:16`. The compiler catches the mistake when the index is a constant (`A.15`), but not when it is computed (`A.i`), and the machine knows nothing about arrays at all. Exercise 1 is about that.

10.6 Tests

```
make test
```

- the conformance suite, version 2, on both machines, and their traces compared on every suite program and sample;
- every sample, against its expected output;
- 300 random expressions from chapter 9's fuzzer, written as `print` statements: a second compiler must agree with the first;
- twelve programs with mistakes, against their error messages.

10.7 Exercises

1. Make array indexes safe. Either have the compiler check every computed index at run time (what code would it generate?), or add a bounds-checked instruction to the machine. Which is better, and for whom?
2. Let calls be expressions: `print gcd[15, 6]`. What has to change in the grammar, the resolver and the generator? Do you still need `rval`?
3. Allow procedures inside procedures, with access to the enclosing procedure's variables. This is what PL/0 and Pascal do, and it needs a *static link* in each frame. Why is the frame pointer of the caller not enough? (Appendix B shows two machines that do it.)
4. Add `for i is 1 to 10 do …`. Can it be translated into a `while` loop in the parser, without touching the generator at all?

Chapter 11

basal/0

A second language, a small BASIC, on the same machine with no change at all: compiled in one pass with no syntax tree, with a little desugaring and a run-time library.



A second language for the same machine. *basal/0* (Swedish for *basic*, pronounced [bɑˈsɑːl]) is a small BASIC in the spirit of Tiny BASIC: one statement to a line, variables that need no declaring, `GOTO` and `GOSUB`.

ch11-basal/samples/gcd.bas

```
# GCD (as per usual)
LET a = 15
LET b = 6

WHILE a <> b DO
  IF b < a THEN
    LET a = a - b
  ELSE
    LET b = b - a
  ENDIF
```

```
11 ENDWHILE
12
13 PRINT a
14 END
```

The program file it compiles to runs on chapter 10's machine with no change at all. That is the point of a virtual machine: enkel/0, basal/0 and anything else compiled to it share one machine, one assembler, one debugger and one specification.

11.1 The language

```
program   = { [ statement ] NEWLINE } .
statement = "!" label
          | "LET" name "=" expression
          | "PRINT" [ ( string | expression ) [ ";" ] ]
          | "IF" condition "THEN" NEWLINE program
               [ "ELSE" NEWLINE program ] "ENDIF"
          | "WHILE" condition "DO" NEWLINE program "ENDWHILE"
          | "FOR" name "=" expression "TO" expression
               [ "STEP" [ "-" ] number ] NEWLINE program "NEXT"
  ↪ name
          | "ON" expression ( "GOTO" | "GOSUB" ) label { ","
  ↪ label }
          | "GOTO" label | "GOSUB" label | "RETURN" | "END" .
condition  = expression ( "=" | "<>" | "<" | "<=" | ">" | ">=" )
             expression .
expression = term { ( "+" | "-" ) term } .
term       = factor { ( "*" | "/" ) factor } .
factor     = name | number | "(" expression ")" | ( "-" | "+" )
  ↪ factor .
```

Keywords are in capitals; names of variables and labels are case sensitive. # starts a comment. Every variable is global and starts at 0. `PRINT x;` stays on the same line.

The old samples from `archive/basal0` run as they were, including `hurkle.bas`, adapted from Tiny BASIC.

11.2 No syntax tree

`basal.py` is built differently from `enkel.py`, on purpose. enkel/0's compiler parses the whole program into a tree, resolves the names

in it, and only then generates code. basal/0's compiler builds no tree. Each parsing function writes the assembly for what it has just recognised, the moment it has recognised it:

ch11-basal/basal.py, lines 418–423

```
418     def expression(self):
419         self.term()
420         while self.at("+", "-"):
421             op = self.take(self.peek().kind).kind
422             self.term()
423             self.emit("ADD" if op == "+" else "SUB")
```

This is chapter 9's post-order code generation, folded into the parser. It is called *syntax-directed translation*. It is how most compilers worked when memory was too small to hold a tree, and BASIC suits it well: a line is a statement, and a statement rarely needs to know anything about the rest of the program.

Rarely, but not never. `GOTO DONE` can come before `! DONE`. The compiler does not need to know where `DONE` is, since it writes a symbolic label and leaves addresses to the assembler. But it does need to report a label that does not exist anywhere, so it keeps a list of the labels used and checks it at the end. Even a one-pass compiler needs a little memory of what it has seen.

11.3 Syntactic sugar

`FOR` and `ON … GOTO` are not in the translator at all. A separate stage, *desugaring*, rewrites them into simpler statements first, working on tokens:

```
$ python3 basal.py -desugar samples/countdown.bas
LET N = 10
WHILE N >= 0 DO
PRINT N ;
PRINT " " ;
LET N = N + - 2
ENDWHILE
...
IF ( K ) = 1 THEN
GOSUB ONE
```

```
ENDIF
```

The translator has never heard of `FOR`. Anything that can be said in the smaller language is better said there once, than handled again in every later stage. The same idea, much grown, is how Lisp macros and C++ templates work, and how languages like Kotlin and Swift lower their convenient syntax into a smaller core.

The rewritten tokens keep the positions of the tokens they came from, so an error inside a `FOR` is still reported at the `FOR`.

11.4 A run-time library

`PRINT X;` must print a number without ending the line, and the machine's `PRINT` always ends it. Rather than add an instruction, the compiler adds a routine, `print_number`, written in assembly, to any program that needs it. It prints digit by digit with `EMIT`.

ch11-basal/basal.py, lines 481–520

```
481 ; run-time library: print_number(n) prints n with no newline
482 print_number:
483         LOAD 0
484         PUSH 0
485         LT
486         JZ print_negate
487         PUSH 45                 ; '-'
488         EMIT
489         LOAD 0                  ; n is already negative
490         JMP print_digits_of
491 print_negate:
492         PUSH 0
493         LOAD 0
494         SUB                     ; -n
495 print_digits_of:
496         CALL print_digits 1
497         RET
498
499 ; print_digits(m), for m <= 0, prints the digits of -m
500 print_digits:
501         LOAD 0
502         PUSH -9
503         LT                      ; m < -9: more than one digit
504         JZ print_last
505         LOAD 0
```

```
        PUSH 10
        DIV
        CALL print_digits 1     ; all digits but the last
        DROP
print_last:
        PUSH 48                 ; '0'
        PUSH 0
        LOAD 0
        PUSH 10
        MOD
        SUB                     ; -(m mod 10), from 0 to 9
        ADD
        EMIT
        PUSH 0
        RET""".splitlines()
```

It does its arithmetic on the number made *negative*. Every 32-bit number has a negative partner, but −2147483648 has no positive one, so a routine that made numbers positive first would fail on exactly one value. `samples/extremes.bas` prints it.

Code that a compiler adds to programs, to provide what the machine does not, is a *run-time library*. C's `printf` is one; this is a very small one.

11.5 GOSUB on a machine with frames

`GOSUB L` becomes `CALL L 0`, and `RETURN` becomes `RET`. A subroutine is a call with no arguments and no locals. But BASIC lets a program `GOTO` out of a subroutine, or `RETURN` without having used `GOSUB`, and the compiler cannot see that coming, so it does not try. The machine catches it when it runs:

```
$ ./vm tests/return.vm
before
trap: return without call at 23
```

Chapter 4's traps, doing their job for a language that did not exist when they were written.

11.6 Tests

```
make test
```

- every sample against its expected output,
- ten programs with mistakes against their error messages,
- `RETURN` without `GOSUB`: compiles, then traps.

11.7 Exercises

1. Add `INPUT X`. The machine cannot read anything yet. Which instruction would you add, how would you specify it, and what should happen at the end of the input?
2. `FOR` evaluates its limit again on every round, because the desugared `WHILE` does. Make it evaluate the limit once. (Hint: a hidden variable.)
3. Add `DIM A(10)` and `A(I)` for arrays. Can it be done entirely by desugaring? What does the translator need to know?
4. Compile the same program, say gcd, in enkel/0 and in basal/0, and compare the assembly. Which is shorter? Why?

Chapter 12

Three ways to parse

One language parsed three ways, by recursive descent, by Pratt parsing and by parser combinators, and proved to build the same trees.

Chapters 9 to 11 all parsed by *recursive descent*: one function per grammar rule. It is the most common way to write a parser by hand, and not the only one. This chapter parses one language three ways, and makes them prove they agree.

The language is chapter 9's, with two comparisons added at the loosest precedence:

ch12-parsing/samples/compare.expr

```
# Comparisons bind more loosely than arithmetic, and give 1 or 0.

1 < 2;
2 + 3 * 7 < 30;
2 + 3 * 7 = 23;
(1 < 2) + (2 < 1) + (3 = 3);
100 - 10 - 1;
-2 * 3 < -5;
```

12.1 The three parsers

`descent.py`: recursive descent. One function per precedence level, each calling the next tighter one. The comparisons cost a new grammar rule and a new function on top of the ladder.

`pratt.py`: Pratt parsing. Vaughan Pratt's 1973 idea, *top down operator precedence*. There is no ladder of functions, only a table of *binding powers* and one loop:

ch12-parsing/pratt.py, lines 31–35

```
31 INFIX = {
32     "<": 10, "=": 10,
33     "+": 20, "-": 20,
34     "*": 30, "/": 30, "%": 30,
35 }
```

To parse an expression, take an operand, then keep taking operators, but only those that bind more tightly than whatever asked for the expression:

ch12-parsing/pratt.py, lines 63–74

```
63     def expression(self, min_power):
64         """Parse an expression, taking only operators that bind
   ↪ more
65         tightly than min_power."""
66         left = self.prefix()
67         while True:
68             token = self.peek()
69             power = INFIX.get(token.kind)
70             if power is None or power <= min_power:
71                 return left
72             self.take(token.kind)
73             right = self.expression(power)          #
   ↪ left-associative: stop at equal power
74             left = Binary(token.kind, left, right, token)
```

The comparisons cost two lines in the table. Pratt parsers are what many modern compilers use for expressions, because languages with fifteen precedence levels, like C, would otherwise need fifteen functions.

`combinators.py`: parser combinators. A parser is a value: a

function from a position in the tokens to what it found there. Bigger parsers are built from smaller ones by ordinary functions: `seq`, `alt`, `many`. The grammar is then written down almost as it reads:

ch12-parsing/combinators.py, lines 134–138

```
134 product = chain_left(unary, "*/%")
135 total = chain_left(product, "+-")
136 comparison = chain_left(total, "<=")
137 statement = apply(seq(comparison, token(";")), lambda v: v[0])
138 program = apply(seq(many(statement), token("end")), lambda v: v[0])
```

This is the idea of the earlier `archive/parse`, here working on tokens and building the same trees as the other two.

All three share the scanner, the tree and the code generator, in `common.py`. Only parsing differs.

12.2 Proving they agree

Each tree can be written as an *s-expression*, a line of text that shows its shape: `2 + 3 * 7 < 30` is `(< (+ 2 (* 3 7)) 30)`.

```
$ python3 compile.py -pratt -tree samples/compare.expr
(< 1 2)
(< (+ 2 (* 3 7)) 30)
...
```

The test takes 500 random expressions from the fuzzer of chapter 9, now with comparisons, and checks with `same.py` that all three parsers build exactly the same 500 trees. It then checks that the three produce byte-for-byte the same assembly, and that the program prints the right values.

A Pratt parser with one mistake in it, making `-` group from the right, fails at the second statement:

```
statement 2: descent and pratt differ
  (/ (- (% (neg ...
  (/ (- (% (neg ...
```

12.3 Where they differ: error messages

Given the same mistakes, the parsers say different things:

```
-descent:     tests/two.expr:1:3: expected ';', found '2'
-pratt:       tests/two.expr:1:3: expected ';', found '2'
-combinators: tests/two.expr:1:3: expected '%' or '*' or '+' or
  ↪ '-'
              or '/' or ';' or '<' or '=', found '2'
```

The combinator parser is *right*: any of those tokens could have come next. But it is less helpful. A hand-written parser knows what the writer of a program most likely meant; a parser built mechanically from a grammar knows only what the grammar allows. The usual fix, used here, is to report the failure that got *furthest* into the input, which finds the place of the mistake reliably. Saying what the mistake was, well, takes knowledge that is not in the grammar.

12.4 Speed

On one computer, parsing about 82,000 tokens: recursive descent 60 ms, Pratt 54 ms, combinators 142 ms. Combinators pay for their flexibility in function calls. For a compiler this rarely matters; for a parser that runs on every keystroke in an editor, it can.

12.5 Which to use?

- **Recursive descent** for whole languages, when you want the best error messages and full control. Most production compilers, including GCC's and Clang's C front ends, are hand-written recursive descent.
- **Pratt** for expressions, inside a recursive descent parser for everything else. This is a very common combination.
- **Combinators** when the grammar changes often, or must be built while the program runs, or in functional languages where they are natural: Haskell's Parsec is the classic.

A fourth family is missing here: *generated* parsers, from tools like

yacc, Bison or ANTLR, which read a grammar and write the parser for you. They are the descendants of the table-driven ideas behind the Metcalfe machine of appendix A, and they deserve a chapter of their own.

12.6 Run it

```
make test
```

12.7 Exercises

1. Add `^` for powers, right-associative, to all three parsers. Which took the fewest changes?
2. Add a conditional expression `a ? b : c` at the loosest precedence. In the Pratt parser, `?` is an infix operator that parses two more expressions. Where does the `:` get checked?
3. Make the combinator parser's error messages better without writing any special cases, using only information in the grammar. (Hint: label the parsers: `comparison` could be called "an expression".)
4. Write a fourth parser: the *shunting-yard* algorithm of Edsger Dijkstra, which uses two stacks instead of recursion. Add it to `same.py`.

Part IV

The run-time

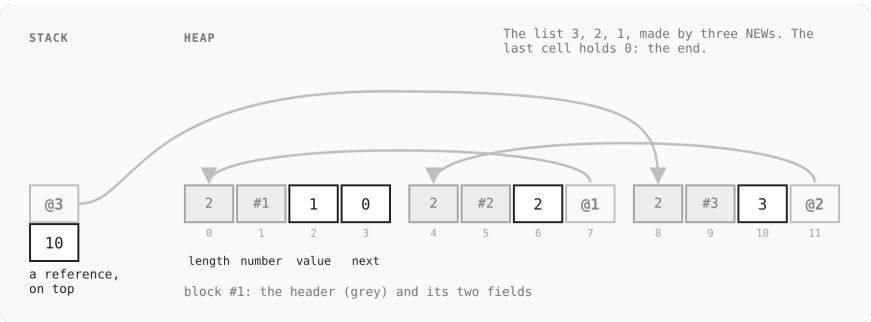
Chapter 13

A heap

Blocks of any size, made whenever a program needs one, and values that know whether they are numbers or references to blocks.

Part IV is about the *run-time*: what the machine does for a program beyond running its instructions. It starts with memory.

So far a program has had two places to keep values. The stack holds values for as long as a call lasts. Memory, since chapter 10, holds 4096 values for as long as the program runs. Neither fits a string whose length is known only when the program runs, or a list that grows, or a record made in one procedure and handed back to another. For those a program needs a *heap*: blocks of any size, made whenever needed, that live for as long as something refers to them.



13.1 Four instructions

Code	Name	Effect
27	NEW	n → r: a new block of n fields, all 0
28	GETF	r i → field i of r
29	SETF	r i v → ; field i of r = v
30	LEN	r → the number of fields of r

A string is a block of character codes. A list cell is a block of two fields, a value and the next cell. A record is a block whose fields mean whatever the program says they mean. The machine does not care; see `samples/strings.asm` and `samples/lists.asm`.

13.2 Values that know what they are

This is the biggest change to the machine since chapter 1. Until now a value was a number. Now a value is *either* an integer *or* a reference to a block, and it carries a tag that says which:

ch13-heap/vm.c, lines 58–61

```
58 struct value {
59     int is_ref;
60     int n;
61 };
```

The tag costs memory: every value is now twice the size. It buys two things.

Safety. Only `NEW` makes references. There is no instruction that turns a number into a reference, so a program cannot point anywhere it was not given. And `GETF` and `SETF` check every index:

ch13-heap/vm.c, lines 193–198

```
193 /* Where field i of the block at b is, checking that there is one.
    ↪ */
194 static int field(int b, int i) {
195     if (i < 0 || i >= heap[b].n)
196         trap("index out of bounds");
197     return b + HEADER + i;
```

```
198 }
```

Remember the sample in chapter 10 that wrote one element past the end of an array, and nothing noticed? `samples/bounds.asm` makes the same mistake on the heap:

```
$ ./vm samples/bounds.vm
trap: index out of bounds at 21
```

Garbage collection. A block that nothing refers to any more can never be used again, and its space can be given to a new block. To find such blocks, a collector must find every reference in the machine, and be sure that what it found *is* a reference and not a number that happens to look like an address. With tags, it can be sure. That is chapter 14.

Doing arithmetic on a reference, or printing one, traps: *not a number*. Comparing two references with `EQ` asks whether they refer to the *same* block, not whether two blocks hold the same values.

13.3 Seeing references

A reference has no address a program can see. In traces it is shown by the number of its block: `@1` for the first block ever made, `@2` for the second, and so on.

```
32    27  NEW             | 9 @1 | 9 @1 @2
33    28  DUP             | 9 @1 | 9 @1 @2 @2
```

This is a deliberate choice. Addresses would differ between the C machine and the Python one, and in chapter 14 a garbage collector will *move* blocks to new addresses while the program runs. Block numbers never change, so traces can still be compared, line by line, across machines and collectors.

13.4 Specification version 3

`SPEC.md` now says what a value is, what a block is, which instructions accept which kind of value, and in what order everything is checked.

It also says something this chapter's machines do *not* do:

> A block that is not reachable can never be used again, and a machine **must** reclaim the space it takes. … a program that keeps only a small amount reachable must be able to make blocks forever.

The machines here never reclaim anything. Each new block goes after the last one, until there is no room left:

ch13-heap/vm.c, lines 175–191

```
175 static struct value allocate(int n) {
176     struct value r;
177     int i;
178 
179     if (n < 0)
180         trap("bad size");
181     if (n > HEAP_SIZE - HEADER - heap_used)
182         trap("out of memory");
183     heap[heap_used] = integer(n);
184     heap[heap_used + 1] = integer(++blocks_made);
185     for (i = 0; i < n; i++)
186         heap[heap_used + HEADER + i] = integer(0);
187     r.is_ref = 1;
188     r.n = heap_used;
189     heap_used += HEADER + n;
190     return r;
191 }
```

The suite's garbage test makes 100,000 blocks and keeps none of them, and both machines fail it:

```
  FAIL garbage
  ./vm: 65 of 66 passed
    garbage: a known failure
```

`conform.py` now accepts `--known garbage`: a failure documented here does not fail the build, but it is still run and still reported every time. If it ever passes, that is reported too, since then this documentation would be wrong. A known failure that is written down is honest; one that is quietly skipped is not.

The Python machine needs no collector to keep its own memory in check, since Python reclaims its `Ref` objects by itself. But it counts

heap space exactly as the C machine does, so that both run out of memory at the same instruction and their traces still agree.

13.5 Run it

```
make test
```

13.6 Exercises

1. Write `concat(a, b)`, which makes a new string holding `a` then `b`. Then print "hello, world" with it.
2. A record for a point could be a block of two fields, `x` and `y`. Write a procedure that takes two points and returns their sum as a new point.
3. Why does `NEW` fill a new block with zeros, rather than leave whatever was in that space before? What could go wrong otherwise? (Think about the tags.)
4. Every value is now twice as big. Find a way to tag values that costs no extra space. (Hint: real machines often steal the lowest bit. What does that do to the range of integers?)

Chapter 14

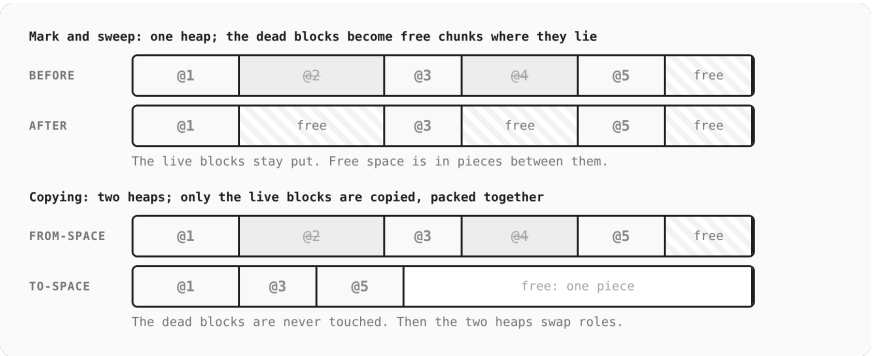
Garbage collection

The machine gives back what a program can no longer reach, with two garbage collectors, mark and sweep and copying, measured side by side and proved invisible.

Chapter 13's machine made blocks and never gave any back. Its heap filled up after a few thousand, and it failed the one test in the suite that asked it to go on making blocks forever while keeping almost none. This chapter's machine passes it, with two different garbage collectors.

```
./vm -g mark program.vm       mark and sweep (the default)
./vm -g copy program.vm       copying
```

The question both answer is the same: *which blocks can the program still reach?* A block is reachable if a reference to it is on the stack, in memory, or inside another reachable block. Every other block is garbage: the program has no way to use it again, so its space can be used for something new. Chapter 13's tags make this possible: the collector can be sure which values are references.



14.1 Mark and sweep

John McCarthy's collector for Lisp, 1960, and the one the old `archive/vm5` began. Two phases:

1. **Mark.** Start from the *roots*, the stack and memory, and mark every block they refer to, then every block *those* refer to, and so on. The machine keeps a list of marked blocks still to look inside, instead of calling itself recursively, so a long linked list cannot overflow the C stack.
2. **Sweep.** Walk the whole heap from end to end. A marked block is kept (and unmarked, ready for next time). Anything else becomes free space, and free space next to free space is merged into one chunk.

ch14-gc/vm.c, lines 235–252

```
235 /*
236  * Mark everything reachable. The roots are the stack and memory:
      ↪ the
237  * only places a reference can be, apart from inside blocks.
238  */
239 static void mark(void) {
240     int i, b;
241
242     mark_depth = 0;
243     for (i = 0; i < sp; i++)
244         mark_value(stack[i]);
245     for (i = 0; i < MEMORY_SIZE; i++)
246         mark_value(memory[i]);
```

```
247     while (mark_depth > 0) {
248         b = mark_stack[--mark_depth];
249         for (i = 0; i < heap[b].n; i++)
250             mark_value(heap[b + HEADER + i]);
251     }
252 }
```

New blocks come from a *free list* of chunks, by *first fit*: the first chunk big enough. The blocks never move, which is simple, and is also the weakness: the free space ends up in pieces between live blocks, and a big block may not fit anywhere even when there is enough free space in total. That is *fragmentation*.

14.2 Copying

C. J. Cheney's algorithm, 1970. The heap is split into two halves, and only one is used at a time. When it is full:

1. Copy every block the roots refer to into the other half, packed together, and leave a *forwarding address* in each old block: where its copy is now.
2. Go through the copies from the start. For each reference in them: if the block it refers to has already been copied, use the forwarding address; if not, copy it now, at the end. When the scan catches up with the end, everything reachable has been copied.
3. Swap the halves.

ch14-gc/vm.c, lines 329–350

```
329 /*
330  * Copy the block a reference refers to into to-space, unless it
      ↪ has
331  * been copied already, and return the reference to the copy. The
      ↪ old
332  * block is left with a forwarding address: where its copy is now.
333  */
334 static struct value forward(struct value v) {
335     int b, words;
336
337     if (!v.is_ref)
338         return v;
```

```
339     b = v.n;
340     if (heap[b].is_ref != FORWARDED) {
341         words = heap[b].n + HEADER;
342         memcpy(&to_space[to_top], &heap[b], words * sizeof (struct
    ↪ value));
343         words_copied += words;
344         heap[b].is_ref = FORWARDED;
345         heap[b].n = to_top;
346         to_top += words;
347     }
348     v.n = heap[b].n;
349     return v;
350 }
```

ch14-gc/vm.c, lines 352–378

```
352 /*
353  * Cheney's algorithm, 1970. Copy what the roots refer to. Then go
354  * through to-space from the start, copying what each copied block
355  * refers to and fixing its fields, until the scan catches up with
356  * the end of what has been copied. To-space then holds exactly the
357  * reachable blocks, packed together, and is the heap from now on.
358  */
359 static void copy(void) {
360     int i, scan = 0;
361
362     to_space = heap == space_a ? space_b : space_a;
363     to_top = 0;
364     for (i = 0; i < sp; i++)
365         stack[i] = forward(stack[i]);
366     for (i = 0; i < MEMORY_SIZE; i++)
367         memory[i] = forward(memory[i]);
368     while (scan < to_top) {
369         int n = to_space[scan].n;
370         for (i = 0; i < n; i++)
371             to_space[scan + HEADER + i] = forward(to_space[scan +
    ↪ HEADER + i]);
372         scan += n + HEADER;
373     }
374     heap = to_space;
375     top = to_top;
376     if (top > most_live)
377         most_live = top;
378 }
```

Garbage is never touched, not even looked at. The free space is always one piece, so new blocks just go at the end. The cost is that

half the heap is always empty, and that every live block is copied, every time.

14.3 Which is better?

`samples/trees.asm` is the classic benchmark: 200 short-lived binary trees, 1.6 million places allocated in a heap of 65,536, while one long-lived tree stays alive throughout. Changing the size of the long-lived tree:

Long-lived tree	Collections	Mark and sweep: places freed	Copying: places copied
8,188 places	28	1,604,848	230,160
16,380 places	33	1,621,224	541,464
32,764 places	49	1,604,848	1,606,416

(From `./vm -s`, which prints these statistics on exit.)

Mark and sweep does work in proportion to the *garbage*: its sweep frees about the same amount, however much is alive. Copying does work in proportion to what is *alive*: with little live data it barely works at all, and as the live data grows, its work grows with it, until at half the heap it does as much as mark and sweep.

That is the whole trade-off, and it is why real collectors are *generational*: most blocks die young, so copy the young ones, where little survives, and leave the old ones, where most do, to a collector that does not copy. The JVM, .NET and V8 all work this way.

14.4 Testing a collector

A garbage collector that misses one root does not fail at once. It frees a block that is still in use, the space is given to a new block, and much later the program reads something that makes no sense. In the meantime most tests pass, since in a small program a collection may never even happen.

So the machine has a *stress mode*, `-S`: collect before every single `NEW`. Under copying, that means every live block moves every time, and a reference the collector failed to update is found at once.

To check that this works, the copying collector was broken on purpose, by leaving out the loop that updates references in memory. Without stress, it passed all 66 tests. With stress:

```
FAIL heap-places
  expected: '7\n7\n1\nstatus 0\n'
  got:      '7\nstatus 1\ntrap: not a number at 36\n'
./badgc -g copy -S: 65 of 66 passed
```

The test suite runs both collectors with and without stress.

14.5 Invisible, and proved so

A collector must change nothing that a program can see. Traces show blocks by number, not by address, so they can say so precisely: the test compares the trace of every suite program under mark and sweep against copying with stress, copying against mark and sweep with stress, and the C machine against the Python one. Every step, every stack, must be the same. They are.

The Python machine has no collector of its own: Python frees its `Ref` objects by itself. But when its count says the heap is full, it counts what is still reachable, which is what the C machine's collectors would find, so it runs out of memory at exactly the same instruction.

14.6 Run it

```
make test
```

14.7 Exercises

1. Add *next fit* to the mark and sweep allocator: start each search where the last one ended. Does it make `trees` faster or slower?
2. Write a program that makes mark and sweep run out of memory while copying, with the same amount of live data, does not. (Think about fragmentation.)
3. The mark phase looks at all 4096 words of memory on every collection, even in a program that uses none of it. How could

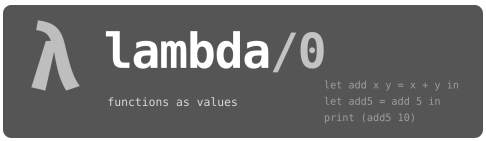
the machine avoid that?

4. Make the copying collector *generational*: a small nursery for new blocks, copied often, and an old space, collected rarely. What happens when an old block is changed to refer to a young one? (This is the *write barrier*, and it is the hard part.)

Chapter 15

Closures

Functions as values: made while the program runs, carrying the variables they need in a block on the heap, and called through one new instruction.



The last chapter of Part IV puts the heap and the garbage collector to work for a new kind of language, one where functions are values:

```
let add x y = x + y in
let add5 = add 5 in           # add, with x fixed at 5
print (add5 10);              # 15

let twice f x = f (f x) in
print (twice add5 0)          # 10
```

`add5` is a function that did not exist when the program was written. It was *made*, while the program ran, out of `add` and the number 5. `twice` takes a function as its argument. Neither enkel/0 nor basal/0 can do either of these things. A procedure there is a fixed place in the code, and the only thing to do with it is call it.

15.1 The language: lambda/0

A small functional language in the style of ML, compiled by `lambda.py`:

- everything is an expression with a value;
- `let x = e in body` binds a name, which never changes;
- `fun x -> e` is a function, and `let f x = e` is short for `let f = fun x -> e`;
- `let rec` lets a function call itself;
- `if c then a else b`, arithmetic, comparisons, and `print e`.

Every function takes exactly one argument. `fun x y -> e` is short for `fun x -> fun y -> e`, and `add 5 10` means `(add 5) 10`: `add` given 5 returns a function that is waiting for the second number. This is *currying*, and it means a function can never be called with the wrong number of arguments: there is only ever one.

15.2 What a closure is

`fun x -> x + n` uses `n`, which is not its own argument. If the function is called after the `let` that bound `n` has finished, where does `n` come from? The function must have taken it along. So a function value is not just the address of its code, but a block on the heap:

```
field 0        the address of the code
field 1 ...    the values of the variables it uses from outside
```

That block is a *closure*: the code, closed over its free variables.

```
; let addn = fun x -> x + n
        PUSH 2
        NEW                     ; a block of two fields
        DUP
        PUSH 0
        PUSH fun_1
        SETF                    ; field 0: the address of the code
        DUP
        PUSH 1
        LOAD 0
        SETF                    ; field 1: the value of n, now
```

The code itself finds its argument as local 1, and the closure it was called through as local 0, so `n` is field 1 of local 0:

```
; fun x -> ..., capturing: n
fun_1:  LOAD 1                  ; x
        LOAD 0
        PUSH 1
        GETF                    ; n, from the closure
        ADD
        RET
```

15.3 Closure conversion

Working out which variables each function must capture is *closure conversion*, and it is the new stage in this compiler. A variable is *free* in an expression if the expression uses it without binding it. `free()` in `lambda.py` computes it in a dozen lines, one rule per kind of expression:

ch15-closures/lambda.py, lines 245–268

```
245 def free(node):
246     kind = node[0]
247     if kind == "num":
248         return set()
249     if kind == "var":
250         return {node[1]}
251     if kind in ("neg", "print"):
252         return free(node[1])
253     if kind == "op":
254         return free(node[2]) | free(node[3])
255     if kind == "if":
256         return free(node[1]) | free(node[2]) | free(node[3])
257     if kind == "app":
258         return free(node[1]) | free(node[2])
259     if kind == "fun":
260         return free(node[2]) - {node[1]}
261     if kind == "let":
262         _, name, value, body, recursive, _ = node
263         if recursive:
264             return (free(value) | free(body)) - {name}
265         return free(value) | (free(body) - {name})
266     if kind == "seq":
267         return set().union(*(free(e) for e in node[1]))
```

```
268     raise ValueError(kind)
```

The free variables of `fun x -> x + n` are `{n}`.

The values are *copied* into the closure when it is made. In `samples/closures.lam`, `addn` keeps the `n` it saw, even after a new `n` is bound:

ch15-closures/samples/closures.lam

```
1  # Functions are values: they can be made, passed, returned and
     ↪ kept.
2
3  let add x y = x + y in
4  let add5 = add 5 in                # add, waiting for its second
     ↪ argument
5  print (add5 10);                   # 15
6
7  let twice f x = f (f x) in
8  print (twice add5 0);              # 10
9
10 let compose f g x = f (g x) in
11 let times3 x = 3 * x in
12 print (compose add5 times3 2);     # 5 + 3 * 2 = 11
13 print (compose times3 add5 2);     # 3 * (5 + 2) = 21
14
15 # A closure remembers the value a variable had when it was made.
16 let n = 100 in
17 let addn = fun x -> x + n in
18 let n = 0 in                       # a new n; addn still has the
     ↪ old one
19 print (addn 1)                     # 101
```

Since lambda/0 variables never change, copying is safe. In a language whose variables can change, two closures that share a variable must share one copy of it, and it must live on the heap. That is how JavaScript and Python do it.

A function that calls itself, `let rec fact n = … fact (n - 1)`, has itself as a free variable. Its closure is made first, and then a reference to the closure is stored in one of its own fields.

15.4 One new instruction

To call a closure, the machine must jump to an address that is not in the code, but in a value on the stack. That is `CALLI n`, for *call indirect*: the address is popped from the stack, then everything happens as for `CALL`. It is code 31, and the whole of version 4 of `SPEC.md`.

Code	Name	Operand	Effect
31	`CALLI`	`n`	`addr` → ; then as `CALL addr n`

A call `f a` is:

```
      (f)                       ; the closure
      (a)                       ; the argument
      OVER
      PUSH 0
      GETF                      ; field 0 of the closure: the
↪ address
      CALLI 2                   ; locals 0 and 1: the closure and a
```

The assembler can now write `PUSH fun_1`, pushing the address of a label as a number. A function pointer, in C's terms.

15.5 Safety, from the tags

lambda/0 has no types to check before running. The machine checks as it goes. Calling a number does `GETF` on something that is not a block; printing a function asks `PRINT` for a number and gets a reference:

```
$ ./vm tests/callnumber.vm
trap: not a reference at 9
$ ./vm tests/printfun.vm
trap: not a number at 12
```

Chapter 13's tags were added for the garbage collector. They also make the machine safe for a language that checks nothing itself.

15.6 Garbage, at last

Every `fun`, and every partial application, makes a block. Most are needed for a moment. `samples/garbage.lam` makes about 65,000 closures, and keeps fewer than 80 places' worth at any time:

```
$ ./vm -g copy -s samples/garbage.vm
16384
gc: 65569 blocks made, 262305 places in all
gc: 4 collections, most live after one: 79 places
gc: 316 places copied
```

This is the program chapter 14's copying collector was made for: nearly everything dies young.

15.7 Data from nothing but functions

`samples/data.lam` builds pairs and lists using only functions. A pair is a function that hands its two parts to whatever it is given; a list is a function that folds its elements. Alonzo Church showed in the 1930s that functions alone are enough for any data, and here it runs on a stack machine.

ch15-closures/samples/data.lam

```
1  # Data made of nothing but functions.
2  #
3  # A pair is a function that hands its two parts to whatever
     ↪ function
4  # it is given. A list is a function that folds: given f and a start
5  # value z, it combines its elements with f, from the right.
6  # (Alonzo Church showed in the 1930s that this is enough for
     ↪ anything.)
7
8  let pair a b = fun f -> f a b in
9  let first p = p (fun a b -> a) in
10 let second p = p (fun a b -> b) in
11 let p = pair 3 4 in
12 print (first p * 10 + second p);                    # 34
13
14 let nil = fun f z -> z in
15 let cons h t = fun f z -> f h (t f z) in
16 let rec range n = if n = 0 then nil else cons n (range (n - 1)) in
```

```
let sum list = list (fun h acc -> h + acc) 0 in
let length list = list (fun h acc -> acc + 1) 0 in
let map g list = fun f z -> list (fun h acc -> f (g h) acc) z in
let numbers = range 20 in
print (sum numbers);                                  # 210
print (length numbers);                               # 20
print (sum (map (fun x -> x * x) (range 10)))         # 385
```

15.8 A limit: 256 frames

lambda/0 has no loops. Repetition is recursion, and every call takes a frame and a few values on a stack of 256. Folding a list of 100 elements runs out. The samples stay small; `garbage.lam` repeats a quarter of a million times only by *doubling* (`power k f` applies `f` 2^k times while nesting `k` calls deep).

Real functional languages handle this with *tail calls*: when the last thing a function does is call another, the new call can reuse the old frame instead of taking a new one, and a loop written as recursion runs in constant space. That is exercise 1.

15.9 Run it

```
make test
```

15.10 Exercises

1. Add `TAILCALLI n` to the machine: like `CALLI`, but it replaces the current frame instead of adding one. Specify it, implement it in both machines, and make the compiler use it for calls in tail position. Then count to a million.
2. Let functions take several arguments at once, without currying. What must a call check now, and where?
3. Add `let rec … and …` for two functions that call each other. Each closure must hold the other. In what order are they made?
4. A closure for `fun x -> x + 1` captures nothing, yet the compiler makes a new block every time the `fun` is evaluated. Make

the compiler build such closures only once. Where can it keep them?

Part V

Further

Chapter 16

Dispatch

The loop at the heart of the machine, built three ways, with a switch, with computed goto and as threaded code, and measured.

Part V is about making the machine better at being a machine: faster, safer, closer to the hardware. It starts with the loop at its heart.

Every chapter so far has run instructions the same way:

```
for (;;) {
    op = code[pc++];
    switch (op) {
    case ADD: ...; break;
    case SUB: ...; break;
    ...
    }
}
```

Getting from one instruction to the next (fetch the opcode, check it, find its code, jump there) is called *dispatch*. Each `ADD` does one addition and then all of that. In a simple interpreter, dispatch can be as much work as the instructions themselves, so it is the first place people look for speed.

16.1 One set of instructions, three loops

`vm.c` is chapter 15's machine, with its loop rewritten so it can be built three ways. The instructions are written once, in `ops.h`:

ch16-dispatch/ops.h, lines 17–21

```
17 OP(ADD) {
18     struct value w = pop(), v = pop();
19     push_int(wrap_add(number(v), number(w)));
20     NEXT;
21 }
```

and `OP` and `NEXT` are macros whose meaning depends on the build. CPython's and Lua's interpreters are organised exactly like this.

vm-switch. `OP(ADD)` is `case ADD:`, and `NEXT` goes back to the top of the loop. The compiler makes the switch into a jump through a table, after checking that the opcode is in range. All instructions share that one jump.

vm-goto. `OP(ADD)` is a label, `L_ADD:`, and `NEXT` fetches the next opcode and jumps straight through a table of label addresses:

```
goto *table[code[pc++]];
```

Every instruction now ends with its own jump. The processor's branch predictor can learn, separately for each one, what tends to come next: after a `LOAD`, often another `LOAD`; after `LT`, nearly always `JZ`. This is "labels as values", an extension of GCC and Clang, not standard C.

vm-threaded. Before the program runs, every word of it is replaced, in a second array, by the address of the label that runs it. `NEXT` is then:

```
goto *thread[pc++];
```

No table lookup, and no range check: a word that is not an instruction got the label of the code that traps. This is *direct threaded code*, the technique of Forth systems since the 1970s. Here are the three builds' macros, side by side in the source:

ch16-dispatch/vm.c, lines 502–544

```
#if DISPATCH == SWITCH

#define OP(x)       case x:
#define NEXT        continue
#define UNKNOWN     default:

static void run(void) {
    int op;

    for (;;) {
        start = pc;
        if (pc < 0 || pc >= code_len)
            trap("ran off the end of the program");
        op = code[pc++];
        switch (op) {
#include "ops.h"
        }
    }
}

#else

#define OP(x)       L_##x:
#define UNKNOWN     L_UNKNOWN:

#if DISPATCH == GOTO
#define NEXT \
    do { \
        start = pc; \
        if (pc < 0 || pc >= code_len) \
            trap("ran off the end of the program"); \
        op = code[pc++]; \
        if (op < 0 || op >= NOPS) \
            goto L_UNKNOWN; \
        goto *table[op]; \
    } while (0)
#else
#define NEXT \
    do { \
        start = pc; \
        goto *thread[pc++]; \
    } while (0)
#endif
```

A subtle point: *every* word gets a label, not just the words where instructions start. A program may jump into the middle of an in-

struction, and the specification says exactly what happens then, so every build must do it the same way.

16.2 They are the same machine

```
./vm-switch -g mark: 71 of 71 passed
./vm-switch -g copy -S: 71 of 71 passed
./vm-goto -g mark: 71 of 71 passed
./vm-goto -g copy -S: 71 of 71 passed
./vm-threaded -g mark: 71 of 71 passed
./vm-threaded -g copy -S: 71 of 71 passed
```

Speed is worth nothing if the answers change, and the conformance suite of chapter 8 makes checking that a single command.

16.3 How much faster?

Four programs in `bench/`: recursive `fib(30)`; a tight `loop` of 20 million rounds; a `sieve` on the heap; and `closures`, a compiled lambda/0 program from chapter 15. `make bench` runs each three times on each build and reports the best. On one computer, milliseconds:

	GCC 13, `-O2`			Clang 18, `-O2`		
	switch	goto	threaded	switch	goto	threaded
fib	137	97	86	76	83	57
loop	1072	883	839	654	703	553
sieve	77	93	72	66	57	52
closures	151	129	134	66	72	53

The measurements are noisy (this was a shared machine), but the pattern held across runs. Threaded code is usually fastest, by 10 to 25 percent. Computed goto sometimes helps and sometimes does not. And the choice of *compiler* made as much difference as the choice of technique.

The gains are smaller than the classic papers report, for two reasons. Processors since about 2013 predict the single jump of a switch far better than older ones did, which took away much of what computed goto used to win. And in this machine, dispatch is not the

biggest cost. Look at what `ADD` does besides adding: two calls to `pop`, each checking for underflow; two calls to `number`, each checking a tag; one call to `push`, checking for overflow; and all of it through memory. The next chapters go after those costs: fewer instructions (chapter 17), better instructions (chapter 18), and checks done once, before running, instead of on every step (chapter 19).

16.4 Run it

```
make test        # all three builds, against the suite and the
  ↪ benchmarks
make bench       # the timings
make CC=clang bench
```

16.5 Exercises

1. Count how often each instruction follows each other one in `bench/loop.asm`. Which pair is most common? Chapter 18 makes that pair into one instruction.
2. Build with `-O0`. How do the three compare now? Why?
3. Make `push`, `pop` and `number` macros, or `static inline`, and look at the generated assembly for `L_ADD` (`gcc -S`). How many instructions does one `ADD` take?
4. *Subroutine threading*: instead of jumping between labels, turn the program into a list of calls to one C function per instruction. Is it faster or slower than direct threading here?

Chapter 17

Registers

A register machine beside the stack machine, a translator from one to the other, and a measurement of which is faster.

A stack machine's instructions are small because they name nothing. `ADD` does not say what to add or where to put the result: the stack says. The price is that values must be moved onto the stack before they are used and off it afterwards. Here is one round of `bench/loop.asm`, thirteen instructions:

```
loop:   LOAD 0
        JZ done
        LOAD 1
        LOAD 0
        PUSH 7
        AND
        ADD
        STORE 1
        LOAD 0
        PUSH 1
        SUB
        STORE 0
        JMP loop
```

A *register machine* names its operands. Here is the same round:

```
S4:     JZ r0, S27
        ANDI r3, r0, 7
```

```
        ADD r1, r1, r3
        SUBI r0, r0, 1
        JMP S4
```

Five instructions. Each is bigger, but there are far fewer of them, and every one does real work. Lua, Dalvik (Android's original virtual machine) and the BEAM (Erlang's) are register machines; the JVM, .NET, CPython and WebAssembly are stack machines. Which is better was argued for decades, and this chapter measures it on our own machine.

17.1 From stack to registers

Nobody wrote the register code above. `translate.py` made it from the stack code, in two stages.

Stage one: slots become registers. In a well-behaved program, the stack is equally deep at an instruction whichever way the program got there. If the depth is known before running, "the top of the stack" at an instruction where the depth is 5 just means "slot 4", and slot k can be called register k:

```
LOAD 0      depth 2 -> 3      MOV  r2, r0
LOAD 1      depth 3 -> 4      MOV  r3, r1
ADD         depth 4 -> 3      ADD  r2, r2, r3
STORE 1     depth 3 -> 2      MOV  r1, r2
```

Working out the depths is a *data-flow analysis*. Start at address 0 with depth 0, and at each function, the target of a `CALL`, with depth equal to its number of arguments. Follow every path, adding up each instruction's effect on the depth. Where two paths meet, they must agree. This gives as many register instructions as there were stack instructions, minus the `DROP`s, which become nothing.

Stage two: remove the copying. Within each straight run of code:

- *Copy propagation*: after `MOV r2, r0`, a later read of r2 can read r0 directly, if neither has changed in between.
- *Constants*: after `MOVI r4, 7`, `AND r3, r3, r4` can be `ANDI r3, r3, 7`, with the 7 inside the instruction.

- *Dead code*: a `MOV` into a register nobody reads again can go. Which registers might still be read? The stack discipline answers for free: those below the depth at that point. Everything at or above it was popped.
- *Coalescing*: `ADD r2, r1, r3` followed by `MOV r1, r2` is `ADD r1, r1, r3`.

`translate.py -l` shows the result, and `-naive` skips stage two.

17.2 What is refused

Some programs have no register translation:

- a program where the depth at some point depends on the path taken (`tests/shared.asm`);
- code shared between two functions, which would need two window sizes;
- a jump into the middle of an instruction;
- `CALLI`: its target is a value computed while the program runs, so where functions begin cannot be known in advance.

Everything else is translated, *including* programs that are certain to trap. An `ADD` reached with one value in the frame becomes a `TRAP` instruction with the stack machine's message, and every register instruction carries the address of the stack instruction it came from. So traps report the same thing on both machines, and the conformance suite of chapter 8 tests the pair:

```
./rvm-run: 66 of 71 passed
  calli: a known failure
  garbage: a known failure
  trap-calli-address: a known failure
  trap-calli-args: a known failure
  trap-calli-reference: a known failure
```

The five known failures are the four `CALLI` programs, refused, and `garbage`: the register machine has chapter 13's heap, without a collector. Twelve real programs, compiled by the enkel/0 and basal/0 compilers of chapters 10 and 11, give exactly the same output on both machines.

17.3 The register machine

`rvm.c`. Each instruction is five numbers: an opcode, three operands, and the address it came from. Registers are numbered within a *window*, the current call's part of a register file of 256. A call `CALL f, window r2, size 4` starts the new window at the caller's r2, exactly where the arguments are, so passing arguments copies nothing. The result comes back in the first register of the window, which is the caller's r2. Lua does the same.

17.4 Measured

Instructions run, counted with `-c`, and time, best of three, milliseconds, built with Clang 18 `-O2`:

	Instructions run			Milliseconds		
	stack	naive	register	stack	naive	register
fib	26,925,370	26,925,370	14,808,955	76	75	41
loop	260,000,007	260,000,007	100,000,005	607	595	236
sieve	33,826,936	33,826,406	17,152,655	96	81	37

("Naive" is the register translation without stage two.) With GCC 13 the register machine wins by about the same factor, and the naive translation already gains more.

Half the instructions, and two to three times as fast. This matches the classic study, *Virtual Machine Showdown: Stack versus Registers* by Shi, Gregg, Beatty and Ertl (2005), which found about 47 percent fewer instructions and a clear gain in time, despite bigger code.

Part of the gain is not about registers at all. The register machine checks nothing about its stack while running: no underflow, no `LOAD` outside the frame. The translator proved, before running, that those cannot happen. That idea is worth a chapter of its own: chapter 19.

Not every program gains as much. basal/0 code keeps its variables in memory, through `LOADM` and `STOREM`, and registers do nothing for that: `basal-table` runs 22 percent fewer instructions, `enkel-bits` 48 percent fewer.

17.5 Run it

```
make test
make bench
```

17.6 Exercises

1. Copy propagation works only within a straight run of code. In `fib`, which copies does it miss because of that?
2. Add `JLT r1, r2, label`, a compare-and-jump, and make the translator use it for `LT` followed by `JZ`. How many instructions does `loop` run now?
3. The register machine needs no `DUP`, `SWAP` or `DROP` in its code. Count, in the samples, how many stack instructions only move values around. What fraction of a stack machine's work is that?
4. Let enkel/0 compile straight to register code, without the stack machine in between. What does the compiler need to decide that the stack machine decided for it?

Chapter 18

Optimising

Less work for the same program: an optimiser that rewrites programs into fewer instructions, and superinstructions that run two instructions for the cost of one dispatch, without changing what any program does.

Two ways to make the machine do less work for the same program. The first changes the *program*: an optimiser that rewrites it into fewer, cheaper instructions before it runs. The second changes the *machine*: superinstructions, which run two instructions for the cost of one dispatch. Neither changes what a program does, and the tests are mostly about proving that.

18.1 A peephole optimiser

`opt.py` reads assembly language, as the compilers of Part III write it, and writes it back improved. It looks at the program through a small window, a few instructions at a time:

Rule	Before	After
constant folding	`PUSH 2`, `PUSH 4`, `MUL`	`PUSH 8`
no effect	`PUSH a`, `DROP` (and `DUP`, `DROP`; `LOAD k`, `DROP`; `SWAP`, `SWAP`)	nothing
double negation	`LT`, `PUSH 0`, `EQ`, `PUSH 0`, `EQ`	`LT`
jump threading	`JMP L` where `L` is `JMP M`	`JMP M`
jump to next	`JMP L` where `L` is the next instruction	nothing
dead code	instructions no path reaches	nothing

It repeats until nothing changes, since one rewrite often makes room for another. Chapter 10 promised that the useless `PUSH 0, RET` after every enkel/0 `return` would go: it is dead code, and it goes. Constant folding does its arithmetic as the machine does:

ch18-optimise/opt.py, lines 73–82

```
73 def fold(op, a, b):
74     """a op b as the machine computes it, or None if it would
   ↪ trap."""
75     if op in ("DIV", "MOD"):
76         if b == 0:
77             return None
78         q = abs(a) // abs(b)
79         q = q if (a < 0) == (b < 0) else -q
80         return wrap(q) if op == "DIV" else wrap(a - b * q)
81     return wrap({"ADD": a + b, "SUB": a - b, "MUL": a * b, "AND":
   ↪ a & b,
82                  "OR": a | b, "XOR": a ^ b, "LT": int(a < b),
   ↪ "EQ": int(a == b)}[op])
```

18.1.1 Changing nothing, including the failures

The hard part is not the rewriting but knowing when it is allowed. `PUSH 0, DROP` looks like it does nothing. On a full stack, though, the `PUSH` traps with *stack overflow*, and without it, nothing does. The optimised program would behave differently, in exactly the case where behaving correctly matters most.

So the window rules are used only where chapter 17's analysis, here in `flow.py`, says how deep the stack is. Then the optimiser can see that a `PUSH` has room, and that no instruction in the window is

one that is certain to trap. And it assumes nothing about values: `x + 0` is *not* simplified to `x`, because if `x` is a reference, the `ADD` traps.

For a program the analysis cannot handle, one that uses `CALLI`, only jump threading and dead code removal are done; they need no depths. A program that uses numbers for code addresses (`JMP 17`) is left alone, since moving code would make those numbers wrong. The compilers always use labels.

Addresses do change, so a trap names a different address. Everything else must be the same, and `same.sh` checks it for every program in the conformance suite, every sample from enkel/0, basal/0, lambda/0 and assembly, and every benchmark: 91 programs. That is a strong test of constant folding in particular. The suite's `wrap` and `division` programs fold down to nothing but constants, so the optimiser's own arithmetic, wrapping at 32 bits and rounding towards zero, must agree with the machine's, case by case.

18.2 Superinstructions

Chapter 16 counted which instructions most often follow which. Across the benchmarks and samples:

Pair	Share of all pairs run
`LOAD, PUSH`	9.6 %
`LOAD, LOAD`	6.0 %
`LT, JZ`	4.1 %
`PUSH, SUB`	3.1 %
`PUSH, LOADM`	2.9 %
`ADD, STORE`	2.9 %
`PUSH, ADD`	2.3 %
`LOAD, ADD`	2.1 %

The threaded machine of chapter 16 now has a *superinstruction* for each of these pairs, in `super.h`: one handler that does both, with one dispatch instead of two. When the threaded code is built, the first instruction of each such pair gets the superinstruction's label. The second keeps its own label, so a jump straight to it still works.

ch18-optimise/super.h, lines 1–18

```
/*
 * Superinstructions: each one is two instructions of ops.h, one
   ↪ after
 * the other, with a single dispatch. Between the two, start moves
   ↪ on
 * to the second instruction, so that a trap there names the right
 * address, and pc steps over the second one's opcode.
 *
 * Where the first instruction leaves a value that the second at
   ↪ once
 * takes, the value need not go through the stack at all, but then
   ↪ the
 * superinstruction must still trap where the pair would: a PUSH
   ↪ onto
 * a full stack must still say "stack overflow".
 */

L_LOAD_PUSH: {
    push(stack[local()]);               /* LOAD k */
    start = pc++;                       /* PUSH n */
    push_int(operand());
    NEXT;
}
```

Nothing in the specification changes. A superinstruction is not an instruction a program can contain, only a way this machine runs two of them. But the two halves must trap exactly as the two instructions would. `PUSH, ADD` need not put the constant on the stack at all, but on a full stack it must still say *stack overflow*, at the `PUSH`. And a trap in the second half must name the second instruction's address. `tests/super` has a program for each such case, including a jump into the middle of a pair.

18.3 Measured

Instructions dispatched, and time in milliseconds (best of five, Clang 18 `-O2`), without and with each technique:

	Instructions dispatched			
	plain	optimised	super	both
fib	26,925,370	26,925,370	18,847,760	18,847,760
loop	260,000,007	260,000,007	200,000,007	200,000,007
sieve	23,636,044	23,636,044	14,743,494	14,743,494
closures	26,477,794	26,477,794	24,380,514	24,380,514
enkel-sieve	56,905,612	56,905,612	41,765,809	41,765,809
basal-loop	14,399,563	13,199,563	11,999,631	10,199,631
	Milliseconds			
fib	62	62	53	56
loop	597	567	560	560
sieve	48	57	60	72
closures	72	54	65	66
enkel-sieve	101	99	92	99
basal-loop	30	26	26	19

Superinstructions cut dispatches by 8 to 38 percent. The optimiser does nothing for the hand-written benchmarks or for enkel/0's output, and folds away 8 percent of `basal-loop`, where the program said `2 * 4` and `10 / 5`.

The times do not tell a clean story, and that is the honest result. The "plain" and "optimised" columns for `fib` run the *same* program, so differences of 10 or 20 percent are noise on this machine. Beyond the noise, superinstructions help a little, sometimes. With GCC 13, `loop` even got slower with them: a bigger interpreter function can make the compiler's job harder, and that can cost more than the dispatches saved.

This agrees with chapter 16: on today's processors, dispatch is no longer the main cost of this machine. Fewer dispatches is only worth it when each one is expensive. What *is* expensive here is checking: every `ADD` checks two tags, every `pop` checks for underflow. The register machine of chapter 17 was twice as fast partly because it had proved, before running, that most of those checks were unnecessary. Chapter 19 does the same for the stack machine.

18.4 Run it

```
make test
make measure
```

18.5 Exercises

1. Add a rule for `STORE k`, `LOAD k`, which is common in enkel/0's output. Is `DUP`, `STORE k` really better? Measure it.
2. Choose superinstructions automatically: profile a set of programs, pick the 8 most common pairs, and generate `super.h` from them.
3. Add superinstructions of *three* instructions, such as `LOAD`, `PUSH`, `ADD`. How many dispatches do they save on `loop`?
4. The optimiser cannot fold `x * 2 * 3`, because the multiplication by 2 comes first. What would it need to know to rewrite it as `x * 6`? Is that rewrite even correct, with wrapping arithmetic?

Chapter 19

Verify once, then trust

A verifier examines a whole program once, when it is loaded, and a program that passes runs on a copy of the machine with those checks taken out.

Since chapter 4, every instruction checks, every time it runs, that nothing has gone wrong: that a pop finds a value in its frame, that a `LOAD` names a local that exists, that a jump lands inside the program, that an instruction has all its operands. For nearly every program, nearly all of these checks can never fail. They are paid for anyway, millions of times.

Chapter 17's register machine showed another way. Its translator proved, before running, that the stack could never underflow, and so the register machine never checked. This chapter brings the idea back to the stack machine, the way the Java Virtual Machine does it: a *verifier* examines the whole program once, when it is loaded. A program that passes runs on a copy of the machine with those checks taken out.

19.1 The verifier

`verify()` in `vm.c` follows every path through the program, from address 0 and from the start of every function, counting how many

values the current frame holds. It refuses the program if, on any path:

- an instruction is unknown, incomplete, or not where an instruction begins when the program is read from the start;
- a jump or call goes outside the program, or the program could run off its end;
- an instruction pops more values than the frame holds, or the frame could hold more values than the stack has room for;
- `LOAD` or `STORE` names a local that the frame does not have;
- a `CALL` passes more values than the frame holds, or one function is called with different numbers of arguments;
- two paths meet with different numbers of values, or one piece of code belongs to two functions;
- `RET` appears outside a function;
- `CALLI` is used: its target is a value, not known before running.

Every rule is about the *program*, not about any one run of it. So a program that passes can never fail any of the checks it covered, on any input, however it runs.

```
$ ./vm -V suite/trap-underflow.vm
trap-underflow.vm: not verified: at 2: a pop from a frame with
  ↪ too few values
```

`tests/verdicts.txt` lists the verdict on every program in the conformance suite. Every ordinary program verifies, except `calli`. Nearly every `trap-` program is refused, which is right: each was written to fail in one of these ways.

19.2 Two machines from one

The instructions are still written once, in `ops.h`, and the run loop once, in `loop.h`. `vm.c` includes them twice. The first time, `pop()` checks for underflow, `operand()` checks that the operand exists, and so on. Before the second time, those are redefined to check nothing, and a few checks inside instructions, written `CHECK(condition, message)` in `ops.h`, become empty:

ch19-verify/vm.c, lines 523–563

```
523 #define RUN run_checked
524 #include "loop.h"
525 #undef RUN
526
527 /*
528  * In a verified program: every pop finds a value in its frame,
      ↪ every
529  * instruction has its operands, every jump stays inside the
      ↪ program,
530  * and every LOAD and STORE names a local that exists.
531  */
532 static inline struct value pop_verified(void) {
533     return stack[--sp];
534 }
535
536 static inline int operand_verified(void) {
537     return code[pc++];
538 }
539
540 static inline int local_verified(void) {
541     return fp + code[pc++];
542 }
543
544 /*
545  * These must be functions, not expressions put in by a macro: in
546  * "pc = address();", a macro giving "pc = code[pc++];" would
      ↪ change pc
547  * twice in one expression, which C leaves undefined.
548  */
549 #undef CHECK
550 #define CHECK(condition, message) do { } while (0)
551 #define pop()       pop_verified()
552 #define operand()   operand_verified()
553 #define address()   operand_verified()
554 #define local()     local_verified()
555
556 #define RUN run_verified
557 #include "loop.h"
558 #undef RUN
559
560 #undef pop
561 #undef operand
562 #undef address
563 #undef local
```

That gives two functions, `run_checked` and `run_verified`, from one set of instructions. The machine verifies each program and picks

one. A program that fails verification still runs, with every check, and fails exactly as the specification says, so the whole conformance suite still passes:

```
./vm: 71 of 71 passed
./vm -C: 71 of 71 passed          (-C: never verify)
```

The verified helpers must be real functions, not macros. The first version used `#define address() (code[pc++])`, and the compiler warned at once: in `pc = address();` that would change `pc` twice in one expression, which C leaves undefined.

19.3 What is still checked

A verifier can only remove checks about things that do not depend on the run. Still checked, every time:

- **stack overflow**, since how deep calls go depends on the data;
- **the kinds of values**: whether `ADD` was given numbers;
- **division by zero**, memory addresses, and block bounds;
- the number of nested calls.

The JVM goes further: its verifier also works out the *type* of every value on the stack, so that an addition of two integers checks no tags at all. That is exercise 1.

19.4 Is the verifier right?

This matters more than anything else in the chapter. If the verifier ever accepts a program it should have refused, that program runs without the checks that would have stopped it, and the machine reads or writes outside its own stack. Instead of a trap, there is silent corruption.

`fuzz.py` writes random programs. Purely random ones almost never verify, so it cheats: it keeps count of the stack depth as it writes, and mostly chooses instructions the depth allows. Now and then it breaks the rules on purpose. `fuzz.sh` then runs each program twice, verified and with every check, on a machine built with

AddressSanitizer and UndefinedBehaviorSanitizer, and with a limit on steps, so that programs that loop forever stop at the same place both times. Any difference between the two runs, or any complaint from a sanitizer, means the verifier let something through.

```
600 programs, 418 of them verified
every verified run was the same as the checked run
```

Over 1,500 programs and three seeds while writing this chapter, there was no difference. To check that the test can fail, the verifier was broken on purpose, by removing its check for pops from a frame with too few values. The fuzzer found programs it let through on the first run, and the sanitizer named the result:

```
runtime error: index -1 out of bounds for type 'value [256]'
```

19.5 Measured

Best of five, milliseconds:

	GCC 13		Clang 18	
	checked	verified	checked	verified
fib	111	81	59	51
loop	844	676	857	812
sieve	87	68	63	54
enkel-sieve	182	176	152	135
basal-loop	47	45	34	38

Between 5 and 27 percent faster, mostly, with the usual noise. The remaining cost is in what is still checked: every `push` tests for overflow, and every `ADD` tests two tags.

19.6 Run it

```
make test
make bench
```

19.7 Exercises

1. Make the verifier track the kind of each value: number, reference, or unknown. Then `ADD` on two values known to be numbers needs no tag checks. Which of the benchmarks get faster?
2. Overflow can be checked once per call instead of once per push: the verifier knows how deep each function's stack can get, so `CALL` can check that much room is left. Does this change what the machine does, as the specification describes it? (Look at *where* a stack overflow would now be reported.)
3. Let the verifier accept `CALLI`, for programs that say, in the file, which addresses are functions and how many arguments each takes. What must `CALLI` still check while running?
4. Shrink a failing fuzz case automatically: remove instructions one at a time while the failure remains, until nothing can be removed. This is *delta debugging*.

Chapter 20

Native code

The interpreter removed: a program translated once, by way of C, into the processor's own code, failing exactly as the interpreted program would.

An interpreter, however good, spends much of its time on the same small job: fetch the next instruction, find out what it is, jump to the code for it. Chapters 16 to 19 made that cheaper. This chapter removes it, by translating the program, once, into the processor's own code. That is what a JIT compiler does while a program runs, and what an ahead-of-time compiler does before.

This chapter does it ahead of time, and through C. `vm2c.py` turns a `.vm` file into a C program, and a C compiler turns that into native code. Going through C means the result runs wherever there is a C compiler: on an x86 PC, on an ARM Mac, on a Raspberry Pi. A JIT that writes machine code itself must be written again for each kind of processor. Compiling to C is how early C++ (Cfront), Nim, Chicken Scheme and many others began.

20.1 From instructions to C

Every instruction becomes the C code the interpreter would have run for it, with its operands written in as constants, behind a label named after its address:

```
A12: start = 12;                          /* JZ 17 */
    { struct value v = pop(); if (!v.is_ref && v.n == 0) goto A17;
  ↪ }
    goto A14;
A14: start = 14;                          /* LOAD 0 */
    push(stack[local_at(0)]);
    goto A16;
```

A jump to a known address is a `goto`. After `RET` and `CALLI`, where the address is known only while running, the code goes through a `switch` on it. Nothing else is left of the interpreter: no loop, no fetching, no decoding.

The machine's state and everything that can fail come from `runtime.h`, which is chapter 19's interpreter with its run loop taken out: the same values and tags, the same stack and frames, both garbage collectors, the same arithmetic, the same `trap()`. So a compiled program fails exactly as the interpreted one does, with the same message, at the same address.

Two details keep that exact:

- **Every address gets a label**, not only those where an instruction starts when the program is read from the beginning. A program may jump into the middle of an instruction, and the specification says what happens then.
- **A damaged file** is reported by `vm2c.py` word for word as the machine would report it.

20.2 The same machine, proved again

`vmc` compiles a program, compiles the C, and runs the result, so the whole chain can be tested as if it were a machine:

```
./vmc: 71 of 71 passed
./vmc -g copy -S: 71 of 71 passed
```

Every program in the conformance suite of chapter 8, as extended since, compiled to native code: same output, same traps at the same addresses, same exit status, with both collectors and under stress.

20.3 Verified programs need fewer checks

A program that passes the analysis of chapters 17 and 19, in `flow.py`, with no instruction that is certain to trap, is compiled without the checks the analysis made. Pops do not test for underflow, `LOAD` and `STORE` do not test their locals, `RET` does not test for a frame, and only the instructions that can be reached get any code. Programs that do not verify, such as those using `CALLI`, get every check. `vm2c.py -C` puts every check in anyway.

20.4 Measured

Native code against the fastest interpreter so far, chapter 19's, which itself verifies and leaves out checks. Best of five, milliseconds, `-O2`:

	Clang 18			GCC 13		
	interpreted	native	faster	interpreted	native	faster
fib	41	14	2.8×	72	35	2.0×
loop	848	50	16.7×	742	267	2.7×
sieve	53	15	3.4×	82	58	1.4×
closures	57	42	1.3×	134	113	1.1×
enkel-sieve	153	7	20.4×	154	37	4.1×
basal-loop	30	8	3.5×	40	16	2.4×

The biggest gains are in loops of plain arithmetic, where a C compiler can see the whole loop at once: it keeps the stack pointer in a register, and removes checks it can prove redundant from one instruction to the next. `closures` gains least. Every call there goes through `CALLI` and the `switch`, which the C compiler cannot see through, and most of its time is spent making blocks and collecting them, which compiling does not change.

And the C compiler matters enormously. On this kind of code, a long function full of `goto`s and small checks, Clang does far better than GCC. Telling both that `trap()` never returns changed nothing measurable; finding out why GCC does not keep `sp` in a register here would be a study of its own.

20.5 What a JIT would add

A JIT compiles while the program runs. It can wait to see which code is *hot*, and compile only that; it can see which values actually arrive, and compile code specialised for them, with a cheap check that falls back to the interpreter if they ever change. That is how the JVM's HotSpot, V8 and LuaJIT reach speeds near C. The price is a compiler inside the machine, writing machine code for one particular processor. Exercise 3 starts one.

20.6 Run it

```
make test
make bench
make NATIVE_CFLAGS="-O2" CC=clang bench
```

20.7 Exercises

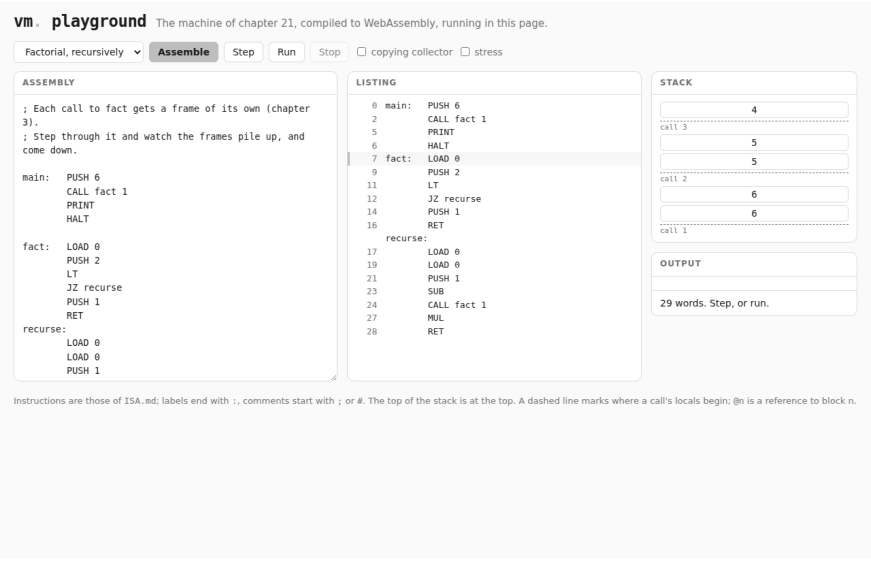
1. Keep the top few stack values in C local variables instead of in `stack[]`, for verified code, where the depth at each instruction is known. (What must happen before a call, or a garbage collection?) How much faster is `loop`?
2. Compile `CALL`s to known functions into C function calls, and `RET` into `return`. What does this do for `fib`? What happens to `frames[]`?
3. Write a small JIT for x86-64 or ARM64: translate `PUSH`, `ADD`, `LOAD`, `STORE`, `JZ` and `JMP` straight into machine code in memory made executable with `mmap`, and fall back to the interpreter for everything else.
4. Compile to WebAssembly instead of C. (Chapter 21 runs the machine in a browser. Compare the two approaches.)

Chapter 21

The machine in a browser

The machine with no operating system at all: a core that needs only two functions from outside, compiled to WebAssembly and run in any web browser.

A virtual machine is a promise that a program will run the same everywhere the machine does. This chapter takes the promise to its widest reach: a web browser, on any computer or phone, with nothing installed.



21.1 A machine with no operating system

Every machine so far has been a C program that uses the C library: `printf` to print, `malloc` for memory, `exit` to stop, `fopen` to read the program. A browser has none of those. So `core.c` is chapter 16's machine with all of it taken out. It uses no library at all, and everything it needs from outside goes through two functions that whoever runs it must provide:

ch21-wasm/core.c, lines 35–36

```
35 IMPORT(host_write) void host_write(int c);
36 IMPORT(host_trap) __attribute__((noreturn)) void host_trap(const
     ↪ char *why, int at);
```

The host gives the machine a program by writing the bytes of a `.vm` file into a buffer and calling `vm_load()`. Then `vm_step()` runs one instruction and `vm_run()` runs to the end. A few more functions let a debugger look inside: the program counter, the stack, the frames.

That is the *embedding* interface, the same idea as Lua's, which is why Lua runs inside so many games and editors: the machine makes no assumption about what it is running in.

21.2 Two hosts

`host.c` is an ordinary C program: `host_write` is `putchar`, `host_trap` is `fprintf` and `exit`. Built with it, the core is a command like every earlier machine, and passes the conformance suite. That checks the core, before anything else is involved.

ch21-wasm/host.c

```
1 /*
2  * The host for core.c, as an ordinary C program: the machine as a
3  * command, like the machines of earlier chapters.
4  *
5  *      vm [-g mark | -g copy] [-S] program.vm
6  */
7
8 #include <stdio.h>
9 #include <stdlib.h>
```

```
#include <string.h>

void host_write(int c) {
    putchar(c);
}

void host_trap(const char *why, int at) {
    fflush(stdout);
    fprintf(stderr, "trap: %s at %d\n", why, at);
    exit(1);
}

unsigned char *vm_input(void);
int vm_input_size(void);
const char *vm_load(int size);
void vm_reset(void);
void vm_set_collector(int copying, int stressed);
void vm_run(void);

int main(int argc, char *argv[]) {
    int i, copying = 0, stressed = 0, size;
    const char *path, *why;
    FILE *f;

    for (i = 1; i < argc - 1; i++) {
        if (strcmp(argv[i], "-g") == 0 && i + 1 < argc - 1)
            copying = strcmp(argv[++i], "copy") == 0;
        else if (strcmp(argv[i], "-S") == 0)
            stressed = 1;
        else
            break;
    }
    if (argc < 2 || i != argc - 1) {
        fprintf(stderr, "usage: vm [-g mark | -g copy] [-S]
  ↪ program.vm\n");
        return 2;
    }
    path = argv[argc - 1];
    f = fopen(path, "rb");
    if (f == NULL) {
        fprintf(stderr, "%s: cannot open\n", path);
        return 2;
    }
    size = (int) fread(vm_input(), 1, vm_input_size(), f);
    fclose(f);
    why = vm_load(size);
    if (why != NULL) {
        fprintf(stderr, "%s: %s\n", path, why);
        return 2;
```

```
    }
    vm_set_collector(copying, stressed);
    vm_reset();
    vm_run();
    return 0;
}
```

The second host is JavaScript. `clang` compiles the same core to WebAssembly, with no C library at all:

```
clang --target=wasm32 -nostdlib -Wl,--no-entry -o web/vm.wasm
  ↪ core.c
```

The result, `web/vm.wasm`, is 11.7 KB. It imports exactly two functions, `host_write` and `host_trap`, and `web/machine.js` provides them. A trap becomes a JavaScript exception that unwinds out of the WebAssembly code.

Two details of going without a library. A C compiler may turn a loop that copies memory into a call to `memcpy`, even if the program never names it, so `core.c` brings its own. And `PRINT` must write numbers without `printf`, so `write_number` does it digit by digit, carefully, since −2147483648 has no positive partner:

ch21-wasm/core.c, lines 481–496

```
/* PRINT, without printf: the digits of n, then a newline. */
static void write_number(int n) {
    char digits[12];
    int k = 0;
    unsigned int u = n < 0 ? 0u - (unsigned int) n : (unsigned
  ↪ int) n;

    if (n < 0)
        host_write('-');
    do {
        digits[k++] = (char) ('0' + u % 10);
        u /= 10;
    } while (u > 0);
    while (k > 0)
        host_write(digits[--k]);
    host_write('\n');
}
```

21.3 The fourth machine

`run.mjs` runs a `.vm` file on the WebAssembly machine under Node.js, as a command. Then the conformance suite of chapter 8 can test it like any other machine:

```
./vm: 71 of 71 passed                          (the core, with a C
  ↪ host)
node run.mjs: 71 of 71 passed                  (the core, as
  ↪ WebAssembly)
node run.mjs -g copy -S: 71 of 71 passed
```

The C machine, the Python machine of chapter 8, the native code of chapter 20, and now WebAssembly in a JavaScript engine: four very different ways of running a program, held to one specification by one set of tests.

21.4 The playground

`web/index.html` is a page for writing and running programs. Choose an example or write assembly language, then assemble it, run it, or step through it one instruction at a time and watch the stack: each call's locals begin at a dashed line, and references to heap blocks show as `@n`, as in the traces of chapter 7. A trap shows where it happened, in the listing.

The assembler is `web/asm.js`, chapter 6's `asm.py` in JavaScript. `test-asm.mjs` checks that it makes exactly the same bytes as `asm.py` for every program in the suite.

`web/index.html` must be served over HTTP (`make serve`, then open `http://localhost:8000`), since a browser will not fetch `vm.wasm` for a page opened from a file. `bundle.py` makes `web/playground.html`, one 36 KB file with the scripts and the machine written into it, which opens straight from disk.

21.5 Run it

```
make test
```

```
make serve
```

`make` needs `clang` with the WebAssembly target and `wasm-ld` (on Debian and Ubuntu, the `lld` package), and Node.js for the tests. `web/vm.wasm` and `web/playground.html` are kept in the repository, built, so that the playground works without building anything.

21.6 Exercises

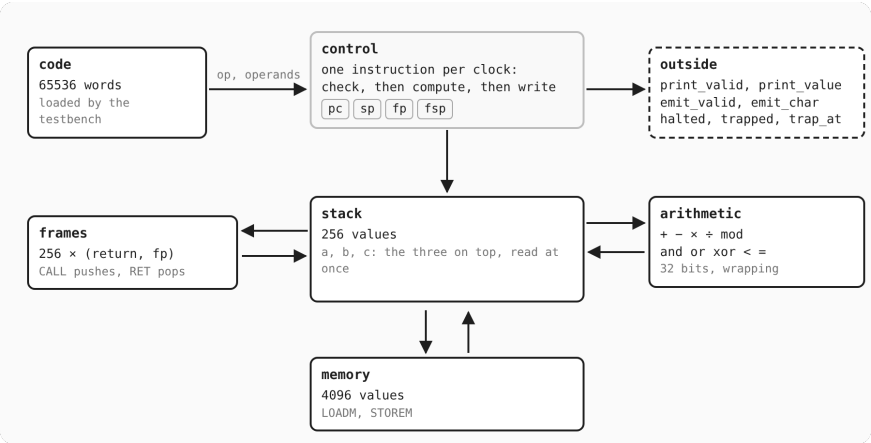
1. Add a *breakpoint* to the playground: click a line of the listing, and running stops there.
2. Add the compilers to the page. enkel/0's compiler is Python; either port it to JavaScript, or run Python itself in the browser with Pyodide. Which is better for the reader?
3. Make `run()` in the page give the browser back control every few milliseconds instead of every 200,000 instructions. What does it cost?
4. Compile `core.c` with `-O0`, `-O2` and `-Os`, and compare the size of `vm.wasm` and the speed of `bench/loop` from chapter 16 under Node.js. Then compare with the native machine. How close does WebAssembly get?

Chapter 22

The machine as hardware

The last layer removed: the stack machine as a circuit in Verilog, one instruction on each tick of a clock, held to the same specification by the same tests.

Every machine in this book has been a program, running on a processor that someone else designed. The last chapter removes that layer: the stack machine as a circuit, described in Verilog, the language chip designers use, and run in a simulator, cycle by cycle.



22.1 A CPU in one page

`cpu.v` implements version 2 of the specification: instructions 0 to 26, with frames, memory, `EMIT`, and every trap. The heap of version 3 and `CALLI` of version 4 are not here. A garbage collector in hardware is a project of its own (Lisp machines of the 1980s had one), so codes 27 and above are unknown instructions, as they were in version 2.

One instruction runs on each rising edge of the clock. Everything it needs is read at once: its opcode and operands from code memory, and the three values on top of the stack, called `a`, `b` and `c`. Then everything it changes is written together at the end of the cycle. Here is `ADD`, with the instructions that share its shape, beside the C of chapter 16:

ch22-metal/cpu.v, lines 119–137

```
119             ADD, SUB, MUL, AND, OR, XOR, LT, EQ, DIV, MOD:
120                 if (room < 2) stop(T_UNDERFLOW);
121                 else if ((op == DIV || op == MOD) && c == 0)
    ↪ stop(T_DIVZERO);
122                 else begin
123                     case (op)
124                     ADD: stack[sp - 2] <= b + c;
125                     SUB: stack[sp - 2] <= b - c;
126                     MUL: stack[sp - 2] <= b * c;
127                     AND: stack[sp - 2] <= b & c;
128                     OR:  stack[sp - 2] <= b | c;
129                     XOR: stack[sp - 2] <= b ^ c;
130                     LT:  stack[sp - 2] <= b < c;
131                     EQ:  stack[sp - 2] <= b == c;
132                     DIV: stack[sp - 2] <= b / c;
133                     MOD: stack[sp - 2] <= b % c;
134                     endcase
135                     sp <= sp - 1;
136                     pc <= pc + 1;
137                 end
```

ch16-dispatch/ops.h, lines 17–21

```
17 OP(ADD) {
18     struct value w = pop(), v = pop();
19     push_int(wrap_add(number(v), number(w)));
20     NEXT;
```

```
21 }
```

The C does it step by step: pop, pop, add, push. The hardware does it all at once: it checks that the frame has two values, writes the sum over the lower one, and moves `sp` down by one, all on the same edge of the clock. And 32-bit wrapping arithmetic, which the C machine needed `to_int` and unsigned arithmetic to get right, is simply what a 32-bit adder does.

Verilog's arithmetic matched the specification without help, and that was checked before writing the CPU, not assumed: `-2147483648 / -1` wraps to `-2147483648`, division rounds towards zero, and the remainder takes the sign of the dividend.

22.2 Talking to the outside

A circuit has no `printf`. The CPU has output *signals* instead. When `PRINT` runs, `print_valid` is 1 for one cycle, with the number on `print_value`; `EMIT` does the same with `emit_valid` and `emit_char`. When the machine stops, `halted` or `trapped` goes to 1 and stays so, with `trap_reason` and `trap_at`. On a real board, these would drive a serial port or a display. In simulation, the testbench `tb.v` watches them and writes what they say.

`vrun.py` wraps the simulation as a command, like every machine before it: it checks the `.vm` file, writes it out as hexadecimal for the testbench, runs the simulation, and turns the end of it into the right messages and exit status.

22.3 The fifth machine

So the conformance suite can test the hardware:

```
python3 vrun.py: 49 of 71 passed
  calli: a known failure
  garbage: a known failure
  ... (22 known failures)
```

The 49 that pass are every test of versions 1 and 2, including the order of the checks, the exact limits of 256 values and 256 frames, and all the arithmetic edge cases. The 22 known failures are exactly the tests that use the heap or `CALLI`. The `Makefile` finds them by looking at what each test contains, so the list cannot drift.

Real programs run too. Programs compiled by the enkel/0 and basal/0 compilers of chapters 10 and 11 run on the simulated CPU and print exactly what they print on the C machine, in one clock cycle per instruction (plus one, at the end, to see that it stopped):

Program	Cycles
enkel-hanoi	803
enkel-bubble-sort	2,264
enkel-sieve	9,142
basal-table	5,949
fib(20)	218,911

A simulator is slow: about 150,000 cycles a second here. On an FPGA running at, say, 50 MHz, `fib(20)` would take 4.4 milliseconds. But that is arithmetic, not a measurement: this design has **not** been built for a real chip, and its single-cycle structure, with a whole divider in one cycle, would limit how fast its clock could go.

22.4 The end of the road

The book began with a loop in C that fetched an instruction, looked at it, and did what it said. It ends with the same loop as a circuit, where fetching, looking and doing all happen at once, on the edge of a clock. In between, the same machine was written in C, in Python, as threaded code, as registers, as native code, as WebAssembly, and now as hardware. Each was held to one specification by one set of tests. That, more than any single machine, is what this book was about.

22.5 Run it

```
make test
python3 asm.py samples/fib.asm fib.vm && python3 vrun.py -c
  ↪ fib.vm
```

`make` needs Icarus Verilog (`iverilog` and `vvp`; on Debian and Ubuntu, the `iverilog` package).

22.6 Exercises

1. Add `NEW`, `GETF`, `SETF` and `LEN` with a heap that is never collected, as in chapter 13. What must change in the stack, now that a value needs a tag?
2. Split each instruction into two stages, *fetch* and *execute*, and overlap them: while one instruction executes, the next is fetched. What happens at a jump?
3. Synthesize `cpu.v` for an FPGA with Yosys (`yosys -p "synth_ice40"`) and look at how big the divider is. Replace `/` and `%` with a divider that takes 32 cycles. What does `DIV` cost now?
4. Write a test that runs the same program on the C machine and on the CPU, and compares the stack after every instruction, as `compare.py` compared traces in chapter 8.

Appendix A

The Metcalfe parsing machine

A machine from 1964 that only parses: one flag, calls that can be undone, and a grammar that becomes its program almost word for word.

In 1964 Howard Metcalfe described a machine for one job: reading a sentence and deciding whether it follows a grammar.[1] It is a virtual machine in the full sense: a small, fixed set of instructions, and programs written for it, one per language. It is also older than almost every machine in this book's background.

A.1 The machine

The machine has an input tape of tokens, an output tape, one flag, and a stack of calls. There are no numbers to add and no values on a stack. Everything it does is a test, and the answer goes to the flag:

[1] Howard H. Metcalfe, "A parameterized compiler based on mechanical linguistics", *Annual Review in Automatic Programming*, vol. 4, ed. Richard Goodman, Pergamon Press, 1964. Also see Mick Farmer, *Compiler physiology for beginners*, Chartwell-Bratt, 1985.

Instruction	Does
`call L`	call the rule at `L`
`return`	return; if the flag is false, undo the rule
`match x`	if the next token is `x`, take it and set the flag, else clear it
`print x`	write `x` on the output tape
`true L`, `false L`	jump if the flag is set, or clear
`flag true`, `flag false`	set or clear the flag
`stop`	stop

A grammar becomes a program almost word for word: each rule is a subroutine, each alternative a test with a jump. Here is a factor, `F = name | number | "(" E ")"`, from `samples/postfix.mc`:

```
F:      match ?
        false F1
        print ?
        true F9
F1:     match (
        false F9
        call E
        false F9
        match )
F9:     return
```

The one subtle instruction is `return`. Each `call` remembers where on both tapes it began. If the rule ends with the flag clear, `return` moves the input back to that point and takes away whatever the rule wrote. So a rule that fails half way leaves no trace, and the caller can try something else.

`metcalfe.py` is the whole machine and its assembler: under 200 lines of Python. The details of `match ?` and `print ?`, which take and copy any name or number, are ours, added so that the examples can carry real names instead of a placeholder.

A.2 Compared with ours

Our machine computes, and a parser for it, in chapter 12, was a program in Python. Metcalfe's machine *only* parses, and the parser

is the program. The instruction set is chosen for the problem: the flag is the only value, because a recognizer only ever needs to know whether the last thing worked.

The postfix sample shows the two meeting. It turns infix into postfix, which is exactly the order of operations a stack machine needs:

```
a + b * c         =>  a b c * +
2 * (x + 1) / y   =>  2 x 1 + * y /
```

Prefix "PUSH" to every name and number, and the operators become `ADD` and `MUL`: that is chapter 9's compiler, as a program for a different machine.

A.3 The limits of undoing

`samples/choice.mc` shows how far `return` can undo. Its grammar is `S = A B`, with `A = "a" "c" | "a"` and `B = "c" "b" | "d"`. Each alternative of `A` is a rule of its own, so when the first fails half way, its `return` puts the input back for the second. But once `A` has returned successfully, nothing makes it try again:

```
a c c b   =>  ac cb
a c b     =>  not accepted
```

`a c b` is in the language: `A` as `a`, then `B` as `c b`. The machine never finds it, since `A` takes `a c` first and `B` is left with `b`. This is *ordered choice*: the first alternative that matches wins. Parsing expression grammars, forty years later, made the same choice on purpose, and the combinators of chapter 12 behave the same way.

Schorre's META II, from the same year, is a close relative: a machine with a flag, tests, calls and output, and a compiler written for it that could compile itself.

A.4 Run it

```
make test
echo "a + b * c" | python3 metcalfe.py samples/postfix.mc
echo "a + b" | python3 metcalfe.py -t samples/postfix.mc
```

With `-t`, the machine shows every step: the address, the instruction, what is left of the input, and what it has written.

A.5 Exercises

1. Change `samples/postfix.mc` so its output is assembly language for our machine: `PUSH 2`, `ADD`, and so on. Names are a problem; what could they become?
2. Add a unary minus to the grammar. Where must it go, and what does it print?
3. Change `samples/choice.mc` so that it accepts `a c b`. Can it be done by reordering alone? What does that cost for other inputs?
4. The machine's undoing is only as deep as one rule. Add an instruction that lets a rule try its alternatives again after it has returned. (This is *backtracking*, and it can take time exponential in the length of the input.)

Appendix B

p-code, Pascal's machine

The machine Pascal travelled on: Wirth's P-machine with its static links, then the P4 machine of Pascal-P4, and pascal/0, a small Pascal compiled to it.

In the early 1970s, Niklaus Wirth's group in Zurich had a problem every language designer has: Pascal should run on many computers, and writing a compiler for each one is a great deal of work. Their answer was the Pascal-P compiler. It compiled Pascal for a machine that did not exist, a simple stack computer, and its code was called *p-code*. To bring Pascal to a new computer, you wrote an interpreter for p-code, a few pages long, instead of a compiler.

A few years later, at the University of California, San Diego, Kenneth Bowles's group took the idea much further. UCSD Pascal compiled to a compact, byte-coded p-code, and around it they built a whole system, the UCSD p-System: editor, compiler, file system, all in p-code, and all able to move to a new computer by writing one interpreter. It ran on the Apple II, and was one of the three operating systems offered with the first IBM PC. Western Digital even built a processor, the Pascal MicroEngine, whose microcode ran p-code directly, much as chapter 22 built our machine as hardware.

The p-System did not last, but the idea did. Java's slogan, "write once, run anywhere", is the p-System's plan, twenty years on.

B.1 The P-machine

The full UCSD p-code is large. Its idea fits in the smaller machine Wirth used for PL/0, a teaching subset of Pascal, in his book *Algorithms + Data Structures = Programs* (1976). `pmachine.py` implements it. It has eight instructions, each with two fields, a *level* `l` and an argument `a`:

Instruction	Does
`lit 0 a`	push `a`
`opr 0 n`	return, or an arithmetic operation, by number
`lod l a`	push the variable at offset `a`, `l` levels out
`sto l a`	pop into the variable at offset `a`, `l` levels out
`cal l a`	call the procedure at `a`, declared `l` levels out
`int 0 a`	make room for `a` words on the stack
`jmp 0 a`	jump
`jpc 0 a`	pop, and jump if it was zero

Much of this is familiar: a stack, `PUSH`, arithmetic, jumps, and calls with frames, as in chapters 1 to 3. The interesting part is the *level*.

B.2 Levels and static links

Pascal lets a procedure be declared inside another, and the inner one may use the outer one's variables. So a variable is found by two numbers: how many levels out it was declared, and its offset in that frame.

Every frame starts with three words: the *static link*, the *dynamic link*, and the return address. The dynamic link is the caller's frame, like the saved frame pointer of chapter 3. The static link is the frame of the procedure that this one was *written inside*. To reach a variable `l` levels out, the machine follows the static link `l` times.

The two links are different whenever a nested procedure calls itself. In `samples/nested.p`, `down` is declared inside `outer`, and counts `outer`'s variable `k` down to zero by calling itself. Here is the stack after two calls of `down`, from the trace (`-t`):

```
0 0 0 5 | 0 0 27 4 | 4 4 22 | 4 8 17
main      outer      down     down
```

Each `down` frame begins with its static link and its dynamic link. The first `down` was called by `outer`, at 4, so both links are 4. The second was called by the first `down`, at 8, so its dynamic link is 8. Its static link is still 4: it still finds `k` in `outer`.

B.3 Compared with ours

Our machine has no levels. In chapter 15, inner functions reached outer variables in another way: a closure, a block on the heap that holds the variables it needs. The difference is about what a language allows. In Pascal, a procedure can be passed as an argument, but never returned or stored, so it can never outlive the frame it was declared in. The frames can stay on the stack, and a static link is enough. In our lambda language, a function can be returned, and live on after its maker's frame is gone, so what it needs must move to the heap. This is the *funarg problem*, and the two machines are its two answers.

The P-machine also has no traps to speak of. Its specification is a Pascal program in Wirth's book, and what that program does with a bad jump or a stack overflow is whatever Pascal does. A real interpreter had to decide, as chapter 4 did.

B.4 Run it

```
make test
python3 pmachine.py samples/nested.p
python3 pmachine.py -t samples/nested.p
```

There is no PL/0 compiler here: the samples are p-code written by hand, with their PL/0 in a comment, as Wirth's compiler would have made it. `opr write`, which pops and prints a number, is ours; PL/0 had no output, and Wirth's interpreter printed every value as it was stored instead.

B.5 P4: the machine Pascal travelled on

The P-machine is the idea. The machine that carried Pascal to new computers was bigger: the P4 machine of Pascal-P4, from Zurich in 1976, the last of the Pascal-P compilers. (UCSD Pascal had grown from the one before it, P2.) `p4.py` is our own implementation of it, written from the description of what each instruction does, and it reads P4 code in the text form the original P4 interpreter read.

P4 keeps the P-machine's levels and static links, and adds what Pascal needs that PL/0 did not:

- **Types.** Most instructions carry a letter: `lodi` loads an integer, `lodc` a character, `lodb` a boolean; `equs` compares two sets, `equm 8` two blocks of eight words. The machine itself does not need the letters, since every word can hold a value of any type, but a real implementation does: a real took two words on some computers, and a set four.
- **A frame with a mark.** `mst` begins a frame before the arguments are pushed: five words, for a function's result, the static link, the dynamic link, the caller's stack limit, and the return address. The arguments follow, so the called procedure finds them as its first variables, at 5, 6 and so on. `cup` jumps, and `ret` returns.
- **Addresses.** `lda` and `lao` push the address of a variable, `ind` loads through an address, `sto` stores through one, and `ixa` indexes an array: `ixa 3` multiplies the index by 3, the size of an element, and adds it to the address below. A `var` parameter is an address, and arrays are reached the same way.
- **Two limits for each frame.** `ent 1 l5` makes room for the variables, and `ent 2 l6` reserves the most the stack will need above them. Both sizes are labels, given values only after the code (`l5=7`): the compiler works in one pass, and knows them only at the end. The machine checks both, so a frame can never run into the heap, which `new` grows from the other end of the store.
- **Standard procedures.** `csp wri` writes an integer, `csp rdc` reads a character, and so on: input and output are not instruc-

tions, but calls into the machine.
- **case by table.** `xjp` jumps to the table entry for the value on the stack, and the entry jumps to its branch.

Two P4 programs written by hand are in `samples`: `hello.p4`, and `nested.p4`, the program above, in P4.

B.6 pascal/0

The P4 machine was made for a compiler, and here is one: `pascal.py` compiles pascal/0, a small part of Pascal described in `LANG.md`, to P4 code. It has integers, booleans and characters, arrays, `for`, `case`, `repeat`, and procedures and functions nested in each other with value and `var` parameters. It works as Wirth's compilers did, in one pass: where chapter 10's compiler first reads the whole program into a tree, this one writes the code for each statement as soon as it has read it. The function in this program

```
function square(x: integer): integer;
begin
    square := x * x
end;
```

becomes

```
l 2
 ent 1 l 3
 ent 2 l 4
 lodi 0 5
 lodi 0 5
 mpi
 stri 0 0
 reti
l 3=6
l 4=2
```

`x` is at 5, after the mark, and the result goes in word 0 of the frame, where `reti` leaves it on the stack for the caller. The frame needs 6 words, and the stack at most 2 more.

The `pascal` folder holds thirteen programs, among them Wirth's eight queens, the sieve of Eratosthenes, and the towers of Hanoi, each

with its P4 code and its output. Those in `pascal/traps` break a rule that only shows when the program runs, and the machine stops them: an index out of range, a division by zero, a `case` value with no label, and a variable used before it has a value. Those in `pascal/errors` are refused by the compiler.

Every program in `pascal` is real Pascal. If Free Pascal is installed, `make test` compiles each one with it too, in its ISO mode, and checks that it prints exactly what the P4 machine printed.

B.7 Run it

```
make test
python3 pmachine.py samples/nested.p
python3 pmachine.py -t samples/nested.p
python3 p4.py -t samples/nested.p4
python3 pascal.py pascal/queens.pas > queens.p4
python3 p4.py queens.p4
```

B.8 Exercises

1. Write the PL/0 compiler. PL/0 is small enough that Wirth's compiler, in Pascal, is a few hundred lines, and chapter 10's enkel/0 compiler is a good start. Most of the new work is the levels: each name must remember where it was declared. Then compare it with `pascal.py`.
2. Where are a procedure's parameters on the P-machine, if the caller pushes them before `cal`? Compare with P4's `mst`, which is there to make room for the mark *before* the arguments.
3. Following `l` static links costs `l` steps on every use. A *display* is an array that holds, for each level, the frame in use now. What must `cal` and the return do to keep it right? (Many Pascal compilers did this.)
4. Write a P4 interpreter in C, and compare its speed with `p4.py` on `queens.pas`. What did the p-System gain by interpreting a compact code, on computers with 64 KB of memory?

5. Add `forward` to pascal/0, so two procedures can call each other. What must the compiler know about a procedure before it can write the call?
6. Add records. The machine needs nothing new: a field is an offset, added to the record's address with `inc`.
7. Add real numbers. The machine has them already (`adr`, `flt`, `csp wrr`); the compiler must decide when an integer becomes a real.

Appendix C

CHIP-8

A virtual machine for games on the computers of the 1970s: registers instead of a stack, graphics by XOR, and what it costs to have no specification.

In 1977, RCA sold the COSMAC VIP, a small computer kit built around its 1802 processor. To make it easy to write games for, Joseph Weisbecker designed CHIP-8: a virtual machine, with its interpreter in the VIP's first 512 bytes of memory. That is why CHIP-8 programs still start at address 0x200. Programs were typed in as hexadecimal, and a game such as Pong fitted in a few hundred bytes.

CHIP-8 outlived its computer. It was carried to the HP-48 graphing calculators in 1990, and from there to almost every computer since: writing a CHIP-8 interpreter is one of the most common first projects in emulation. (`archive/chip8` has one in C, by Dustin Roepsch, which draws on the terminal and takes the keyboard.)

C.1 The machine

`chip8.py` is the whole machine, about 300 lines. It has:

- 4096 bytes of memory, with the program at 0x200 and a font of the sixteen hexadecimal digits at 0x050;
- sixteen 8-bit registers, `V0` to `VF`, and a 12-bit address register `I`;

- a stack of up to sixteen return addresses;
- two timers, which count down 60 times a second;
- a screen of 64 by 32 pixels, and sixteen keys.

Every instruction is two bytes, read as four hexadecimal digits: the first says what to do, the rest are a register (`X`, `Y`), a byte (`NN`) or an address (`NNN`). `6XNN` puts `NN` in `VX`, `8XY4` adds `VY` to `VX`, `DXYN` draws. There are 35 of them.

This version has no window and no keys. It runs until the program jumps to itself (the usual way a CHIP-8 program ends), and prints the screen. Here is `samples/answer.hex`, which works out six times seven by adding, since CHIP-8 has no multiplication:

```
..................####.#..#.####.............................
..................#..#.#..#....#.............................
..................#..#.####.####.............................
..................#..#....#.#................................
..................####....#.####.............................
```

There is no instruction to print a number. `FX33` splits `VX` into its three decimal digits in memory, `FX29` points `I` at the font's picture of a digit, and `DXYN` draws it.

C.2 Drawing by XOR

`DXYN` draws a sprite: `N` bytes from memory at `I`, each a row of eight pixels. It does not just set pixels: it flips them. And if any pixel that was on is turned off, `VF` becomes 1.

That one rule does two jobs. A sprite is erased by drawing it again, so moving one is drawing it twice. And `VF` tells a game that two things have collided, with no extra code. `samples/collide.hex` draws a block twice, overlapping: the overlap goes dark, and `VF`, drawn on the right, is 1.

```
........####....####......................#.....................
........####....####....................##.....................
```

C.3 Compared with ours

CHIP-8 is a register machine with a stack only for return addresses. Its instructions are fixed in size and packed tightly, with registers in the instruction itself, as in chapter 17's register machine. And its instruction set is chosen for its use: drawing, keys, timers and decimal digits are built in, and there is no multiplication.

The deeper difference is that CHIP-8 has no specification. The VIP's interpreter *was* the definition. When CHIP-8 was rewritten for the HP-48, a few instructions came out differently: `8XY6`, which shifted `VY` into `VX`, became a shift of `VX` itself, and `FX55` and `FX65` moved `I` differently, and later not at all. Programs written for one version break on the other, and to this day a CHIP-8 interpreter needs settings for these *quirks*. This one does what the VIP did.

Chapter 8 exists to prevent exactly this. A written specification, and a suite of tests that every machine must pass, would have caught the difference the day the second interpreter was written.

C.4 Run it

```
make test
python3 chip8.py samples/answer.hex
python3 chip8.py -t samples/answer.hex
```

With `-t`, the machine shows every instruction, in words, with `I` and the registers. `-n` sets how many instructions it may run, and `-s` seeds the random numbers of `CXNN`, so that a run can be repeated.

C.5 Exercises

1. Write an assembler for CHIP-8, so that `samples/answer.hex` can be written as `V0 := 0`, `loop: V0 += 6`, and so on. (Octo, a popular CHIP-8 environment, has a language much like this.)
2. Add a setting for the HP-48 quirks, and write a program that shows the difference.

3. Give the machine a window and keys. With Python's `curses` module it can run in a terminal, as the C version in the archive does. Then run a real game, from one of the collections of public domain CHIP-8 programs.
4. CHIP-8 has no traps: on the VIP, an unknown instruction did whatever the interpreter happened to do with it (this one stops). Which of chapter 4's traps would make sense for CHIP-8, and which would break existing programs?

Appendix D

SWEET16

Wozniak's 16-bit machine hidden inside the Apple II's 6502 code: a virtual machine chosen for size, not speed.

The 6502 processor of the Apple II is an 8-bit processor, and doing anything with 16-bit numbers, such as addresses, takes many instructions: add the low bytes, then the high bytes with the carry, and store each separately. In 1977 Steve Wozniak, writing Integer BASIC for the Apple II, got tired of that, and wrote himself a better processor: SWEET16, a 16-bit machine interpreted by about 300 bytes of 6502 code in the Apple II's ROM.[1]

What makes SWEET16 unusual is how it was used. A 6502 program calls `SWEET16` as a subroutine, and the bytes that follow the call are SWEET16 code, not 6502 code. When the SWEET16 code says `RTN`, the interpreter returns to running 6502 code after it. A program changes machines in the middle of a routine: native code where speed matters, SWEET16 where 16-bit work makes the native code long.

D.1 The machine

`sweet16.py` is the machine and an assembler for it. It has sixteen 16-bit registers (on the Apple II, they lived in the first 32 bytes of

[1] Steve Wozniak, "SWEET16: The 6502 Dream Machine", *BYTE*, November 1977.

memory). Some have jobs: R0 is the accumulator, R12 the stack pointer for subroutine calls, R13 gets the result of a compare, and R15 is the program counter.

Most instructions are one byte. The high four bits say what to do, and the low four which register:

Code	Instruction	Does
1n	SET Rn, v	Rn := v (two more bytes)
2n, 3n	LD Rn, ST Rn	R0 := Rn, Rn := R0
4n, 5n	LD @Rn, ST @Rn	a byte to or from the address in Rn, then Rn += 1
6n, 7n	LDD @Rn, STD @Rn	the same with a 16-bit word, then Rn += 2
8n, 9n, Cn	POP @Rn, STP @Rn, POPD @Rn	back down: Rn first, then the byte or word
An, Bn	ADD Rn, SUB Rn	R0 += Rn, R0 -= Rn
Dn	CPR Rn	R13 := R0 - Rn, to compare
En, Fn	INR Rn, DCR Rn	Rn += 1, Rn -= 1

Codes 00 to 0C are RTN, the branches, and BS and RS to call and return. A branch is two bytes: the code, and a distance, from -128 to 127, from the next instruction.

The auto-increment is where SWEET16 saves the most. Here is the loop that copies a block of bytes, from samples/copy.s16:

```
copy:   LD @R1          ; R0 := the byte at R1, and move R1 on
        ST @R2          ; the byte at R2 := R0, and move R2 on
        DCR R3
        BNZ copy
```

Five bytes. On the 6502, with 16-bit addresses and a 16-bit count, the same loop is several times as long.

D.2 The prior result

BNZ above does not say which register to test. SWEET16 remembers which register the last instruction changed, the *prior result*, and the branches test that: after DCR R3, BNZ tests R3. After an arithmetic instruction it is R0; after CPR, R13. BC and BNC test the carry, which

`ADD`, `SUB` and `CPR` set. `samples/fib.s16` uses the carry to stop when a Fibonacci number grows past 16 bits.

The original kept the prior result and the carry in `R14`; this version keeps them apart, but the branches do the same.

D.3 Compared with ours

SWEET16 makes the trade of chapters 16 and 20 the other way round. Those chapters made the machine faster, in the end by turning programs into native code. SWEET16 is much slower than the 6502 code it replaces, and that is accepted: what it buys is *size*, on a computer whose ROM had room for very little. Dense code was also the p-System's reason for p-code (study B), and one byte per instruction, with the register packed into it, is how SWEET16 gets there. Our instructions are 32-bit words, which is simple, and roomy.

The way it is entered is chapter 21's *embedding* taken to its extreme. Our WebAssembly core is called by a host through a few functions. SWEET16's host and its programs are mixed in the same stream of bytes, and the processor changes machines at a call.

And its conditions are a third design, next to ours and CHIP-8's. Our `JZ` tests the value on top of the stack, and pops it. CHIP-8 has no conditional jumps, only instructions that skip the next one. SWEET16 remembers where the last result went, as real processors do with condition flags.

D.4 Run it

```
make test
python3 sweet16.py -m 0900:090F samples/copy.s16
python3 sweet16.py -t samples/copy.s16
```

When the program ends, the machine shows the registers, and with `-m` the memory between two addresses, in hexadecimal. `samples/fib.s16` writes its table as 16-bit words, low byte first, as the 6502 does: `90 00 E9 00` are 144 and 233. With `-t`, it shows every instruction.

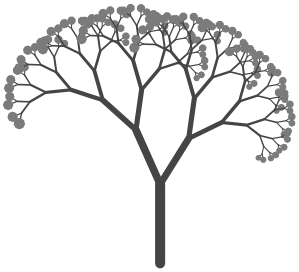
D.5 Exercises

1. SWEET16 has no shifts, so no fast multiplication. Write a 16-bit multiply with `ADD R0`-style doubling, and a subroutine. How many instructions does 300 times 200 take?
2. SWEET16 cannot print: on the Apple II, the 6502 code around it did. Give it a way out to the host, using the unused code `0D` to print `R0` in decimal, and print the table of `samples/fib.s16`. Would it have been better to return to the host with `RTN`, and let it print?
3. The original used one trick to save space in its dispatch: it pushed the address of the routine for an instruction on the 6502's stack, and "returned" to it. Find out how, and why it is shorter than a table of jumps.
4. Write SWEET16 in C, with the registers in an array of bytes, as on the Apple II. Is the prior result easier to keep as a register number, or as an address?

Appendix E

PostScript

The stack machine inside a printer: dynamic names, procedures as data, and a page as a program.



In 1985 Apple's LaserWriter printer had a more powerful processor than the Macintosh that sent it pages. It needed one: a page was not sent as dots, but as a *program*, in Adobe's new language PostScript, and the printer ran it. The same program printed the same page on any PostScript printer, at whatever resolution it had. That promise, device independence, is the promise of a virtual machine, made for paper.

E.1 The machine

PostScript is a stack machine, like ours, and its programs look like assembly language for one, written backwards from what most languages do:

```
3 4 add 5 mul =
```

Each number is pushed on the operand stack; `add` and `mul` pop two and push the result; `=` pops and prints. But it is a far richer machine. Its values are typed: numbers, strings, names, arrays, dictionaries, and procedures, written in braces. And there is a second stack, of *dictionaries*, where names get their meaning:

```
/square { dup mul } def
12 square =
```

`/square` is a literal name, and is pushed. `{ dup mul }` is a procedure, and is pushed too, not run. `def` puts the name and the procedure in the dictionary on top of the dictionary stack. When the interpreter then meets the name `square`, it looks it up, finds a procedure, and runs it.

`if`, `ifelse`, `for` and `repeat` are not syntax but operators, which take procedures from the stack:

```
/fact {
    dup 2 lt
    { pop 1 }
    { dup 1 sub fact mul }
    ifelse
} def
```

`ps.py` is an interpreter for a subset of PostScript, about 600 lines: the arithmetic, the stack, dictionaries, procedures, control, arrays, strings for printing, and enough painting for paths, fills and colour. It writes the painted page as an SVG file.

E.2 Painting

The painting operators build a *path*, `moveto` and `lineto` and `arc`, and then `stroke` or `fill` it. Every coordinate goes through the *current transformation*, which `translate`, `rotate` and `scale` change, and `gsave` and `grestore` save and bring back.

`samples/tree.ps` draws the tree above with that alone. A branch is a line; at its end the coordinate system is turned and shrunk, and a smaller tree is drawn in it, twice. The program never works out where any point is on the page, or how thick a twig should be: the transformation does it, since the line width shrinks with the scale. `samples/star.ps` draws one petal twelve times, turning between each.



E.3 Compared with ours

PostScript and our machine are both stack machines, but they differ in when a name gets its meaning. Our assembler turns every label into an address before the program runs, and chapter 10's compiler turns every variable into a memory address. PostScript looks a name up in the dictionary stack each time it is run. In `samples/calc.ps`, `fourth` is defined as `square square`; after `square` is redefined, `fourth` does something else. This is *dynamic binding*: flexible, and slow, since every name is a search.

A procedure is just an array of objects, kept as data until something runs it, so a PostScript program can build, print and change its own procedures. And because the program is text, read and run token by token, there is no compile step at all.

Its errors will look familiar. `stackunderflow`, `typecheck`, `rangecheck`, `undefinedresult` for division by zero: the traps of chapters 4 and 13, under other names. Typed values, checked on every operation, are what chapter 13's tags were for.

PostScript's descendant, PDF, made the opposite choice. A PDF page is a description, not a program: the painting operators are kept, and the loops, procedures and dictionaries are gone. A program can take any time to run, or never finish, and a document reader should not have to run one to show a page.

E.4 Run it

```
make test
python3 ps.py samples/calc.ps
python3 ps.py samples/tree.ps tree.svg
```

The page size comes from the `%%BoundingBox:` comment, as printers took it. Any web browser shows the SVG. `archive/ps` has another interpreter for a smaller subset, in JavaScript, which paints on a canvas in the browser.

E.5 Exercises

1. Add `curveto`, which draws a Bézier curve. SVG has the same curve, so it can be passed straight through, after the transformation. (An arc can be made of curves too, as real PostScript does.)
2. Add `show`, to paint text: `/Helvetica findfont 12 scalefont setfont` and then `(Hello) show`. How should a font get into the SVG?
3. Real PostScript has `bind`, which replaces each name in a procedure with the operator it names now, once. Implement it, and measure `samples/tree.ps` with and without. Which names must not be bound?
4. Write a small PostScript program that prints a table of the squares and cubes of 1 to 10, right-aligned, using only `=`, `print`,

`cvs` and `string`.

Appendix F

The Java Virtual Machine

The best-known stack machine of all: a constant pool, typed instructions, and every program checked before it runs.

When Java appeared in 1995, the language was only half of what was new. The other half was the machine: Java programs compiled not to the code of any processor, but to *bytecode* for the Java Virtual Machine, and the same `.class` file ran on any computer that had one, even inside a web page. The p-System (study B) had made the same promise twenty years earlier. This time it came with the web, and it took.

It is still one of the most widely used virtual machines there is, and many other languages, such as Kotlin, Scala and Clojure, compile to it.

F.1 The machine

The JVM is a stack machine, and close to ours. Here is a recursive Fibonacci in Java, as `javac` compiled it, and the same function written for our machine:

```
fib(I)I                                 fib:    LOAD 0
     0  iload_0                                 PUSH 2
     1  iconst_2                                LT
     2  if_icmpge 7                             JZ more
```

```
 5  iload_0                              LOAD 0
 6  ireturn                              RET
 7  iload_0                     more:    LOAD 0
 8  iconst_1                             PUSH 1
 9  isub                                 SUB
10  invokestatic Fib.fib:(I)I            CALL fib 1
13  iload_0                              LOAD 0
14  iconst_2                             PUSH 2
15  isub                                 SUB
16  invokestatic Fib.fib:(I)I            CALL fib 1
19  iadd                                 ADD
20  ireturn                              RET
```

Almost line for line, the same: arguments as numbered local variables, arithmetic on the stack, a call that takes its arguments from the stack and leaves the result there.

`jvm.py` reads class files and runs them: about 450 lines, for a subset of the JVM, which is ints, arrays of ints, static methods, string constants and printing. Three programs in `src/` were compiled with `javac`, and their class files are in `classes/`. `jvm.py` prints exactly what the real JVM prints for each, including the stack trace when `Divide` divides by zero:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Divide.share(Divide.java:4)
    at Divide.main(Divide.java:10)
```

The test checks both machines against the same expected output, the way chapter 8 checked ours.

F.2 Compared with ours

The differences are in what the class file carries.

The constant pool. Our `CALL` holds an address. The JVM's `invokestatic` holds the number of an entry in the class's *constant pool*, which names the class, the method and its type: `Fib.fib:(I)I`. Nothing is an address until the program runs. A class can then be loaded when it is first used, from a file or from the network, and replaced by a newer version without recompiling those that use it. The price is a lookup, which a real JVM does once, and then remembers.

Typed instructions. Our `ADD` adds whatever is on the stack, and chapter 13 had to tag every value, and check the tags while running, to tell numbers from references. The JVM has `iadd` for ints, `ladd` for longs, `fadd` and `dadd` for floats and doubles; and `iload` and `aload` for ints and references. The type is in the instruction, so it can be checked *before* the program runs.

Verification. That check is the JVM's verifier, which is chapter 19's verifier grown up. Before running a method, it works out what type every stack slot and local variable has at every instruction, and refuses the class if any instruction could get the wrong type, or the stack could overflow or underflow. Each method in the class file declares its `max_stack` and `max_locals` (`fib` has 3 and 1), which chapter 19 had to compute. `jvm.py` does not verify, but it does stop if a method's stack ever grows past its `max_stack`.

Line numbers. The class file keeps a table from bytecode offsets to source lines, which is how the stack trace above can say `Divide.java:4`. Chapter 7's traces pointed at addresses; this is the next step, back to the source.

A real JVM, such as HotSpot, also does what Part IV did: it starts by interpreting, counts which methods run often, and compiles those to native code, while the program runs. Chapters 16 and 20, in one machine.

F.3 Run it

```
make test
python3 jvm.py classes/Fib.class
python3 jvm.py -c classes/Fib.class
```

`-c` shows the bytecode of every method, as `javap -c` does. `make classes` compiles the sources again, with `javac`; the tests do not need it. If `java` is installed, `make test` runs the real JVM on the same classes too.

The archive has more: `archive/jvm1` and `archive/jvm2` read class files in Python, and `archive/pitiful` is PitifulVM, a small JVM in C from National Cheng Kung University, under its own BSD

licence.

F.4 Exercises

1. Add `long`: `lconst_0`, `ladd`, `lload` and the rest. Longs take two local variable slots and two stack slots each. Why did the JVM's designers do that, and what does it cost?
2. Add objects: `new`, `invokespecial` for the constructor, `getfield` and `putfield`. Where does `jvm.py` find out how many fields an object has?
3. Write a verifier for the subset: for each instruction, the types on the stack before it, as chapter 19 did with depths. Then compile a class whose bytecode is wrong (edit the bytes) and see the real JVM refuse it with a `VerifyError`.
4. Add `try` and `catch`: the exception table, which `jvm.py` now skips, says which range of bytecode each handler covers.

Appendix G

The instruction set

Every instruction of the machine, with its code, its operands, its effect on the stack, and the chapter where it first appears.

Every instruction is one integer, followed by its operands, if any, as further integers. Values are 32-bit signed integers; from chapter 13 on, a value can also be a reference to a block on the heap. From chapter 5 on, programs are stored in files as described in `ch05-files/FORMAT.md`.

An instruction never changes number or meaning once added. The *since* column gives the chapter where it first appears.

Stack effects are written `before -> after`, with the top of the stack on the right.

Code	Name	Operands	Stack effect	Since	Meaning
0	HALT			1	Stop the machine.
1	PUSH	n	`-> n`	1	Push the number n.
2	ADD		`a b -> a+b`	1	
3	SUB		`a b -> a-b`	1	
4	MUL		`a b -> a*b`	1	
5	PRINT		`a ->`	1	Print a in decimal, then a newline.
6	JMP	addr		2	Continue at addr.
7	JZ	addr	`a ->`	2	Continue at addr if a is 0.
8	DUP		`a -> a a`	2	

Code	Name	Operands	Stack effect	Since	Meaning
9	DROP		`a ->`	2	
10	SWAP		`a b -> b a`	2	
11	OVER		`a b -> a b a`	2	
12	ROT		`a b c -> b c a`	2	
13	LT		`a b -> a<b`	2	1 if true, 0 if false.
14	EQ		`a b -> a==b`	2	1 if true, 0 if false.
15	LOAD	k	`-> local[k]`	3	Push local k of the current frame.
16	STORE	k	`a ->`	3	local[k] = a.
17	CALL	addr n	(see chapter 3)	3	Call addr; the top n values become locals 0 to n-1.
18	RET		(see chapter 3)	3	Return the top value to the caller.
19	DIV		`a b -> a/b`	4	Rounded toward zero. Traps if b is 0.
20	MOD		`a b -> a%b`	4	Sign follows a. Traps if b is 0.
21	AND		`a b -> a&b`	10	Bitwise.
22	OR		`a b -> a|b`	10	Bitwise.
23	XOR		`a b -> a^b`	10	Bitwise.
24	LOADM		`a -> mem[a]`	10	Read memory. Traps outside 0 to 4095.
25	STOREM		`a v ->`	10	mem[a] = v. Traps outside 0 to 4095.
26	EMIT		`c ->`	10	Write the byte c, 0 to 255.
27	NEW		`n -> r`	13	A new block of n fields, all 0.
28	GETF		`r i -> v`	13	Field i of block r. Traps out of bounds.
29	SETF		`r i v ->`	13	Field i of block r = v. Traps out of bounds.
30	LEN		`r -> n`	13	The number of fields of block r.
31	CALLI	n	`t -> ...`	15	As CALL, to the address t taken from the stack.

G.1 Traps

From chapter 4 on, the machine checks every step. When a step would fail, the machine stops with exit status 1 and prints `trap: <reason> at <address>` on standard error, where the address is that of the failing instruction. Output printed before the trap is kept. The reasons are listed in chapter 4.

In chapters 1 to 3 the same situations are undefined.

Until chapter 8, overflow of `ADD`, `SUB`, `MUL` and `DIV` is undefined.

G.2 The specification

From chapter 8 on, the machine is defined by a specification: version 1 in `ch08-spec/SPEC.md`, and version 2, which adds memory and codes 21 to 26, in `ch10-enkel/SPEC.md`, and version 3, which adds the heap, references and codes 27 to 30, in `ch13-heap/SPEC.md`, and version 4, which adds code 31, in `ch15-closures/SPEC.md`. This table is a summary. Where they differ, the specification is right. It settles what the table leaves open: arithmetic wraps at 32 bits, every instruction carries out its steps in a fixed order, and `STORE` pops its value before it checks the local.

Appendix H

The file format and the specification

The machine written down in full: the format of its program files, and the specification, version 4, as it stands at the end of the book.

H.1 The .vm file format, version 1

A `.vm` file holds one program. All numbers in it are 32-bit signed integers, stored *little-endian*: least significant byte first.

Offset	Size	Contents
0	3	the magic number: the bytes `S`, `V`, `M`
3	1	the format version: `1`
4	4	n, the number of code words, 1 to 65536
8	4 × n	the code words, from address 0 upward

The file ends right after the last code word.

A file that breaks any of these rules is refused before it runs: the machine prints `<file>: <reason>` on standard error and exits with status 2. A program that loads but then fails while running is a trap, with exit status 1, as in chapter 4.

H.1.1 Why each part is there

The magic number lets the machine tell a program from any other file at once, rather than running a JPEG and trapping a few instructions later. Most binary formats start with one: `\x7fELF` for Linux programs, `CAFEBABE` for Java class files, `%PDF` for PDFs.

The version makes it possible to change the format later without old machines quietly misreading new files.

The length lets the machine check that the whole program is there, and that nothing follows it. A file cut short by a failed copy is caught on loading, not discovered as a strange trap halfway through a run.

Little-endian, always means a file written on one computer runs on another, whatever order that computer keeps the bytes of its own integers in. The loader builds each word from its four bytes, so it never depends on the byte order of the computer it runs on.

H.1.2 What is not here

There is nothing but code. Real formats also carry data (strings, constants), a table of names for debugging, and the address to start at. This book never needs them: its compilers build strings with instructions, and traps name addresses, which the assembler's listing turns back into lines. Designing a version 2 that carries names is a good exercise; chapter 7 suggests why a debugger would want it.

H.2 The machine: specification, version 4

This document says what the machine does. An implementation that does something else is wrong, however sensible its own behaviour may seem. Where this document is silent, the machine's behaviour is not specified, and a program that depends on it is wrong.

The two implementations in this chapter, `vm.c` and `vm.py`, follow it. The test suite in `suite/` checks that they do.

Version 2 added memory and the instructions `AND`, `OR`, `XOR`, `LOADM`, `STOREM` and `EMIT` (codes 21 to 26).

Version 3 adds the heap: values are now either integers or references to blocks, and the instructions `NEW`, `GETF`, `SETF` and `LEN` (codes 27 to 30) make and use blocks. A program that never uses codes 27 to 30 can never make a reference, so it behaves exactly as under version 2.

Version 4 adds `CALLI` (code 31): a call to an address taken from the stack. The file format is unchanged.

H.2.1 1. Values

A *value* is either an *integer* or a *reference*.

An integer is a signed 32-bit number, from −2147483648 to 2147483647. Arithmetic *wraps*: a result is reduced into that range modulo 2^{32}. So 2147483647 + 1 gives −2147483648, and 65536 × 65536 gives 0.

A reference refers to a *block* on the heap (section 2a). Only `NEW` makes references. No instruction turns an integer into a reference, or a reference into an integer.

Wherever an instruction below says it takes a *number*, it means an integer: if it is given a reference instead, it traps: *not a number*. Where it says it takes a *block*, it means a reference: if it is given an integer instead, it traps: *not a reference*.

H.2.2 2. The state of the machine

Part	What it is
code	a sequence of n values, at addresses 0 to n − 1, fixed when the program is loaded
pc	the address of the next instruction, initially 0
stack	a sequence of at most 256 values, initially empty; its last element is the *top*
frames	a sequence of at most 256 frames, initially empty
fp	the frame pointer: a position in the stack, initially 0
memory	4096 values, at addresses 0 to 4095, initially all 0
heap	a collection of blocks, initially empty

Code values, and so the operands of instructions, are always integers.

H.2.3 2a. The heap

A *block* is a sequence of values, its *fields*, numbered from 0. Its length is fixed when it is made. Each block also has a *number*: the first block made is number 1, the next number 2, and so on. The number identifies the block in traces (section 8), and nowhere else: a program cannot see it.

A block is *reachable* if a reference to it is on the stack, in memory, or in a field of a reachable block. A block that is not reachable can never be used again, and a machine **must** reclaim the space it takes. How much space there is, and how the machine finds and reclaims it, is up to the machine. `NEW` traps with *out of memory* only when the machine cannot find room for the new block, and a program that keeps only a small amount reachable must be able to make blocks forever.

This is how the specification of the Java Virtual Machine treats its heap too: it says that unreachable objects are reclaimed, and does not say how.

A *frame* is a pair (*return address*, *saved fp*).

The values on the stack at positions fp and above belong to the current call; they are its *locals*, and local k is the value at position fp + k. Positions below fp belong to callers and cannot be reached.

H.2.4 3. Running

The machine repeats the following until it halts or traps:

1. If pc is not an address in the code, trap: *ran off the end of the program.*
2. Let *start* = pc. Read the value at pc as the *opcode*, and add 1 to pc.
3. Carry out the instruction with that opcode, as given in section 5. If there is none, trap: *unknown instruction.*

When the machine halts, it exits with status 0.

H.2.5 4. Basic operations

The instructions in section 5 are described in terms of these operations. Each either succeeds or traps.

operand: if pc is not an address in the code, trap: *missing operand*. Otherwise the result is the value at pc, and pc increases by 1.

address: take an operand. If it is not an address in the code, trap: *jump out of program*. Otherwise it is the result.

push v: if the stack holds 256 values, trap: *stack overflow*. Otherwise append v.

pop: if the stack holds exactly fp values (the current call has no locals left), trap: *stack underflow*. Otherwise remove the top value; it is the result.

The steps of an instruction happen in the order given, and the first one to trap ends the instruction. Order matters: `JZ 9999` traps with *jump out of program* even when the stack is empty, because the address is taken before anything is popped.

H.2.6 5. Instructions

`a b` means: pop b, then pop a. So for `SUB`, a is the value that was under the top, and b the value that was on top.

For `ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `AND`, `OR`, `XOR` and `LT`, a and b must both be numbers; this is checked after both are popped, and before anything else.

Code	Name	Steps
0	HALT	Halt.
1	PUSH	n = operand; push n.
2	ADD	pop b, pop a; push a + b, wrapped.
3	SUB	pop b, pop a; push a − b, wrapped.
4	MUL	pop b, pop a; push a × b, wrapped.
5	PRINT	pop a, a number; write a in decimal, with a leading - if negative, then a newline, to standard output.
6	JMP	t = address; pc = t.
7	JZ	t = address; pop a; if a is the integer 0, pc = t. A reference never jumps.

Code	Name	Steps
8	DUP	pop a; push a; push a.
9	DROP	pop a.
10	SWAP	pop b, pop a; push b; push a.
11	OVER	pop b, pop a; push a; push b; push a.
12	ROT	pop c, pop b, pop a; push b; push c; push a.
13	LT	pop b, pop a; push 1 if a $<$ b, else 0.
14	EQ	pop b, pop a; push 1 if a and b are the same integer, or references to the same block; else 0. Never traps on references.
15	LOAD	k = operand; if k $<$ 0, or fp + k is not a position in the stack, trap: *no such local*; push local k.
16	STORE	k = operand; pop a; if k $<$ 0, or fp + k is not a position in the stack, trap: *no such local*; local k = a.
17	CALL	t = address; n = operand; if n $<$ 0, or the current call has fewer than n locals, trap: *not enough arguments*; if there are 256 frames, trap: *too many nested calls*; add the frame (pc, fp); fp = (size of stack) − n; pc = t.
18	RET	if there are no frames, trap: *return without call*; pop a; remove every value from position fp up; remove the last frame (r, f) and set pc = r, fp = f; push a.
19	DIV	pop b, pop a; if b = 0, trap: *division by zero*; push the quotient a ÷ b rounded towards zero, wrapped.
20	MOD	pop b, pop a; if b = 0, trap: *division by zero*; push a − b × q, where q is the quotient of `DIV`, wrapped.
21	AND	pop b, pop a; push the bitwise *and* of a and b, as 32-bit two's complement values.
22	OR	pop b, pop a; push the bitwise *or*.
23	XOR	pop b, pop a; push the bitwise *exclusive or*.
24	LOADM	pop a, a number; if a is not a memory address, trap: *no such memory address*; push the value at memory address a.
25	STOREM	pop v, pop a, a number; if a is not a memory address, trap: *no such memory address*; store v at memory address a.
26	EMIT	pop a, a number; if a $<$ 0 or a $>$ 255, trap: *not a character*; write the single byte a to standard output.

Code	Name	Steps
27	NEW	pop n, a number; if n $<$ 0, trap: *bad size*; if there is no room for a block of n fields, trap: *out of memory*; make a new block of n fields, all the integer 0, and push a reference to it.
28	GETF	pop i, pop r; r must be a block, then i a number; if i is not between 0 and the length of r minus 1, trap: *index out of bounds*; push field i of r.
29	SETF	pop v, pop i, pop r; r must be a block, then i a number; if i is out of bounds, trap: *index out of bounds*; field i of r = v.
30	LEN	pop r, a block; push its length.
31	CALLI	n = operand; pop t, a number; if t is not an address in the code, trap: *jump out of program*; then as `CALL t n`, from its check of n onwards.

Some consequences, all checked by the suite:

- `DIV` rounds towards zero: $-7 \div 2$ is -3, and -7 `MOD` 2 is -1. The remainder has the sign of a.
- $-2147483648 \div -1$ is 2147483648, which wraps to -2147483648; the remainder is 0.
- In `CALL t n`, the n values on top of the stack become locals 0 to n $-$ 1 of the called function, with the deepest as local 0.
- `RET` discards everything the call left on the stack, including its arguments, and gives the caller exactly one value.
- `STORE` pops before it checks, so `STORE 0` with a single value in the frame traps: after the pop, there is no local 0 left to store into.
- Memory belongs to no call. A value stored by one function can be loaded by any other, and survives every `RET`.
- `STOREM` takes the address from under the value: push the address first, then compute the value, then `STOREM`.
- `EMIT` writes a byte, not a character in any particular encoding. `PRINT` and `EMIT` write to the same standard output, in the order they run.

H.2.7 6. Traps

A trap stops the machine. It writes

```
trap: <reason> at <start>
```

to standard error, where *reason* is the text given in italics above and *start* is the address of the instruction that trapped. Anything already written to standard output stays written, and appears before the trap message when the two go to the same place. The machine exits with status 1.

H.2.8 7. Program files

A program is loaded from a file in the format of `ch05-files/FORMAT.md`. If the file cannot be opened, or breaks that format, the machine writes

```
<path>: <reason>
```

to standard error, where *path* is the file name as given and *reason* is one of: *cannot open*, *too short to be a program*, *not a program (wrong magic number)*, *unknown format version*, *bad code length*, *code ends early*, *extra bytes after the code*. It exits with status 2, without running anything.

The checks are made in the order of that list.

H.2.9 8. Tracing

An implementation may offer tracing with `-t`. If it does, then after each instruction that does not trap, including the final `HALT`, it writes one line to standard error:

- the step number, counting from 1, right-aligned in 6 characters,
- a space, then the instruction's start address, right-aligned in 5,
- two spaces, then the instruction: its name and its operands, separated by single spaces,

- then, if the stack is not empty or there is at least one frame: spaces to fill the instruction out to 14 characters (none if it is already longer), two more spaces, and the stack.

Memory and the heap are not shown. A reference on the stack is written `@` followed by the number of its block: `@1`, `@27`. The stack is written bottom first, values separated by single spaces, with a `|` before the first value of each call's locals. A call with no locals still gets its `|`, at the position where its locals would begin.

Before writing each line, the machine flushes standard output, so that a program's output and its trace interleave correctly.

H.2.10 9. What is left unspecified

- How the machine is started, and its messages for wrong usage.
- The `-d` debugger of `vm.c`.
- What happens if a value written to standard output cannot be written.

Index

activation record, 23, *see also* frame
ADD, **10**
ahead-of-time compilation, **137**
AND, **72**
Apollo Guidance Computer, 2
Apple II, 169
argument, **22**
assembler, **44**
associativity, **66**

basal/0, **75**
block, **90**
bounds checking, 90
branch prediction, 114
breakpoint, **53**
byte order, *see* little-endian
bytecode, *see* file format

CALL, **24**
CALLI, **107**
Cfront, 137
Cheney, C. J., 97
CHIP-8, **165**
Church, Alonzo, 108
clock cycle, 148
closure, **104**
    sharing variables, 106
closure conversion, **105**
coalescing, 121
code generation, **66**
compiler, **63**
computed goto, **114**
conformance suite, **59**
constant, 71
constant folding, **126**
constant pool, 179
copy propagation, **120**
copying collection, **97**
CPython, 114
Crafting Interpreters, 4
currying, **104**

data-flow analysis, **120**
DDT, 52
dead code, 121, 126
debugger, **52**
delta debugging, 136
desugaring, **77**
differential testing, **60**
Dijkstra, Edsger, 85
disassembler, **48**
dispatch, **113**
DIV, **32**
division
    rounding towards zero, 58

division by zero, 28
Dragon Book, 4
DROP, **17**
DUP, **17**
dynamic binding, 173

embedding, **142**
EMIT, **72**
emulator, 1, **2**
enkel/0, **70**
EQ, **17**
error messages
    of a parser, 84
exit status, 32, 39

file format, **36**, 187
first fit, 97
FORTH, 10, 16, 114
forwarding address, **97**
FPGA, 150
fragmentation, **97**
frame, **21**
frame pointer, **21**
free list, **97**
Free Pascal, 157
free variable, **105**
funarg problem, 157
function, 21
fuzzing, 60, 67

garbage collection, **95**
    copying, 97
    generational, 99
    mark and sweep, 96
    roots, 96
GC, *see* garbage collection
gdb, 52
generational collection, 99
GETF, **90**
global variable, **71**
GOSUB, 79
grammar, **64**

HALT, **10**
heap, **89**
HotSpot, 140

Icarus Verilog, 151
instruction set, **183**
    as data, 47
Intel 4004, 2
interpreter, 1, **2**

Java Virtual Machine, 2, **179**
    verifier, 131
JIT, *see* JIT compiler
JIT compiler, **140**
JMP, **17**
jump, **15**
    conditional, **15**
jump threading, 126
JVM, *see* Java Virtual Machine
JZ, **17**

known failure, **92**
Knuth, Donald, 4

label, **44**
labels as values, *see* computed goto
lambda/0, **104**
LEN, **90**
Lisp
    macros, 78
listing, of an assembler, **48**

little-endian, **36**
`LOAD`, **24**
`LOADM`, **72**
local variable, **22**, 71
loop, 15
    of the machine, 9
`LT`, **17**
Lua, 114, 120, 142
LuaJIT, 140

machine code, *see* native code
magic number, **36**
mark and sweep, **96**
McCarthy, John, 96
memory, **72**
Metcalfe parsing machine, **153**
Metcalfe, Howard, 153
MIX, 4
`MOD`, **32**
`MUL`, **10**

Nand2Tetris, 4
native code, **137**
`NEW`, **90**
Node.js, 145

optimiser, *see* peephole optimiser
`OR`, **72**
`OVER`, **17**
overflow, arithmetic, **58**

P4 machine, **157**
p-code, **157**
p-machine, 1, **157**
Parsec, 84
parser, **64**
    combinators, 82
    generated, 85
    Pratt, 82
    recursive descent, 82
parser combinators, **82**
Pascal, 157
pascal/0, **157**
peephole optimiser, **125**
PL/0, 157
playground, **145**
post-order, **66**
postfix, *see* Reverse Polish Notation
PostScript, **173**
Pratt parsing, **82**
Pratt, Vaughan, 82
precedence, **66**
`PRINT`, **10**
program counter, **9**
program file, *see* file format
`PUSH`, **10**
Python machine, 58

recursion, 23
recursive descent, **65**
reference, **90**
register machine, **119**
register window, **122**
replay, **54**
resolving, **71**
`RET`, **24**
return address, **21**
reverse debugging, *see* time-travel debugger
Reverse Polish Notation, 10
roots, **96**
`ROT`, **17**
run-time library, **79**
`rval`, **73**

s-expression, **83**
sanitizer, 135
scanner, **64**
SETF, **90**
shunting-yard algorithm, 85
SICP, 4
single step, **51**
specification, **57**
    version 4, 187
stack, **9**
    juggling, **16**
stack machine
    versus register machine, 122
stack overflow, **28**
stack underflow, **28**
static link, 74, **157**
STORE, **24**
STOREM, **72**
stress mode, **99**
SUB, **10**
superinstruction, **127**
SWAP, **17**
SWEET16, 2, **169**
syntax tree, **65**
syntax-directed translation, **77**
Sypytkowski, Bartosz, 5

tag, **90**
tail call, 109
testbench, **149**
threaded code, **114**
time-travel debugger, 54
Tiny BASIC, 75
token, **64**
trace, **53**
trap, **27**
two-pass assembler, **45**

UCSD Pascal, 1, 157
undefined behaviour, 57

V8, 140
verifier, **131**
Verilog, **147**
version number, 36
virtual machine, **9**

WebAssembly, **144**
Wirth, Niklaus, 4, 65, 157
Wozniak, Steve, 2, 169
wrapping arithmetic, **58**
write barrier, 101

XOR, **72**